

Chapter 6

Applied Machine Learning



Example 1—ML Trend Scanning with MLFinlab

Predicting	Price direction
Technology	Regression/classification
Asset Class	Crypto
Difficulty	Easy
Type	Research notebook
Source Code	qnt.co/book-example1

Summary

Applying the trend scanning package from the MLFinLab library to detect price trends (down-trend/no-trend/up-trend) to invest in the primary trend.

Motivation

Accurate trend prediction is essential for algorithmic trading strategies for these use cases:

1. Strategy selection—accurate trend identification helps select the most suitable trading strategy for the current market conditions, as different market phases benefit different strategies.
2. Optimal trade entry/exit—identifying the market trends enables traders to determine the most profitable moment for entering or exiting a trade.
3. Trend following—traders can ride market waves by accurately identifying and following trends.

4. Contrarian strategies—identifying overextended trends and positioning for expected reversals can help profit from market corrections.

Model

Model	A time series of the close prices of BTCUSD.
Features	
Predicted Label	-1, 0, 1 for downward trend, no trend, or upward trend.
Model	<code>trend_scanning_labels</code> method from the MLFinLab’s trend scanning packages.

We use the `trend_scanning_labels` method, a component of the MLFinLab’s trend_scanning package open-sourced for this book, introduced by Marcos Lopez de Prado (see gnt.co/book-trend-scanning).

```
def trend_scanning_labels(price_series: pd.Series, t_events: list = None, observation_window: int
```

The method is valuable for the classification of market regimes (down-trend, no-trend, and uptrend):

- It maps a vector of observations (in our case, close prices) to -1 for a downward trend, 0 for no trend, and 1 for an upward trend.
- It works by creating a set of linear regression models over different lookback periods:
 - The independent variable is `1, 1=0, ..., L-1` where `L` is the length of the observation window of choice – in our case, `L = 20`, that is, we’re looking for the trailing 20 data points to determine the trend.
 - The dependent variable is the closing price `1` steps into the future.
 - For each model, we calculate the t-value associated with the estimated regressor coefficient (beta) and select the model with the largest t-value in absolute value; it’s the most statistically significant model. In linear regression, the t-value determines the significance of the individual regression coefficient. The larger the value, the more significant the coefficient is.
 - The trend label is then the sign of the largest t-value.
- The method returns a data frame with several columns, the following most importantly:
 - `Index` —(pd.Timestamp) TimeIndex of the labels.
 - `t1` —(pd.Timestamp) Label end-time where the observed trend has the highest `t-value`, i.e., the end of a strong upward trend (for look forward) and the start of a solid downward trend (for look backward).
 - `t-value` —(float) Sign of `t-value` represents the direction of the trend. A positive `t-value` represents an upward trend.
 - `ret` —(float) Price change percentage from index time to label end time.

- `bin` — (int) Binary label value based on the sign of price change.

Let us illustrate with an example of analyzing a time series of closing prices with a chart (Figure 6.1) showing the closing prices time series in blue and the most significant t-values time series in red. The green markers indicate an upward trend (label 1), the yellow markers indicate no trend (label 0), and the red markers indicate a downward trend (label -1).

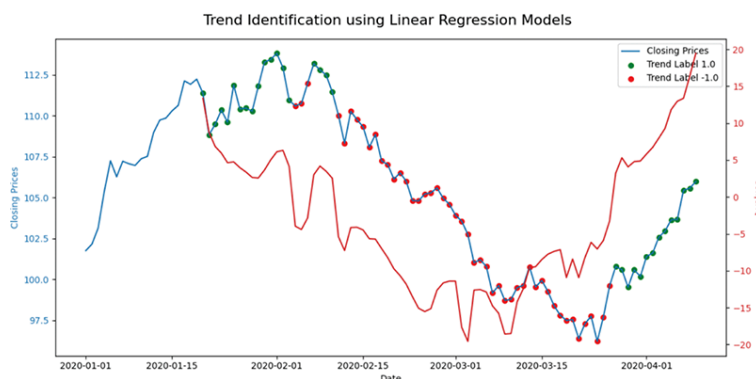


Figure 6.1 Example of trend identification using linear regression models.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from trend_scanning_labels import trend_scanning_labels

# Generate sample data for demonstration (closing prices)
np.random.seed(0)
dates = pd.date_range(start='2020-01-01', periods=100)
closes = pd.Series(np.random.randn(100).cumsum() + 100, index=dates)

# Parameters
L = 20

# Get trend labels using the trend_scanning_labels function
trend_labels_df = trend_scanning_labels(closes, closes.index, look_forward=False, observation_wind

# Extract t-values and trend labels
t_values = trend_labels_df['t_value'].dropna()
trend_labels = trend_labels_df['bin'].dropna()

# Plotting
fig, ax1 = plt.subplots(figsize=(12, 6))

# Plot closing prices
color = 'tab:blue'
ax1.set_xlabel('Date')
ax1.set_ylabel('Closing Prices', color=color)
ax1.plot(closes.index, closes, color=color, label='Closing Prices')
ax1.tick_params(axis='y', labelcolor=color)

# Create a second y-axis for the t-values
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('t-values', color=color)
ax2.plot(t_values.index, t_values, color=color, label='t-values')
ax2.tick_params(axis='y', labelcolor=color)
```

```
# Add trend labels
colors = {1: 'green', 0: 'yellow', -1: 'red'}
for i in range(len(trend_labels)):
    label = trend_labels.iloc[i]
    date = trend_labels.index[i]
    ax1.scatter(date, closes.loc[date], color=colors[label], s=30, label=f'Trend Label {label}')

# Handle legend for trend labels to avoid duplicates
handles, labels = ax1.get_legend_handles_labels()
unique_labels = dict(zip(labels, handles))
ax1.legend(unique_labels.values(), unique_labels.keys())

# Add title
fig.suptitle('Trend Identification using Linear Regression Models', fontsize=16)
fig.tight_layout()
```

Trading Universe

We consider trading only Bitcoin (BTCUSD).

Portfolio Construction

Model Training Time	Once a day.
Portfolio Rebalancing Time	Once a day.
Portfolio Weights	100% invested in Bitcoin whenever the trend is upward and 0% otherwise.

The portfolio weight of Bitcoin is

- 100% whenever the trend is upward, or
- 0% otherwise.

Trading Logic

While being simple, this strategy of holding 100% of the portfolio in BTCUSD whenever the trend is upward, and 100% in USD otherwise, does yield better results than the simple buy and hold bitcoin strategy ([Figure 6.2](#)):

```
from trend_scanning_labels import trend_scanning_labels
from backtestlib import rough_daily_backtest

trend = trend_scanning_labels(
    closes, closes.index, observation_window=20, look_forward=False,
    min_sample_length=5
)['bin'].dropna()
portfolio_weights = pd.DataFrame(trend.where(trend != -1, 0))
```



```
portfolio_weights.columns = [symbol]
rough_daily_backtest(qb, portfolio_weights)
```

Where `closes` are the close prices of BTCUSD stored in a data frame.

```
qb = QuantBook()
symbol = qb.add_crypto("BTCUSD", Resolution.DAILY).symbol
history = qb.history(symbol, datetime(2016, 1, 1), datetime(2024, 1, 1))
closes = history.loc[symbol]['close']
```



Figure 6.2 Equity curves of the strategy and selected benchmark.

Implementation Insights

We implement this strategy in QuantConnect's research notebook.

Let us assess the accuracy of the predictions, which is defined as the number of correct predictions divided by the total number of predictions.

```
accuracy = sum(correct_predictions) / len(correct_predictions)
```

To calculate the `correct_predictions`, we calculate the percentage change in the closing price between consecutive days, shift the result by one day back since we're predicting the *future* sign of change, align the result with the expected values, ignore the last value since it's a `NaN` due to the shift, convert the results to the values -1, 0, or 1 depending on the results' sign, and compare it with the sign of the predicted value.

```
correct_predictions = np.sign(
    closes.pct_change().shift(-1).loc[trend.index].iloc[:-1]
) == np.sign(trend.iloc[:-1])
```

For this setup, the accuracy is 51.2%, meaning the predictions are slightly better than random guesses. Despite this, the trading strategy is far more profitable than just holding the BTCUSD (Figure 6.3).



Figure 6.3 Predictions and observed log price.



Example 2—Factor Preprocessing

Techniques for Regime Detection

Predicting	Future return
Technology	Classification and PCA
Asset Class	US index
Difficulty	Easy
Type	Research notebook
Source Code	gnt.co/book-example2

Summary

This example illustrates the significant impact of different transformations on factor values before training an ML model.

Motivation

Raw financial markets data is often noisy, skewed, and can encompass a variety of distributions and scales, making it challenging to find the proper relationship between the variables. By applying appropriate pre-processing techniques, the machine-learning models are trained on a more suitable representation of the underlying data, which usually results in substantial improvements in the model’s predictive power.

Model

Model	Random factors.
Features	
Predicted Label	The label is one if the future weekly return for the SPY market index from open to open is positive; otherwise, the label is 0.
Model	Multiclass random forest model.

We explore the effects of various factor-preprocessing techniques on the model’s performance, with out-of-sample (OOS) accuracy as our benchmark. The OOS accuracy is the proportion of correctly classified instances in the test set.

The model we build is a multiclass random forest classification model predicting the probability of the input factors being associated with each of two classes:

- Class label 1 indicates that the weekly return from the market open to the next week's market open is positive.
- The class label is 0 otherwise.

We study the following factor preprocessing techniques:

- Raw factors.
- Turning factors into stationary factors.
- Standardization of the factors.
- PCA factorization.

Trading Universe

We consider the ETF SPY's daily data from January 1, 2000, to January 1, 2024.

Implementation Insights

Step 1: Build the label.

The label is calculated from the history data frame as follows (see also [Figure 6.4](#)):

```
label = history.loc[symbol]['open'].pct_change(5).shift(-5).dropna().apply(
    lambda x: int(x > 0)
)
```



[Figure 6.4](#) Scatter plot that colors each closing price based on the direction of future returns.

The label's distribution indicates that label one is more frequent in the dataset in [Figure 6.5](#).

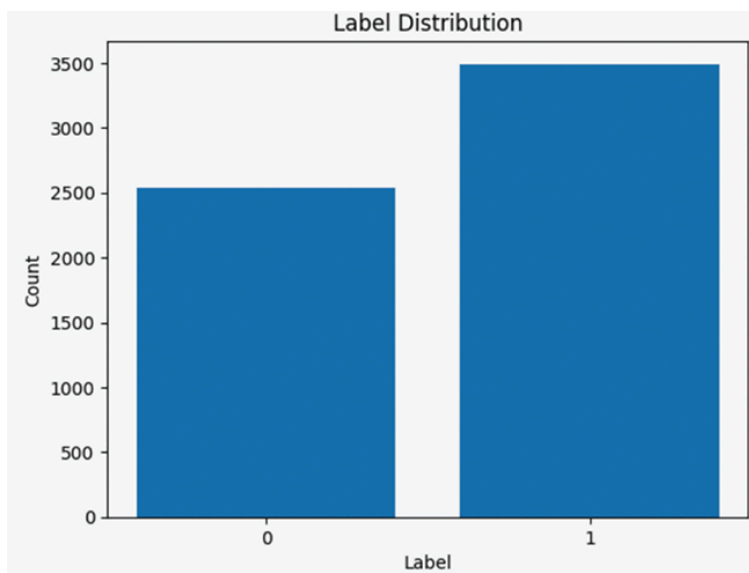


Figure 6.5 A bar chart showing the count of each label.

Step 2: Define the factors used to predict the label.

Next, we must define the factors to input into the model to predict the label.

For simplicity, we introduce random factors and make one of the factors non-stationary.

```
np.random.seed(2)
num_factors = 4
num_samples = len(label)
factors = np.random.rand(num_samples, num_factors)
# Make one of the factors non-stationary.
factors[:, -1] = factors[:, -1].cumsum()
```

Step 3: Define the performance benchmark for evaluating different factors.

The method `oss_accuracy` tests how well the given factors explain the given labels:

1. We create a train-test split of the factors and labels where we train the model on 75% of the data and test it on the remaining 25%.
2. We train the lightgbm multiclass classifier model (see [gnt.co/book-lightgbm](https://quantconnect.com/docs/quantconnect-book/lightgbm)) on the training dataset.
3. We evaluate how well the trained model performs on the test dataset and print the resulting OOS accuracy, defined as the proportion of correctly classified instances in the test set.

```
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def oos_accuracy(factors, label):
    X_train, X_test, y_train, y_test = train_test_split(
        factors, label, test_size=0.25, shuffle=False
```

```

)
model = lgb.train(
    {
        'seed': 1234,
        'verbose': -1,
        'boosting_type': 'rf',
        'feature_fraction': 0.8,
        'objective': 'multiclass',
        'num_class': 2,
        'bagging_freq': 5,
        'bagging_fraction': 0.8,
        'is_unbalanced': True
    },
    train_set=lgb.Dataset(
        data=X_train, label=y_train, free_raw_data=True
    ).construct()
)
predictions = model.predict(X_test)
x = list(range(len(predictions)))
go.Figure(
    [
        go.Scatter(x=x, y=predictions[:, 0], name=0),
        go.Scatter(x=x, y=predictions[:, 1], name=1)
    ],
    dict(
        title="Probability of Each Label<br><sup>Class 1 gets a greater"
            + "probability because the SPY has an upward bias</sup>",
        xaxis_title="Date", yaxis_title="Probability"
    )
).show()
y_hat = predictions.argmax(axis=1)
print(f"Accuracy: {round(accuracy_score(y_hat, y_test), 4)}")

```

Step 4: Evaluate the model's performance on the raw factors.

The first test is using just the raw factor values. The OOS accuracy is 0.6101 (Figure 6.6).

```
oss_accuracy(factors, label)
```

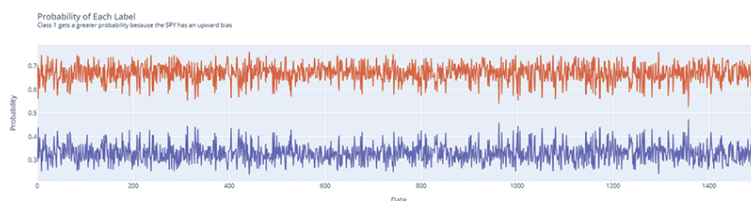


Figure 6.6 Time series plots that show the probability of each label from the model's predictions.

Step 5: Test model accuracy using stationary factors.

Lopez De Prado (2018) explains that “*supervised learning algorithms typically require stationary features.*”

Let us test if our features are stationary using the augmented Dickey-Fuller test for stationarity.

```
from statsmodels.tsa.stattools import adfuller
```

```
for factor_idx in range(num_factors):
    factor = factors[:, factor_idx]
    test_results = adfuller(factor, maxlag=1, regression='c', autolag=None)
    # Check the p-value.
    output = "Stationary" if test_results[1] <= 0.05 else "Not stationary"
    print(f"Factor {factor_idx}: {output}")
```

The outputs confirm all factors but one, in our case, is already stationary as we designed them to be.

```
Factor 0: Stationary
Factor 1: Stationary
Factor 2: Stationary
Factor 3: Not stationary
```

Step 6: Make factors stationary, if necessary.

If the raw factors aren't stationary, we can transform them to make them stationary.

In our case, using the fractional differentiated factors gives us an OOS accuracy of 0.5849 ([Figure 6.7](#)).

```
oss_accuracy(stationary_factors.values, label)
```

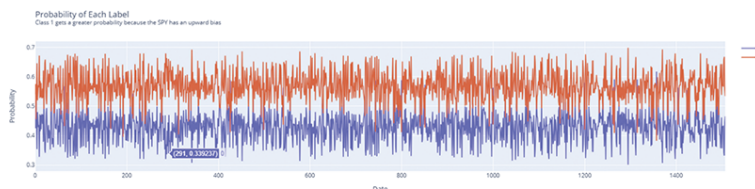


Figure 6.7 Time series plots that show the probability of each label from the model's predictions with stationary factors.

Step 7: Introduce standardization of factors.

Next, we test standardization, transforming the factor values into a mean of zero and a standard deviation of one. Standardized variables are necessary to assess their contribution to a model—using different units or scales would make the comparison difficult. Furthermore, standardizing the factors is an essential preprocessing step before applying PCA.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
standardized_factors = scaler.fit_transform(stationary_factors)
```

Using the standardized factors gives us an OOS accuracy of 0.5882 ([Figure 6.8](#)).

```
oss_accuracy(standardized_factors, label)
```

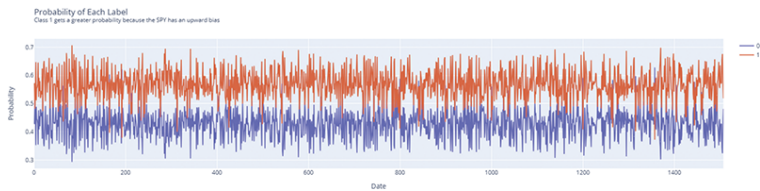


Figure 6.8 Time series plots that show the probability of each label from the model's predictions with standardized factors.

Step 8: Introduce PCA factorization.

Next, we test using PCA.

```
from sklearn.decomposition import PCA

pca = PCA(random_state=0)
principal_components = pca.fit_transform(standardized_factors[1:,:])
principal_components
```

Using the transformed data points in the reduced-dimensional space to train and test the ML model gives us an OOS accuracy of 0.5962 (Figure 6.9).

```
oss_accuracy(principal_components, label)
```

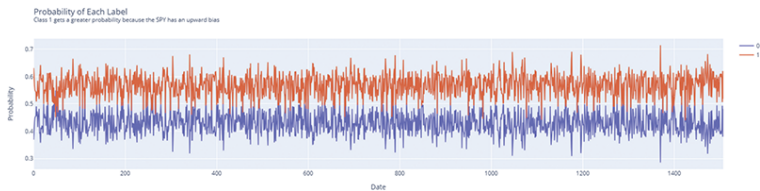


Figure 6.9 Time series plots that show the probability of each label from the model's predictions with the principal components.

Step 9: Compare the OSS accuracies.

In this particular use case, none of the factor preprocessing techniques lead to any significant improvement over raw data due to the factors being random numbers. Choosing the best factor preprocessing technique to deliver the best OSS is naturally case-specific.

Factor Preprocessing Technique	OSS Accuracy
Raw Data	0.6101
Stationary Factors	0.5849
Standardized Factors	0.5882
PCA Factors	0.5962



Example 3—Reversion vs. Trending: Strategy Selection by Classification

Predicting	Volatility
Technology	Neural networks
Asset Class	US equities
Difficulty	Easy
Type	Research notebook
Source Code	gnt.co/book-example3

Summary

We are applying neural networks to forecast whether the next trading day will favor momentum or reversion risk exposure by categorizing periods of reversion vs. trending, and then using different strategies (we switch between holding the SPY index and TLT bonds) based on the current volatility regime (using volatility, VIX, ATR, RSI).

Motivation

Momentum strategies capitalize on the tendency of stock prices to continue moving in the same direction for a certain period. In the context of this strategy, holding the SPY index tracking the S&P 500 allows the traders to gain exposure to broader market momentum. When the market is trending, the SPY index is expected to continue in its direction.

On the other hand, the reversion risk strategies, sometimes called mean reversion strategies, are based on the belief that the asset prices will revert to their historical mean levels after deviating from them for a certain period. TLT, which tracks the long-term US Treasury bonds, is often used in a reversion risk strategy since bonds tend to perform well during market reversion. When the markets show the first sign of reversion, holding TLT bonds provides a reasonable hedge against market declines.

In highly dynamic financial markets, the ability to forecast whether the next trading day will favor momentum or reversion risk exposure allows investors to adjust their portfolios dynamically. This strategy ensures that investors are better positioned to capitalize on momentum gains during trends and mitigate losses during reversion periods.

Model

Model	
Features	• Relative strength index. • Average true range. • The standard deviation of daily returns. • Value of the VIX index.
Predicted Label	Label 1 = Momentum regime; Label 0 = reversion regime.
Model	Sequential neural network.

We create the label of 1 when the next day favors the momentum factor exposure and 0 otherwise by using the momentum and short-term reversal factor time series from Kenneth R. French's website (see qnt.co/book-french-data-library).

```
# Label 1 = Momentum regime; Label 0 = Reversion regime.
label = (momentum > reversion.reindex(momentum.index)).dropna().astype(int)
# The label is the regime of the following day, not day that just passed
label = label.shift(-1).dropna()
```

We introduce a matrix of these factors/indicators as a proxy for the current volatility regime:

1. Relative strength index (RSI) of the SPY over the last 21 days (a month).
2. Average true range (ATR) of the SPY over the previous 21 days (a month), normalized by 21-day SMA of price.
3. The standard deviation of daily returns of SPY over the last 21 days (a month).
4. Chicago Board Options Exchange's (CBOE's) Volatility Index (VIX index).

Before inputting the factors into the neural network training, we standardize them and split them into the training and test datasets.

We create a simple **sequential neural network** with two dense layers using the Keras library:

- The network has four inputs, one for each factor.
- The first layer has eight neurons/nodes.
- The outer layer has one neuron and a sigmoid activation function, which returns a value between 0 and 1, representing the probability of the positive class (momentum-friendly regime).

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from keras.utils import set_random_seed
from sklearn.metrics import accuracy_score
```

```
set_random_seed(0)

model = keras.Sequential(
    [
        layers.Input(shape=(X.shape[1],)),
        layers.Dense(8, activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ]
)
```

We use the adam optimizer and binary_crossentropy as the loss function.

```
model.compile(
    optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']
)
```

The optimizer minimizes the value of the loss function by updating the neural network (NN) weights. The Adam optimizer (an extension to stochastic gradient descent) is an adaptive learning rate optimization algorithm used widely due to its efficiency and low memory requirements.

The loss function evaluates how well the neural network fits the dataset—the further the predictions are from the actual results, the higher its value is. The goal of the training is to make the predictions as accurate as possible, that is, to minimize the loss function's value.

The loss function `binary_crossentropy` is a loss function used in binary classification problems.

Metrics are used to evaluate the model's performance; for classification problems, the most common metric is accuracy.

We train the model across 100 epochs using just the training dataset.

```
model.fit(X_train, y_train, epochs=100, verbose=0)
```

The epochs parameter specifies the number of times the network is trained on the training dataset. One hundred epochs is a reasonable number of iterations for this minor model, chosen by experience.

We predict the model's accuracy using the testing dataset (the data the model has not seen yet).

```
y_hat = (model.predict(X_train, verbose=0) > 0.5).astype(int)
print(f"In-sample accuracy: {accuracy_score(y_hat, y_train)}")

y_hat = (model.predict(X_test, verbose=0) > 0.5).astype(int)
print(f"Out-of-sample accuracy: {accuracy_score(y_hat, y_test)}")
print(f"OOS Label counts: {np.unique(y_hat, return_counts=True)[1]}")
```

With the results

```
In-sample accuracy: 0.554843141744905
Out-of-sample accuracy: 0.5276615527661552
OOS label counts: [ 592 1559]
```

We chose a benchmark for this strategy to the percentage of up-days of the SPY ETF, which over a 10-year period 2014–2024 had 54.4% up-days. The model results showed an in-sample accuracy of 55.48%—better than the underlying benchmark or random-guessing (with an accuracy of 50%). The out-of-sample accuracy (tested on not data not seen by the model) of 52.77% is lower than in-sample accuracy, which is usual and confirms that the model is only slightly better than random guesses, and thus suggests better input data may be required, along with enhancing the NN with more layers and/or factors and other tweaks.

Trading Universe

We consider only two assets, either SPY or TLT.

Portfolio Construction

Model Training Time	Once before we start backtesting.
Portfolio Rebalancing Time	Once a day at the market open.
Portfolio Weights	100% in SPY, if the model predicts the current regime favors momentum, and otherwise 100% in TLT.

The strategy rules are as follows:

- If the model predicts the current regime favors momentum over reversion, long SPY. Otherwise, long TLT.
- Rebalance at market open.

Trading Logic

The portfolio is rebalanced daily at market open—we predict if today’s market is likely to benefit momentum (i.e., predictions == 1), and then set the portfolio weights as such.

```
portfolio_weights = pd.DataFrame({
    spy_symbol: predictions,
    qb.add_equity("TLT", Resolution.DAILY).symbol: abs(predictions-1)
})
```

Implementation Insights

The RSI and ATR features are implemented using the standard QuantConnect Indicators. For the STD factor, we set up a custom indicator.

```
spy_symbol = qb.add_equity("SPY", Resolution.DAILY).symbol

# Define the parameters.
period = 21
start_date = datetime(1990, 1, 1)
end_date = datetime(2024, 1, 1)

# Get the RSI indicator data.
rsi = qb.Indicator(
    RelativeStrengthIndex(period), spy_symbol, start_date, end_date
)['relativestrengthindex']

# Get the ATR indicator (normalized by SMA of price) data.
atr = qb.Indicator(
    AverageTrueRange(period, MovingAverageType.SIMPLE),
    spy_symbol, start_date, end_date
)['averagetrueaverage']
atr /= qb.Indicator(
    SimpleMovingAverage(period), spy_symbol, start_date, end_date
)['simplemovingaverage']

# Get the STD indicator (STD of daily returns) data.
roc = RateOfChange(1)
std = IndicatorExtensions.of(StandardDeviation(period), roc)
history = qb.history[TradeBar](spy_symbol, start_date, end_date)
window = {column: [] for column in ['time', 'std_of_roc']}
def update_window(sender, updated):
    if not sender.is_ready:
        return
    window['time'].append(updated.end_time)
    window["std_of_roc"].append(updated.value)
std.updated += update_window
for bar in history:
    roc.update(bar.end_time, bar.close)
std = pd.DataFrame(window).set_index('time')['std_of_roc']

# Get the VIX factor data.
vix_symbol = qb.add_data(CBOE, "VIX", Resolution.DAILY).symbol
vix = qb.history(vix_symbol, start_date, end_date).loc[vix_symbol]['close']
vix.name = "vix"

# Combine factors into a matrix
factors = pd.concat([rsi, atr, std, vix], axis=1).dropna()
```



Example 4—Alpha by Hidden Markov

Models

Predicting	Volatility regime
Technology	Hidden Markov models (HMM) clustering
Asset Class	US equities, equity options, and index options
Difficulty	Easy-medium
Type	Full strategy
Source Code	qnt.co/book-example4

Summary

We use a Markov regression model to predict the market volatility regime (a low-volatility or a high-volatility environment) and allocate our funds accordingly. We offer three different portfolio allocation algorithms; we start with investing in two ETFs, SPY/TLT, then expand to invest in an SPY straddle option, and finally, we use an index option.

Motivation

Volatility, often defined as the rate and magnitude of price changes in a market, directly influences the risk and return profiles of assets.

Anticipating volatility regimes is a critical part of any effective trading strategy.

Volatility regimes can be broadly categorized into low- and high-volatility environments:

- In a **low-volatility regime**, market prices tend to be more stable with smaller, more predictable price movements and bullish trends, where growth-oriented assets like equities perform well.
- Conversely, a **high-volatility regime** is characterized by more significant, erratic price swings, often indicative of market uncertainty or bearish trends. During such periods, defensive assets like bonds or strategies that benefit from volatility, such as options trading, become more attractive.

Predicting these regimes allows traders to adjust their asset allocation dynamically ([Figure 6.10](#)).

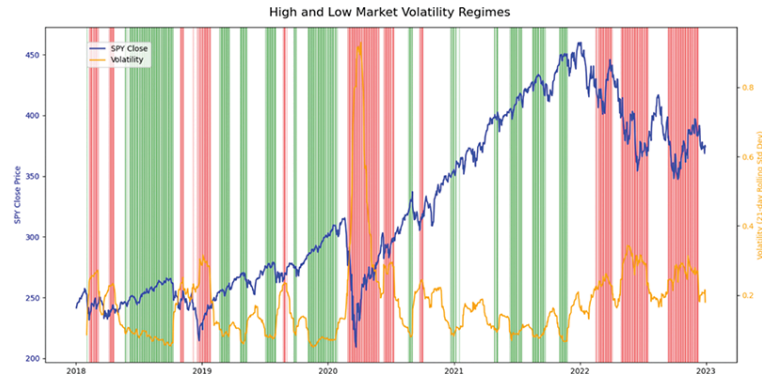


Figure 6.10 The price and volatility of SPY, along with highlighted zones of low/high volatility regimes.

This visualization of SPY close price and volatility (the annualized, rolling 21-day standard deviation of daily SPY close price returns) helps understand how market volatility fluctuates. It highlights high (periods where volatility exceeds the 75th percentile threshold [red shading]) and low (periods where volatility is below the 25th percentile threshold [green shading]) volatility.

With the **Markov regression model**, we can accurately predict high and low market volatility environments and allocate our funds strategically to optimize returns and manage risk.

Model

Model	Daily returns of SPY.
Features	
Predicted Label	The current market regime: label 0 for a low volatility one and label 1 for the high volatility.
Model	Markov-switching regression.

We implement a Markov-switching regression model (qnt.co/book-markov-regression) to detect two distinct volatility regimes:

- Low-volatility regime.
- High-volatility regime.

To train the model, we use the trailing 3 years of daily returns of the S&P 500 market index.

Trading Universe

In this example, we implement three distinct portfolio allocation algorithms.

We begin with a straightforward investment in two ETFs: SPY, representing the S&P 500, for capitalizing on market gains, and TLT, representing long-term US Treasury bonds, for stability during market downturns.

We then incorporate an SPY straddle option, which profits from increased volatility regardless of market direction, adding potential profitability during turbulent times (Figure 6.11).

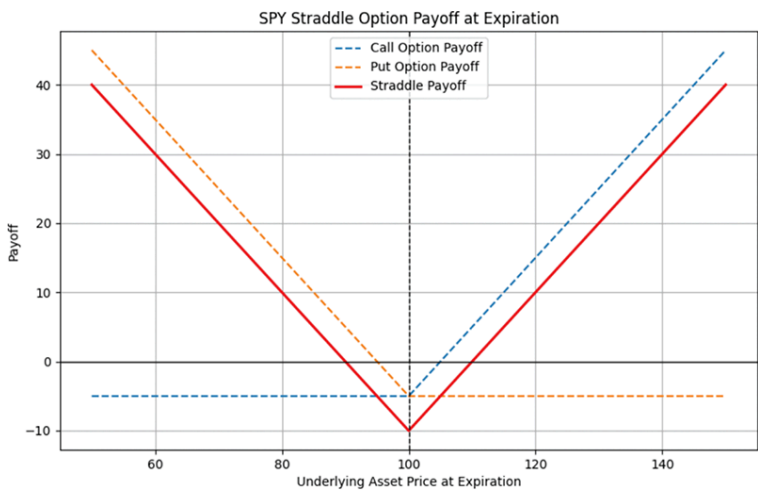


Figure 6.11 Payoffs of a call option, put option, and an option straddle

The Straddle options strategy involves buying both a call option and a put option with the same strike price and expiration date. The combined payoff of the straddle strategy in red shows profitability when the underlying asset price moves significantly away from the strike price in either direction, benefiting from increased volatility. This combined payoff curve is a combination of the following:

- **Call Option Payoff (dashed line):** The payoff for the call option increases as the underlying asset price rises above the strike price minus the premium paid.
- **Put Option Payoff (dashed line):** The payoff for the put option increases as the underlying asset price falls below the strike price minus the premium paid.

Finally, we integrate an index option, leveraging its ability to provide amplified returns. Instead of using SPY as the underlying asset, this algorithm uses the SPX index, a more accurate representation of the S&P 500.

Portfolio Construction

Model Training Time	Every day, 1 minute after the market opens.
Portfolio Rebalancing Time	After the model training.
Portfolio Weights	See following description.

We provide three different trading strategies, each exploiting the identified volatility regimes with varying instruments of the market:

	A low-volatility environment	A high-volatility environment
Algorithm 1	We allocate 100% of the portfolio to a risk-on investment: the SPDR S&P 500 ETF Trust, SPY.	We allocate 100% of the portfolio to a risk-off investment: the iShares 20 Plus Year Treasury Bond ETF, TLT.
Algorithm 2	We short one SPY option straddle.	We long one SPY option straddle.
Algorithm 3	We short one SPX Index option straddle.	We long one SPX Index option straddle.

Trading Logic

We train the model at the beginning of each day, and the algorithm only rebalances when the regime switches low → high or high → low volatility; we aim at anticipating market downturns and remain invested during periods of market growth.

The trading logic is based on the predicted regime value.

```
# Create the markov model.
# `k_regimes` defines the number of regimes. In this case,
# we want 2 regimes (high and low volatility).
# `switching_variance` defines whether or not each regime
# can have its own variance.
model = MarkovRegression(
    self._daily_returns, k_regimes=2, switching_variance=True
)

# Get the current market regime (0 => low volatility; 1 => high
# volatility).
regime = model.fit().smoothed_marginal_probabilities.values\
    .argmax(axis=1)[-1]
```

Tearsheet

Algorithm 1

The results show the following:

- A lookback period of 4 years achieves the most significant Sharpe ratio.
- The Sharpe ratio is susceptible to changes in the lookback period.
- A lookback period of 1 year leads to a negative Sharpe ratio.
- See [Figures 6.12](#) and [6.13](#).

Backtest parameters:

- `lookback_years` : 3

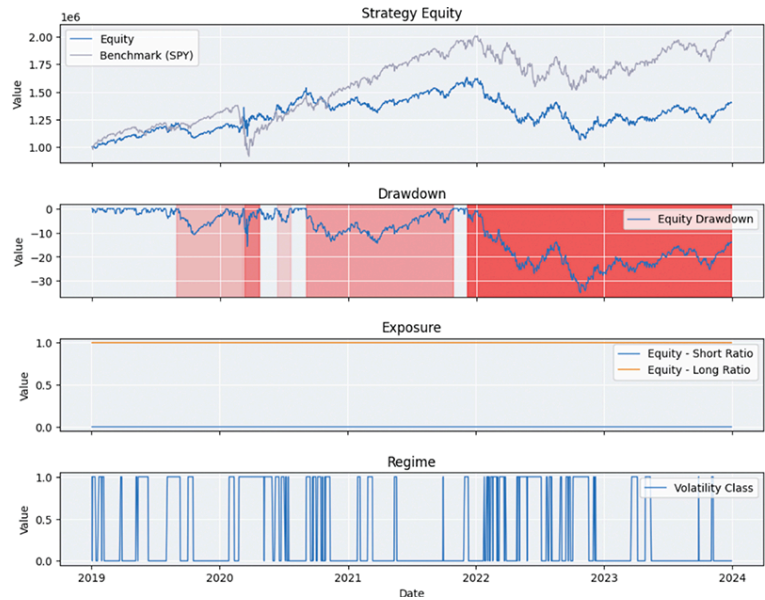


Figure 6.12 Equity curve, performance plots, and custom plots for Example 4.1.

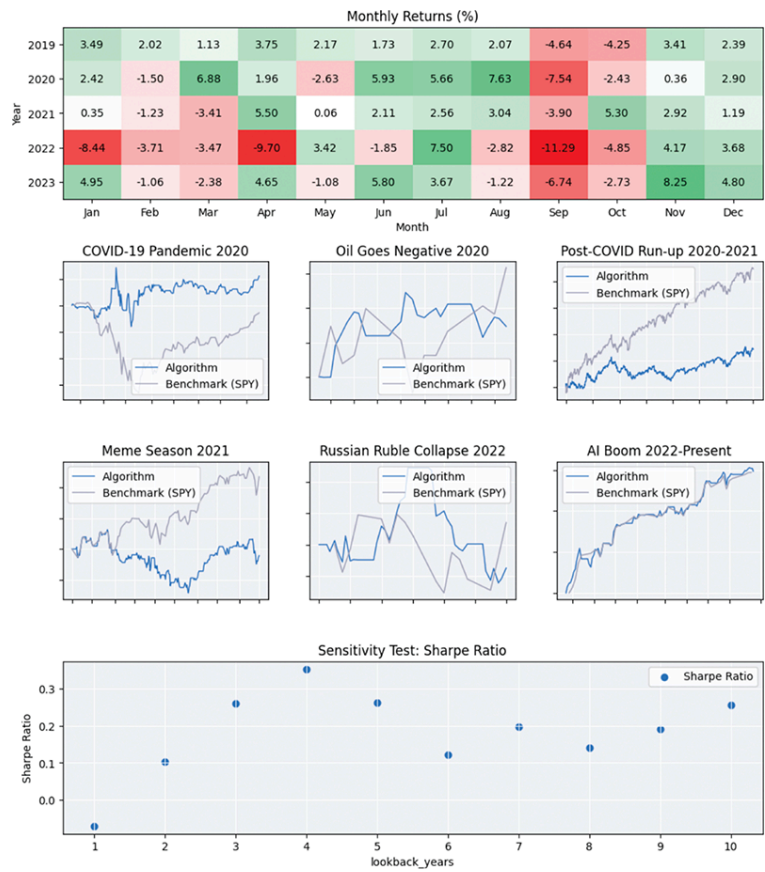


Figure 6.13 Monthly returns, crisis events, and sensitivity tests for Example 4.1.

Algorithm 2

See [Figures 6.14](#) and [6.15](#).

Backtest parameters:

- `lookback_years` : 3
- `min_expiry` : 90
- `max_expiry` : 365
- `min_hold_period` : 7

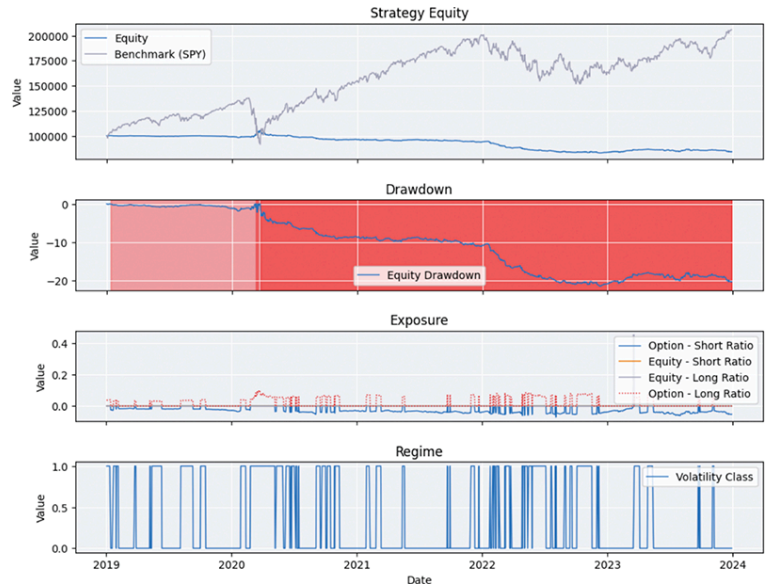


Figure 6.14 Equity curve, performance plots, and custom plots for Example 4.2.

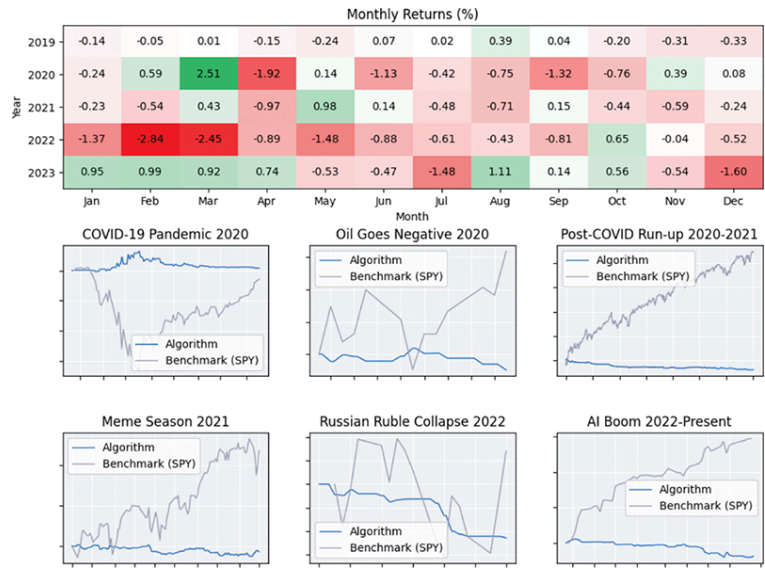


Figure 6.15 Monthly returns and crisis events for Example 4.2.

Algorithm 3

See [Figures 6.16](#) and [6.17](#).

Backtest parameters:

- `lookback_years : 3`
- `min_expiry : 0`
- `max_expiry : 180`
- `min_hold_period : 0`

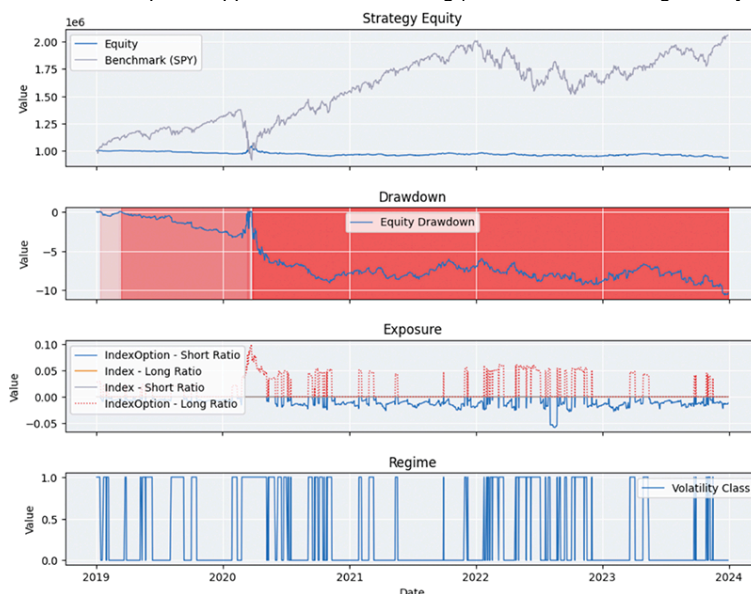


Figure 6.16 Equity curve, performance plots, and custom plots for Example 4.3.

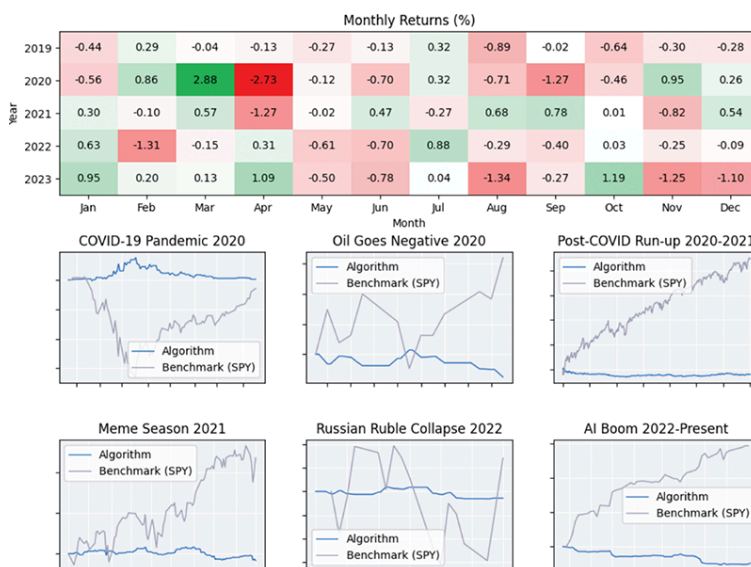


Figure 6.17 Monthly returns and crisis events for Example 4.3.

Implementation Insights

Algorithm 1

When the model detects a high volatility regime, the algorithm allocates 100% of the portfolio to TLT. When the model detects a low volatility regime, the algorithm allocates 100% of the portfolio to SPY. We retrain the model every day, 1 minute after the market opens. If the identified volatility regime changes, we rebalance the portfolio.

```
def _trade(self):
    # Create the markov model.
    # `k_regimes` defines the number of regimes. In this case,
    # we want 2 regimes (high and low volatility).
    # `switching_variance` defines whether or not each regime
    # can have its own variance.
    model = MarkovRegression(
        self._daily_returns, k_regimes=2, switching_variance=True
    )
```

```

# Get the current market regime (0 => low volatility; 1 => high
# volatility).
regime = model.fit().smoothed_marginal_probabilities.values\
    .argmax(axis=1)[-1]
self.plot('Regime', 'Volatility Class', regime)

# Rebalance when the regime changes.
if regime != self._previous_regime:
    self.set_holdings(
        [
            PortfolioTarget(self._tlt, regime),
            PortfolioTarget(self._spy, int(not regime))
        ]
    )
self._previous_regime = regime

```

Algorithm 2

This algorithm extends the previous algorithm; however, instead of rotating between SPY and TLT, we open straddles with SPY option contracts.

The rebalance logic changes as follows:

- When in a low volatility regime, we open a short straddle.
- When in a high volatility regime, we open a long straddle.

We pick the contracts with the closest expiration and are at the money to form the straddles. Since we are trading American-style options and selling a contract when we open a straddle, the buyer can exercise the option before it expires. In this case, we liquidate the remaining leg of the straddle and open a new straddle at the next market opening.

When we trade equity options, we need to adjust our data structures storing options trade information each time a split occurs.

The trading algorithm is just replacing resulting action based upon the predicted volatility regime change. The position sizing is not performed to keep the algorithm simple.

```

def _trade(self):
    # Create the markov model.
    # `k_regimes` defines the number of regimes. In this case,
    # we want 2 regimes (high and low volatility).
    # `switching_variance` defines whether or not each regime
    # can have its own variance.
    model = MarkovRegression(
        self._daily_returns, k_regimes=2, switching_variance=True
    )

    # Get the current market regime (0 => low volatility; 1 => high
    # volatility).
    regime = model.fit().smoothed_marginal_probabilities.values\
        .argmax(axis=1)[-1]
    self.plot('Regime', 'Volatility Class', regime)

    # Rebalance when the regime changes or when we aren't invested.
    if regime != self._previous_regime or not self.portfolio.invested:

```

```

# Close the previous straddle if there is one.
for symbol in self._equity.hedge_contracts:
    self.liquidate(symbol)
self._equity.hedge_contracts = []

# Get the option chain.
option_chains = self.current_slice.option_chains
if self._option_symbol not in option_chains:
    return
chain = option_chains[self._option_symbol]
min_expiry_date = self.time + self._min_expiry + self._min_hold_period
expiries = [
    contract.expiry
    for contract in chain
    if contract.expiry >= min_expiry_date
]

# Define the straddle type. Low volatility regime => short
# straddle; High volatility regime => long straddle.
if regime == 0:
    option_type = OptionStrategies.short_straddle
    expiry = min(expiries)
else:
    option_type = OptionStrategies.straddle
    expiry = max(expiries)

# Get the ATM strike price.
strike = sorted(
    [contract for contract in chain if contract.expiry == expiry],
    key=lambda contract: abs(
        chain.underlying.price - contract.strike
    )
)[0].strike

# Create the strategy. Low volatility regime => short
# straddle; High volatility regime => long straddle.
option_strategy = option_type(
    self._option_symbol, strike, expiry
)

# Open the straddle trade.
tickets = self.buy(option_strategy, 1)

# Save a reference to each of the contracts.
self._equity.hedge_contracts = [t.symbol for t in tickets]
self._previous_regime = regime

def on_order_event(self, order_event):
    # When one of the straddle legs are assigned/exercised
    # and we now have a position in the underlying Equity,
    # liquidate everything and remove the hedges.
    if (order_event.status == OrderStatus.FILLED and
        self._equity.invested and
        self._equity.hedge_contracts):
        self.liquidate(self._equity.symbol)
        for symbol in self._equity.hedge_contracts:
            self.liquidate(symbol)
        self._equity.hedge_contracts = []

```

Algorithm 3

This algorithm is an extension of the previous one:

- Instead of using equity options, we use a European-style index options to prevent early exercise of the contracts.
- Instead of using SPY as the underlying asset, this algorithm uses the SPX index, a more accurate representation of the S&P 500.

The algorithm code remains the same, except the buyer of the contracts we sell can only exercise the contract on the expiry day. Thus, we remove the logic that liquidates the remaining legs in the straddle.

```
def _trade(self):
    # Create the markov model.
    # `k_regimes` defines the number of regimes. In this case,
    # we want 2 regimes (high and low volatility).
    # `switching_variance` defines whether or not each regime
    # can have its own variance.
    model = MarkovRegression(
        self._daily_returns, k_regimes=2, switching_variance=True
    )

    # Get the current market regime (0 => low volatility; 1 => high
    # volatility).
    regime = model.fit().smoothed_marginal_probabilities.values\
        .argmax(axis=1)[-1]
    self.plot('Regime', 'Volatility Class', regime)

    # Rebalance when the regime changes, when we aren't invested,
    # or when we need to rollover the contracts.
    if (regime != self._previous_regime or
        not self.portfolio.invested or
        self._expiry - self.time < self._min_expiry):
        # Close the previous straddle if there is one.
        self.liquidate()

    # Get the option chain.
    option_chains = self.current_slice.option_chains
    if self._option_symbol not in option_chains:
        return
    chain = option_chains[self._option_symbol]
    min_expiry_date = self.time + self._min_expiry + self._min_hold_period
    expiries = [
        contract.expiry
        for contract in chain
        if contract.expiry >= min_expiry_date
    ]
    if not expiries:
        return

    # Define the straddle type. Low volatility regime => short
    # straddle; High volatility regime => long straddle.
    if regime == 0:
        option_type = OptionStrategies.short_straddle
        expiry = min(expiries)
    else:
        option_type = OptionStrategies.straddle
        expiry = max(expiries)
```

```

# Get the ATM strike price.
strike = sorted(
    [contract for contract in chain if contract.expiry == expiry],
    key=lambda contract: abs(
        chain.underlying.price - contract.strike
    )
)[0].strike

# Create the strategy. Low volatility regime => short
# straddle; High volatility regime => long straddle.
option_strategy = option_type(
    self._option_symbol, strike, expiry
)

# Open the straddle trade.
tickets = self.buy(option_strategy, 1)

self._index.hedge_contracts = [t.symbol for t in tickets]
self._expiry = expiry
self._previous_regime = regime

def on_order_event(self, order_event):
    # When one of the straddle legs are assigned/exercised,
    # the other leg is automatically liquidated so let's
    # remove the reference to each leg.
    if (order_event.status == OrderStatus.FILLED and
        order_event.is_assignment):
        self._index.hedge_contracts = []
        self._expiry = datetime.min

```



Example 5—FX SVM Wavelet Forecasting

Predicting	Price
Technology	Support vector machines regression
Asset Class	Forex
Difficulty	Medium
Type	Full strategy
Source Code	gnt.co/book-example5

Summary

The algorithm uses support vector machines (SVMs) and wavelets to predict the future price of forex pairs. It first breaks down the past closing prices of each pair into components using wavelet decomposition. Then, it uses the SVM to make a one-step-ahead forecast for each element. Finally, it combines the components to get the overall estimates of the SVM-wavelet model.

Motivation

Various factors influence forex markets, including economic indicators, market sentiment, and geopolitical events with multiple frequencies and patterns. In addition, forex markets are inherently non-stationary (their statistical properties change over time), and forex data is very noisy, making it challenging to identify the underlying patterns.

Wavelet decomposition is particularly valuable in this context since it naturally captures complex dynamics (it decomposes dynamics into individual components), enhances predictive accuracy by isolating the significant time series components, ignoring the noise, and quickly adapts to market changes by capturing both short-term fluctuations and long-term trends.

SVM has been chosen to model the wavelet-decomposed components because of its robustness in handling non-linear relationships. SVM produces forecasts for each decomposed component, which are then used to reconstruct the predicted components.

Model

Model Features	Closing FX prices.
Predicted Label	Forecasted FX price one day ahead.
Model	SVM runs on decomposed wavelets.

We implement a class `SVMWavelet` with only one public method, `forecast` :

- 1. We use the `pywt` Python package to work with wavelets.

```
import pywt
```

- 2. Wavelet choice: To break down the time series into different frequency components, we use the Symlet 10 wavelets ("`sym10`"), a family of wavelets used for their smoothing properties.

```
w = pywt.Wavelet('sym10')
```

- 3. To determine the size of the input vector, we use the formula and solve for `len(data)`:
$$\log_2 \left(\frac{\text{len data}}{\text{wave length} - 1} \right) = \text{levels}$$
where
 - 1. We intend to decompose the time series into the three levels of components.
 - 2. The length of a Symlet 10 wavelet is 20, and we focus on implementing three levels of components.Which results in `len(data) == 152`.

4. We set a threshold scaling factor for denoising the wavelet coefficients to 0.5. The factor itself can be any number between 0 and 1.

```
threshold = 0.5
```

5. We decompose the input array into several arrays of coefficients using the chosen wavelet.

```
coeffs = pywt.wavedec(data, w)
```

6. We denoise all the coefficients except the first approximation ones, apply the SVR to predict the net value of the current coefficients, and store the forecasted value at the end of the coefficients array.

```
for i in range(len(coeffs)):
    if i > 0:
        # Don't threshold the approximation coefficients.
        coeffs[i] = pywt.threshold(coeffs[i], threshold*max(coeffs[i]))
        forecasted = self._svm_forecast(coeffs[i])
        coeffs[i] = np.roll(coeffs[i], -1)
        coeffs[i][-1] = forecasted
```

7. Finally, we reconstruct the data from the updated wavelet coefficients using the inverse wavelet transformation and return the last value, the forecasted future value.

```
datarec = pywt.waverec(coeffs, w)
return datarec[-1]
```

Trading Universe

We adopt EURUSD as the benchmark. We trade these currency pairs: EUR/JPY, GBP/USD, AUD/CAD, and NZD/CHF.

Portfolio Construction

Model Training Time	Upon receipt of new daily FX data.
Portfolio Rebalancing Time	After model training.
Portfolio Weights	See following description.

We set the weights as follows:

- The ideal weight is the expected return forecasted FX value over yesterday's close as long as it's above `self._weight_threshold` with its default value of 0.005.
- We always hold the weight times the `self._leverage` with its default value of 20.

```
prices = np.array(list(security.window))[:-1]
forecasted_value = self._wavelet.forecast(prices)
weight = (forecasted_value / bar.close) - 1
if abs(weight) > self._weight_threshold:
    self.set_holdings(security.symbol, weight * self._leverage)
```

Trading Logic

We place orders in this algorithm in the consolidation handler, which receives daily data and predicts the next close value. If the potential return from investing today (i.e., forecast value/current price - 1) is more significant than a specified threshold, we invest in the currency pair under defined leverage.

Tearsheet

Parameter `period`:

- The minimum is 63 because it's 3 months of trading days.
- The maximum is 189 because it's 9 months of trading days.
- The step size is 21 because it's 1 month of trading days.

The results show the following:

- The Sharpe ratio is maximized with a `period` of 8 months, but results are volatile between each consecutive period (high sensitivity).
- The Sharpe ratio is typically greater with `weight_threshold > 0.004` rather than `<= 0.004`.
- See [Figures 6.18](#) and [6.19](#).

Backtest parameters:

- `period`: 152.
- `leverage`: 20.
- `weight_threshold`: 0.005.

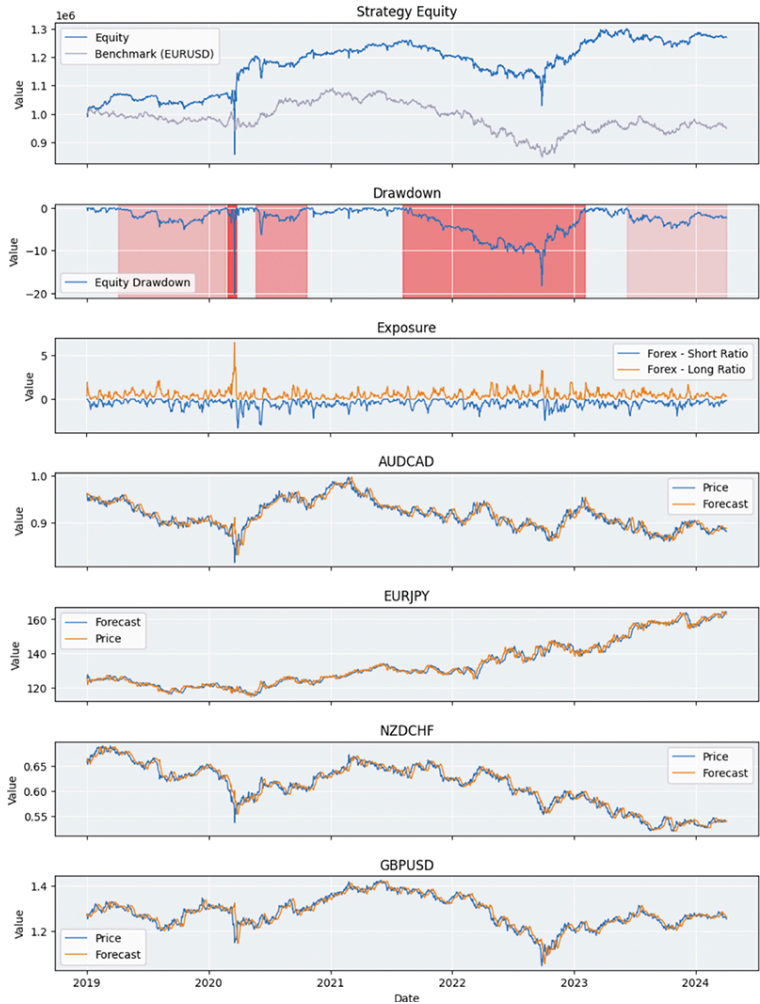
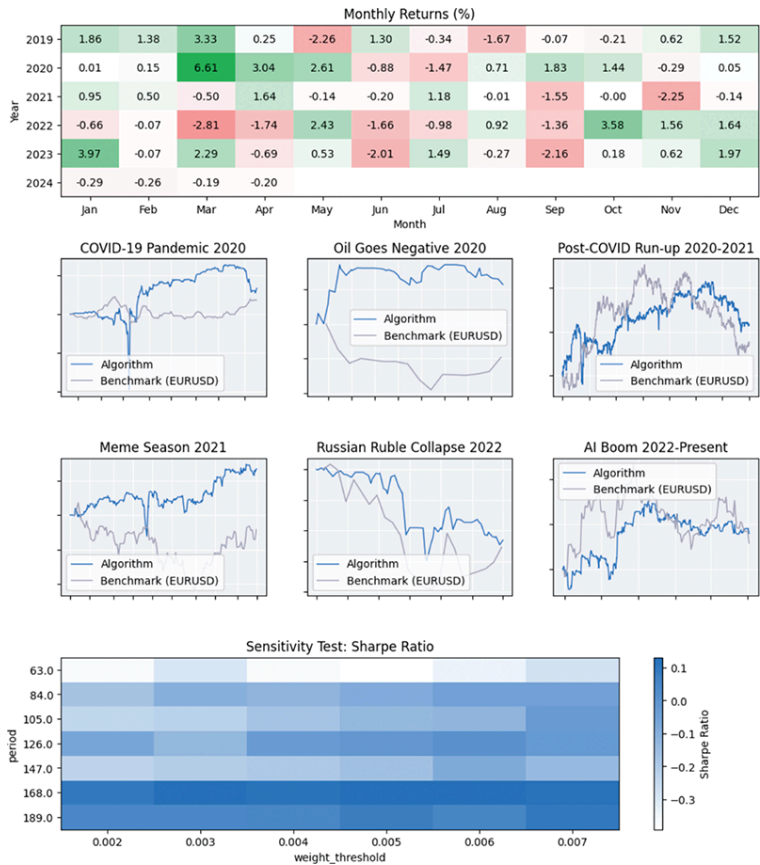


Figure 6.18 Equity curve, performance plots, and custom plots for Example 5.



[Figure 6.19](#) Monthly returns, crisis events, and sensitivity tests for [Example 5](#).

Implementation Insights

The training code is significant for two reasons: how it builds the training dataset and how it finds the optimal values of SVR hyperparameters.

In detail, the SVR method proceeds as follows:

1. We first partition the data into chunks of the size of 10.

```
X, y = self._partition_array(data, size=sample_size)
```

To illustrate how this method works, let's consider the following example:

```
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
sample_size = 3 # Call the method with sample data and a sample size X
y = example._partition_array(data, size=sample_size)
```

which returns

```
Partitions (X): [[1 2 3] [2 3 4] [3 4 5] [4 5 6] [5 6 7] [6 7 8] [7 8 9]]
One-step ahead values (y): [ 4 5 6 7 8 9 10]
```

In other words, it prepares the features and labels for the SVR training.

2. We execute a grid search to find the optimal range of SVR's hyperparameters C and epsilon.

```
gsc = GridSearchCV(
    SVR(),{
        'C': [.05, .1, .5, 1, 5, 10],
        'epsilon': [0.001, 0.005, 0.01, 0.05, 0.1]
    },
    scoring='neg_mean_squared_error'
)

model = gsc.fit(X, y).best_estimator_
```

3. We return the forecasted value for the next step.

```
model.predict(data[np.newaxis, -sample_size:])[0]
```

In sum, the `_svm_forecast` method is defined as follows:

```
def _svm_forecast(self, data, sample_size=10):
    """
    Partitions `data` and fits an SVM model to this data, then
    forecasts the value one time-step into the future
    """
    X, y = self._partition_array(data, size=sample_size)
```

```
gsc = GridSearchCV(
    SVR(),
    {
        'C': [.05, .1, .5, 1, 5, 10],
        'epsilon': [0.001, 0.005, 0.01, 0.05, 0.1]
    },
    scoring='neg_mean_squared_error'
)

model = gsc.fit(X, y).best_estimator_
return model.predict(data[np.newaxis, -sample_size:])[0]
```

<>

Example 6—Dividend Harvesting Selection of High-Yield Assets

Predicting	Dividend rank
Technology	Regression models
Asset Class	US equities
Difficulty	Medium-hard
Type	Full strategy
Source Code	gnt.co/book-example6

Summary

We build a portfolio of the assets with the highest predicted dividend yield. We use a decision tree regression model to predict future dividend yields based on several financial ratios: PE ratio, revenue growth, free cash flow to operating cash flow ratio, dividend payout ratio, and current ratio.

Motivation

One of the most effective strategies to maximize returns while minimizing risks is dividend harvesting, where an investor invests in assets with the highest dividend yields (defined as dividend per share divided by the price per share). This results in a steady income stream and provides additional stability to the portfolio: companies paying regular dividends are usually financially stable.

Model

Model	
Features	• <u>PE ratio</u>: It is calculated by dividing the current market price of a stock by its earnings per share (EPS). • <u>Revenue growth</u>: Growth in revenue between the last two financial reports. • <u>Free cash flow to operating cash flow ratio</u>: The ratio of free cash flow to operating cash flow in the last financial report. • <u>Dividend payout ratio</u>: The ratio of dividends paid to net income during the last quarter. • <u>Current ratio</u>: Current assets divided by current liabilities.
Predicted Label	Dividend yield.
Model	Decision tree regressor.

This example demonstrates how to concentrate the portfolio on the assets likely to produce the most significant dividend yield from their next dividend payment.

Due to the complexities of the relationships between these factors, we decided to implement the model using the decision tree regression model (see [qnt.co/book-decision-tree](https://quantconnect.com/blog/decision-tree/)).

Trading Universe

The universe of the assets is the top 100 constituents of the QQQ ETF, with the most significant weights selected at the beginning of each month.

Portfolio Construction

Model Training Time	On the first trading date of every month, 30 minutes before the market opens.
Portfolio Rebalancing Time	Immediately after training the model.
Portfolio Weights	Each asset's weight is computed as its predicted dividend yield divided by the total sum of predicted yields.

Trading Logic

The trading method itself rebalances the portfolio, so the weight of each asset is proportional to the expected dividend yield (assign greater weight to assets we think will produce larger dividend yields).

```
def _trade(self):
    r_squared_values = []
    prediction_by_symbol = {}
    for symbol in self._universe.selected:
        symbol_data = self._symbol_data_by_symbol[symbol]
        r_squared = symbol_data.train()
        if r_squared is None:
            continue
        r_squared_values.append(r_squared)
        prediction_by_symbol[symbol] = symbol_data.predict()

    prediction_sum = sum(prediction_by_symbol.values())
    portfolio_targets = [
        PortfolioTarget(symbol, prediction / prediction_sum)
        for symbol, prediction in prediction_by_symbol.items()
    ]
    self.set_holdings(portfolio_targets, True)
    self.plot("Portfolio Size", "Count", len(portfolio_targets))
```

Tearsheet

The results show the following:

- The Sharpe ratio only ranges from 0.476 to 0.617.
- The Sharpe ratio is typically more significant with a larger universe.
- The results are more sensitive to changes in `universe_size` than `lookback_years`.
- All parameter combinations lead to a positive Sharpe ratio.
- See [Figures 6.20](#) and [6.21](#).

Parameter `universe_size`:

- The minimum is 20, so at least one asset will have a dividend payment.
- The maximum is 100 to select all assets in the QQQ ETF (the universe).
- The step size is 20 because it leads to round numbers.

Parameter `lookback_years`:

- The minimum is 4 because dividend payments don't happen very frequently.
- The maximum is 8 because anything further into the past may no longer be relevant.
- The step size is 1 because it's the smallest possible step size.

Backtest parameters:

- `universe_size`: 100.
- `lookback_years`: 5.

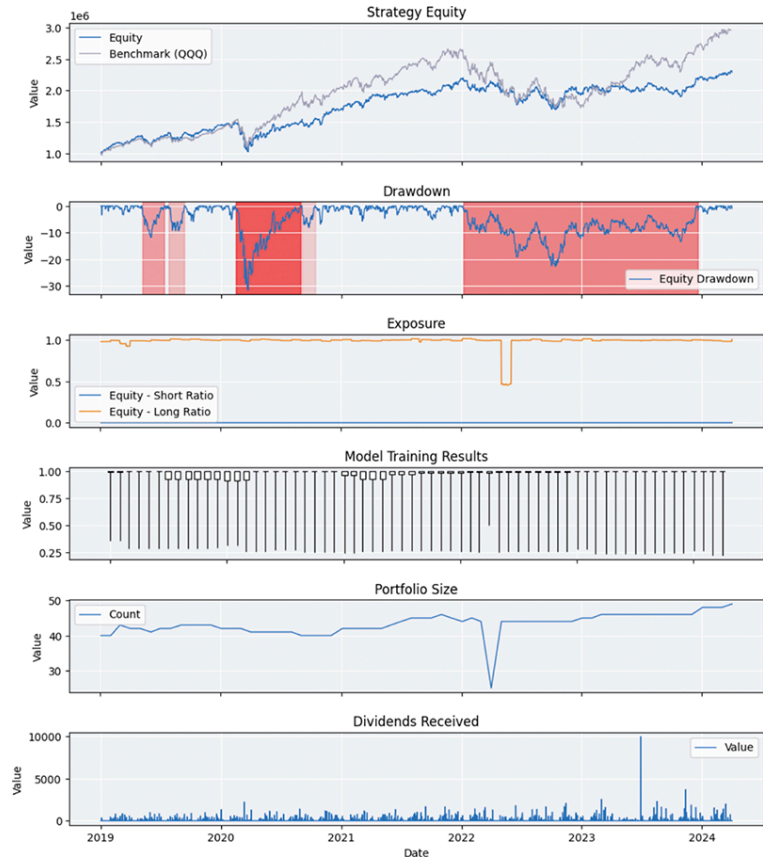


Figure 6.20 Equity curve, performance plots, and custom plots for Example 6.

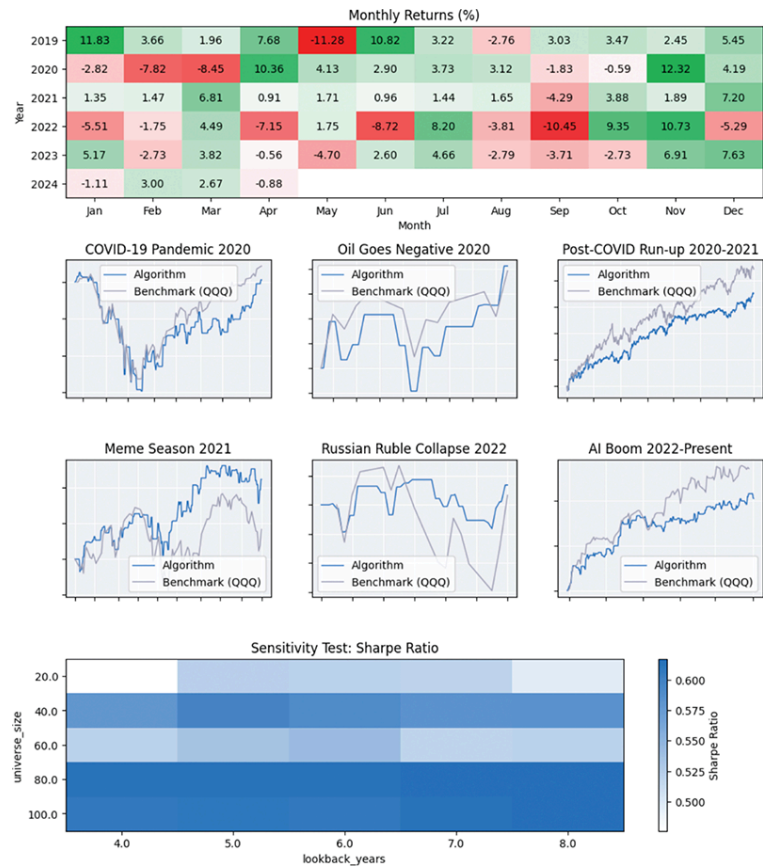


Figure 6.21 Monthly returns, crisis events, and sensitivity tests for Example 6.

Implementation Insights

During the selection of the trading universe, we create for each constituent of the ETF an instance of the class `SymbolData`, which encapsulates each ETF constituent:

- Five years of history of all fundamental factors, 5 years of history of the label (i.e., the dividend yield), along with the class methods to build the history.
- An instance of the sklearn's `DecisionTreeRegressor`.
- The training method is mainly technical; it constructs the training dataset for the regression of current period factors and the next dividend yield.
- The prediction method just applies the latest values of the factors and uses the trained model to return the next period's expected dividend yield.

The trading algorithm also logs the total dividends received:

- On each new market data received (`on_data` method), it updates the Dividends Received chart with the new dividends received.
- At the end of the backtesting of the algorithm (`on_end_of_algorithm`), we log all the dividend payments received throughout the backtesting.

```
def on_data(self, data):
    dividends_received = 0
    for symbol, dividend in data.dividends.items():
        security_holding = self.portfolio[symbol]
        if security_holding.invested:
            dividends_received += (
                dividend.distribution * security_holding.quantity
            )
    self.plot("Dividends Received", "Value", dividends_received)

def on_end_of_algorithm(self):
    self.log("Dividends received:")
    dividend_by_symbol = {
        security.symbol: security.holdings.total_dividends
        for security in self.securities.total
        if security.holdings.total_dividends
    }
    sorted_by_dividends_earned = sorted(
        dividend_by_symbol.items(), key=lambda x: x[1], reverse=True
    )
    for symbol, total_dividends in sorted_by_dividends_earned:
        self.log(f"- ${total_dividends} ({symbol.value})")
    self.log("-----")
    self.log(f"Total: ${sum(dividend_by_symbol.values())}")
```



Example 7—Effect of Positive-Negative Splits

Predicting	Returns (equities)
Technology	Regression
Asset Class	US equities
Difficulty	Easy-medium
Type	Full strategy
Source Code	gnt.co/book-example7

Summary

This algorithm tries to profit from the expected volatility of stock splits. It uses a multiple linear regression model to estimate the future return when a stock split is imminent. It trades in the same direction, and then liquidates the position after 1 week.

Motivation

Due to innovative products, the investments they attract and dynamic business models, tech companies often experience more rapid growth than firms in traditional industries, leading to substantial increases in their stock prices.

When the stock prices reach levels perceived as too high, companies often resort to stock splits. A stock split increases the number of shares while proportionally decreasing the share price, enhancing liquidity and making the stock more accessible to a broader base of investors.

Historically, stocks tend to perform well after a stock split announcement as the perceived lower price and increased accessibility draw in more investors, driving up demand and, hence, the stock price. In addition, the market views stock splits as a positive signal, indicating that the company's management is confident in its prospects.

This strategy leverages historical data and market psychology surrounding stock splits to capitalize on the upcoming volatility caused by stock splits. It creates a linear regression model on the split factors (split direction) announced at the event of a split warning, the trailing 1-month return of the tech sector ETF (sector momentum), and the future price return after the intended holding duration.

Model

Model	
Features	• The reported split factor for each stock split. • The sector monthly return using the ETF XLK.
Predicted Label	The look-ahead future return of holding the corresponding stock for 3 days as of the day of the split announcement.
Model	Linear regression.

Trading Universe

We consider the fundamental universe of all the Morningstar technology classified assets.

Portfolio Construction

Model Training Time	At midnight of the first trading day of each month.
Portfolio Rebalancing Time	After each reported split.
Portfolio Weights	We limit the number of open trades; we aim at placing up to four simultaneous open positions and allocate for each position 25% of the total allocated cash.

Trading Logic

The trading algorithm monitors the split events; each split triggers the event handler `on_splits`, which receives all split events for subscribed stocks.

The method proceeds as follows:

Step 1: We filter out any split event of the tech sector ETF since we do not intend to trade it.

Step 2: Depending on the type of split, we act as follows:

- For all split warnings, we first check if we can open another trade. (We limit the number of open trades—we aim at placing up to four simultaneous open positions and allocate 25% of the total allocated cash for each position.) If we can, we predict the expected return given the split factor and the current value of the rate of change sector indicator. Suppose the predicted return is positive, and the optimal number

of trades is positive. In that case, we place the corresponding trade by creating an instance of the class `trade`, which we append to the list of active trades of any given symbol in the dictionary

`_trades_by_symbol`.

- For all splits that occurred, we update the instance of the corresponding trade object with the actual split parameters.

The results show that the algorithm dramatically outperforms the XLK benchmark.

Tearsheet

The results show the following:

- A `hold_duration` of 3 days generates the most significant Sharpe ratio.
- All the Sharpe ratios are ≥ 0.7 .
- See [Figures 6.22](#) and [6.23](#).

Backtest parameters:

- `max_open_trades`: 4.
- `hold_duration`: 3.
- `training_lookback_years`: 4.

Parameter `hold_duration`:

- The minimum is 1 because it's the smallest positive integer.
- The maximum is 5 because it equals one trading week.
- The step size is one because it's the smallest possible step size.

Parameter `training_lookback_years`:

- The minimum is 3 because split payments don't happen very frequently.
- The maximum is 6 because anything further into the past may no longer be relevant.
- The step size is 1 because it's the smallest possible step size.

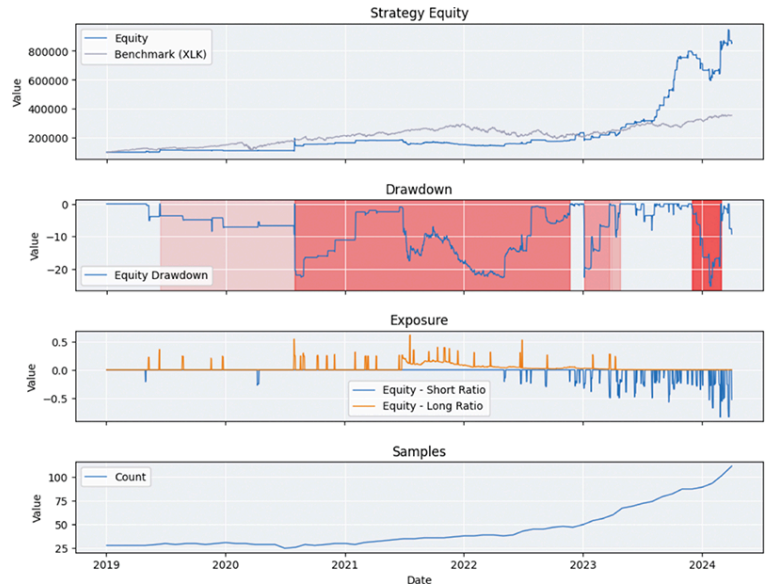


Figure 6.22 Equity curve, performance plots, and custom plots for Example 7.



Figure 6.23 Monthly returns, crisis events, and sensitivity tests for Example 7.

Implementation Insights

We hold a record of all trades in the global dictionary `_trades_by_symbol` ; for a symbol, it stores the list of active trades in instances of the class `trade`. This class places the trade for each selected split, adjusting the quantity of the stocks bought by the actual `split_factor` when the split happens and closing the trade after the hold period elapses.

```
class Trade:
    def __init__(self, algorithm, symbol, hold_duration, quantity):
        self.closed = False
        self._symbol = symbol
        self._close_time = algorithm.time + hold_duration
        self._quantity = quantity
        algorithm.market_on_open_order(symbol, quantity)

    def on_split_occurred(self, split):
        self._quantity = int(self._quantity / split.split_factor)

    def scan(self, algorithm):
        if not self.closed and self._close_time <= algorithm.time:
            algorithm.market_on_open_order(self._symbol , -self._quantity)
            self.closed = True
```

We utilize Scheduled Events to run a function at midnight every day to re-train the models, update the future return, and exit trades:

The `_train` method proceeds as follows:

1. First, it retrieves the history of splits for the monitored stocks within the monitored lookback period.
2. Then, it retrieves the historical pricing of the monitored stocks with splits.
3. Next, it builds row-by-row the training numpy array with the three columns—the reported split factor, sector return, and the look-ahead return.
4. Finally, we train the linear regression model to predict the future return using the reported split factor and sector return.



Example 8—Stop Loss Based on Historical Volatility and Drawdown Recovery

Predicting	Downside volatility for stop loss
Technology	Regression/forecasting
Asset Class	US equities and US options
Difficulty	Medium-hard
Type	Full strategy
Source Code	gnt.co/book-example8

Summary

We employ regression models to safeguard a portfolio against downside risks. *We start with* a simple model featuring a fixed percentage-based stop-loss order as a benchmark. We then progress to a more complex model, dynamically adjusting the stop-loss distance using

LASSO regression. In the final model, we substitute the stop market order with a put option, with parameters optimized through LASSO regression.

Motivation

Market volatility and unforeseen economic and company-specific news can lead to significant losses if appropriate risk management strategies are not in place.

Traditional risk management techniques often involve stop-loss orders, which automatically sell a security when its price falls to a predetermined level.

While simple and widely used, fixed percentage-based stop-loss orders have limitations. They do not account for the dynamic nature of market conditions. They can lead to premature exits during temporary market dips, potentially resulting in missed recovery opportunities and increased trading costs.

Recognizing these limitations, we employ a series of increasingly sophisticated regression models to enhance the effectiveness of stop-loss strategies and better protect a portfolio against downside risks:

1. Fixed Percentage-based Stop-loss Order: We use a simple model with a fixed percentage-based stop-loss order as a starting point. This model serves as a benchmark for our strategy, providing a baseline measure of protection against downside risks. While straightforward, this approach does not adapt to changing market conditions, which can lead to suboptimal performance.
2. Dynamic Stop-loss Distance Using LASSO Regression: To address the rigidity of fixed stop-loss orders, we advance to a more complex model that dynamically adjusts the stop-loss distance. By employing LASSO regression, we identify key factors influencing price movements and determine the optimal stop-loss levels based on these factors. This adaptive approach helps balance protecting against significant losses and avoiding unnecessary exits during short-term volatility.
3. Put Option Substitution with LASSO Regression Optimization: In the final model, we seek to enhance our risk management strategy by replacing the stop market order with a put option. Put options provide a guaranteed floor price for the underlying asset, offering a more robust form of protection. The parameters of the put option, including strike price and expiration date, are optimized using LASSO regression, which allows us to tailor the option strategy to current market conditions and ensure that the portfolio is effectively shielded from downside risks while minimizing costs.

Advanced regression models ensure that our strategies are data-driven and adaptive, providing superior protection compared to traditional methods.

Model

The first trading algorithm places a fixed price percentage stop-loss order.

The second and the third trading algorithms share the same model. It was not changed between experiments, so the performance of the second and third algorithms could be compared better.

Model	
Features	• VIX. • Average true range of the last n months. • The standard deviation of the previous n months.
Predicted Label	• Weekly low return: the return from the opening price of the week to the minimum low price over the following 5 trading days (1 week).
Model	LASSO regression model.

LASSO regression is optimal for this trading strategy because it effectively selects the most relevant features, reduces noise by excluding less essential variables, and prevents overfitting through regularization. Additionally, LASSO regression handles multicollinearity, ensuring robust performance with correlated predictors. These result in more stable and generalizable models that adapt dynamically to changing market conditions.

Trading Universe

We trade KO stock, and in algorithm three, we include KO put options.

Portfolio Construction

Model Training Time	Two minutes after the market opens on the first trading day of each week.
Portfolio Rebalancing Time	Immediately after the model is trained.
Portfolio Weights	100% of the portfolio is held in the KO stock.

Trading Logic

Each of the three algorithms buys 100,000 USD worth of shares of KO stocks at the beginning of each week and liquidates the position within a week:

- [Algorithm 1](#): A benchmark algorithm buying and selling the same number of shares with a stop-loss order at 95% of the original purchase price at 9:32 am at the start of each week, and then liquidating the remaining shares, if any, the following week at 9 am before the market opens.
- [Algorithm 2](#): An identical algorithm where we replace the 95% stop-loss order with a stop-loss order \$0.01 below the LASSO regression predicted low price of the stock.
- [Algorithm 3](#): An identical algorithm where we replace the stop-loss order with a put option placed below the LASSO regression predicted low price of the stock.

Tearsheet

Algorithm 1: Benchmark—Fixed Percentage Stop Loss

The results show the following:

- The Sharpe ratio collapses when the stop loss is too tight (≥ 0.985).
- 17/20 of the stop-loss distances tested outperform buy-and-hold (0.263 Sharpe).
- See [Figures 6.24](#) and [6.25](#).

Backtest parameters:

- `stop_loss_percent` : 0.95.

Parameter `stop_loss_percent` :

- The minimum is 0.9 because KO isn't very volatile. It only dropped >20% once during the backtest period.
- The maximum is 0.995 because anything tighter than a 0.5% stop loss will likely trigger the stop loss in every trade.
- The step size is 0.005 because it gives us 20 different parameter values to test with just one decimal place.

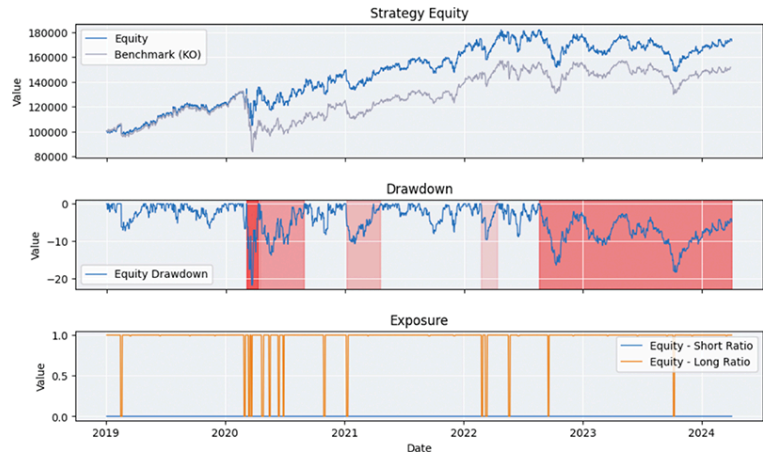


Figure 6.24 Equity curve and performance plots for Example 8.1.

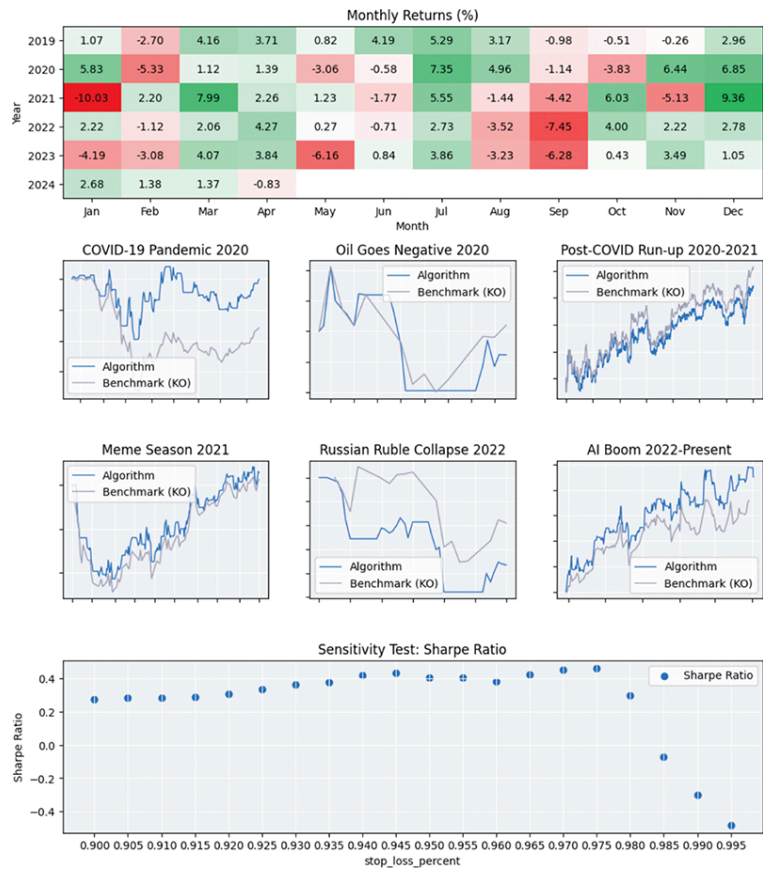


Figure 6.25 Monthly returns, crisis events, and sensitivity tests for Example 8.1.

Algorithm 2: ML Placed Stop Loss

The results show the following:

- The Sharpe ratio generally increases with a larger `stop_loss_buffer` over our tested range.
- 28/30 parameter combinations tested outperform buy-and-hold (0.263 Sharpe).
- All parameter combinations lead to a positive Sharpe ratio.
- See [Figures 6.26](#) and [6.27](#).

Backtest parameters:

- `indicator_lookback_months : 3`

- `stop_loss_buffer` : 0.01
- `alpha_exponent` : 4

Parameter `indicator_lookback_months` :

- The minimum is 1 because it's the smallest integer to choose.
- The maximum is 6 because it's half a year.
- The step size is 1 because it's the smallest integer to choose.

Parameter `stop_loss_buffer` :

- The minimum is 0.01 because it's the minimum price variation of the asset.
- The maximum is 1.01 because it's almost exactly \$1, and anything more significant will place the stop relatively far away from the model's prediction.
- The step size is 0.25 because it equals one quarter and gives us several samples between the minimum and maximum.

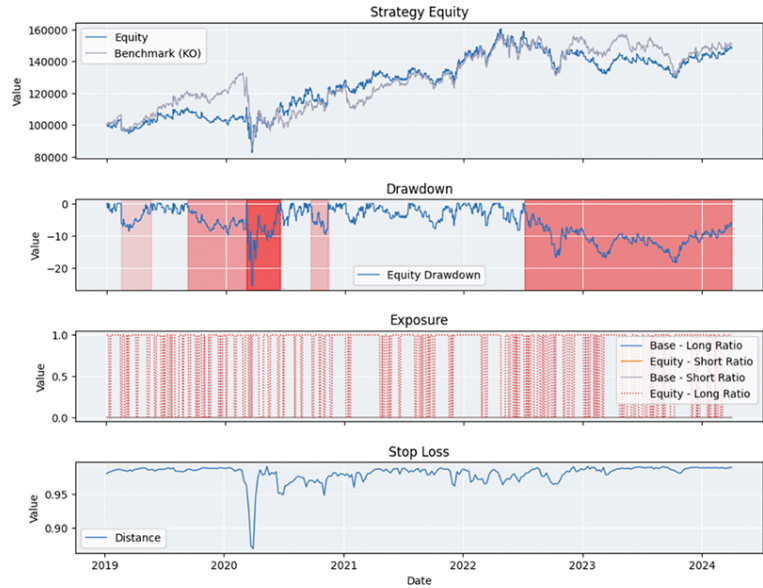


Figure 6.26 Equity curve, performance plots, and custom plots for Example 8.2.

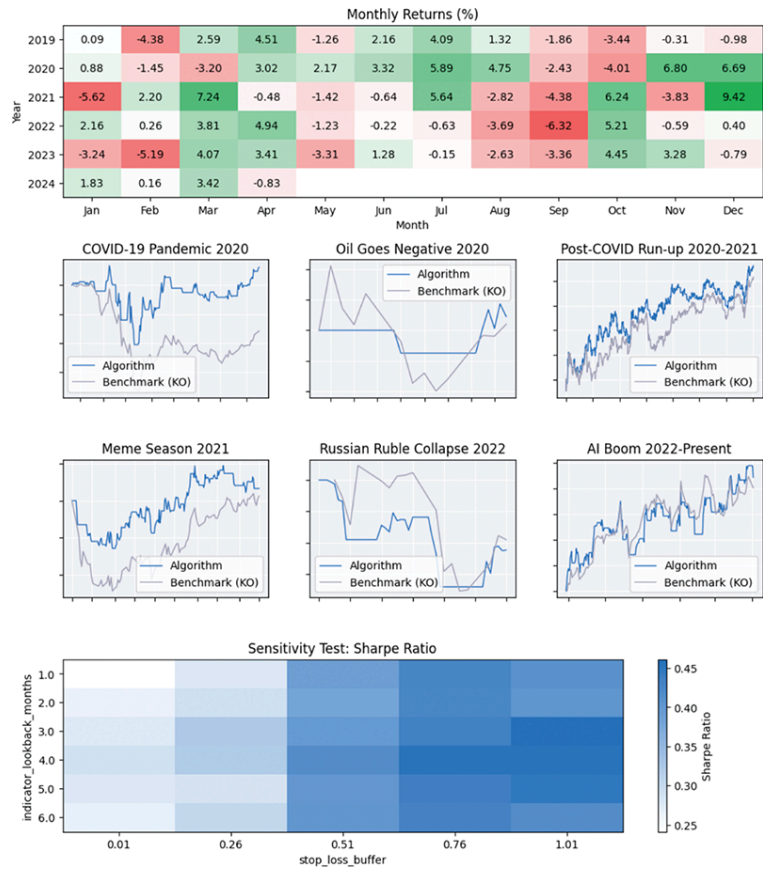


Figure 6.27 Monthly returns, crisis events, and sensitivity tests for Example 8.2.

Algorithm 3: ML Put Option Hedge

See [Figures 6.28](#) and [6.29](#).

Backtest parameters:

- `alpha_exponent` : 4.

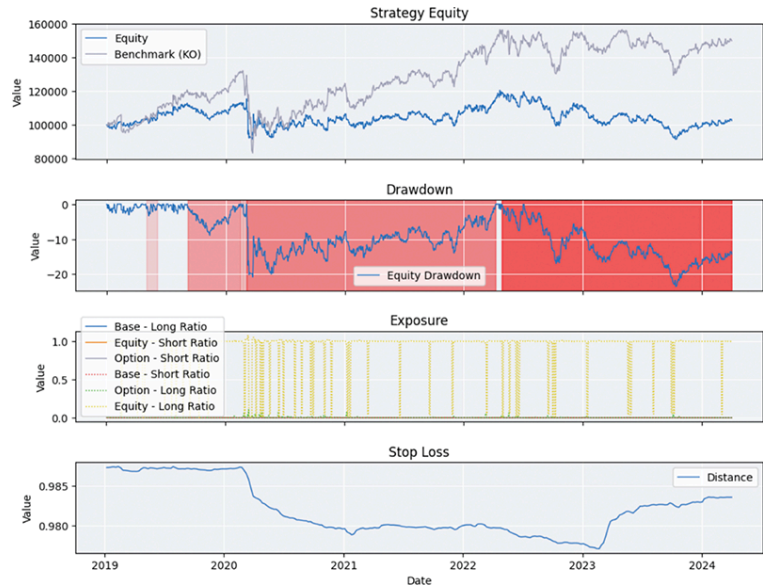


Figure 6.28 Equity curve, performance plots, and custom plots for Example 8.3.

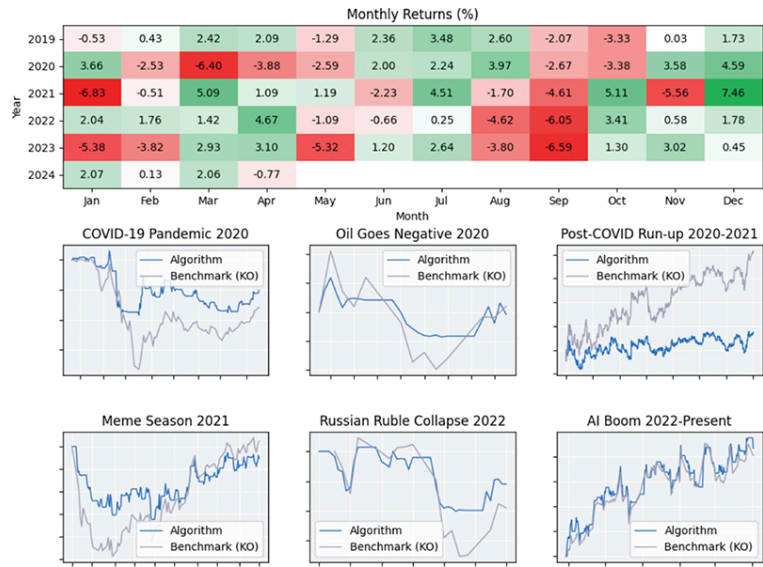


Figure 6.29 Monthly returns and crisis events for Example 8.3.

Implementation Insights

Algorithm 1: Benchmark—Fixed Percentage Stop Loss

We place a market order for 100,000 USD worth of KO stocks at the beginning of each week, 2 minutes after the market opens at 9:30 am. At the same time, we place a sell-stop market order for the identical quantity of stocks whenever the price drops to a given percentage (self._stop_loss_percent defined as 95%) of the purchase price.

If the stop-loss order is not executed during the week, we liquidate the position with a Market on Open at the beginning of the following week; see the method liquidate.

Algorithm 2: ML Placed Stop Loss

The input factors for our model are as follows:

- VIX.
- Average True Range of the last 22 daily closing prices of the KO stock.
- The standard deviation of the last 22 daily closing prices of the KO stock.

```
def initialize(self):
    ...
    self._security = self.add_equity(
        "KO", data_normalization_mode=DataNormalizationMode.RAW
    )
    self._symbol = self._security.symbol

    period = 22 * self.get_parameter('indicator_lookback_months', 1)
    self._vix = self.add_data(CBOE, "VIX", Resolution.DAILY).symbol
    self._atr = AverageTrueRange(period, MovingAverageType.SIMPLE)
    self._std = StandardDeviation(period)
```

This version of the algorithm uses a LASSO regression model to adjust the stop-loss distance to improve performance dynamically.

The regression model is run on the data frame `_samples` containing 3 years of samples.

```
def initialize(self):
    # ...
    # Create a DataFrame to store the factors and labels.
    self._samples = pd.DataFrame(
        columns=['vix', 'atr', 'std', 'weekly_low_return'],
        dtype=float
    )
    self._samples_lookback = timedelta(3 * 365)
```

The columns of the model data frame are as follows:

- `vix`—the CBOE VIX daily values.
- `atr`—average true range of the AAPL stock over the last 22 days.
- `std`—standard deviation of the AAPL stock over the previous 22 days.
- `weekly_low_return`—the label we train the model to predict; the rate of change from today's open price to the low price over the upcoming 5 trading days (1 week).

The code populating `_samples` data frame is highly technical, and we set up consolidators to produce the daily bars from the minute bars the algorithm receives. In addition, we repopulate the entire data frame whenever there is a stock split so that our indicators reflect the new adjusted prices. Finally, we update the training data frame with updated VIX values whenever they arrive.

The algorithm is ready to trade once we have populated the training dataset stored in the `_samples` data frame. It works as follows—at the beginning of each week:

1. We buy the KO stock.

2. We predict the low price of the week.
3. We place a stop market order with a stop price of \$0.01 (the `self._stop_loss_buffer`) below the predicted weekly low price; the stop loss is only hit if the market is more volatile than we expected. If the stop loss isn't hit throughout the week, we cancel it and liquidate the long position in the underlying with a Market On Open order.

```
def initialize(self):
    # ...
    self._stop_loss_buffer = self.get_parameter('stop_loss_buffer', 0.01)

    date_rule = self.date_rules.week_start(self._symbol)
    self.schedule.on(
        date_rule,
        self.time_rules.after_market_open(self._symbol, 2),
        self._enter
    )
    self.schedule.on(
        date_rule,
        self.time_rules.after_market_open(self._symbol, -30),
        self.liquidate
    )

    self._model = Lasso(alpha=10*(-self.get_parameter('alpha_exponent', 4)))

def _enter(self):
    # Train a model to predict the return from today's open to the
    # low price of the upcoming week.
    training_samples = self._samples.dropna()
    self._model.fit(
        training_samples.iloc[:, :-1],
        training_samples.iloc[:, -1]
    )

    prediction = self._model.predict(
        [self._samples.iloc[:, :-1].dropna().iloc[-1]]
    )[0]
    predicted_low_price = self._security.open * (1 + prediction)
    self.plot("Stop Loss", "Distance", 1 + prediction)

    # Place the entry order.
    quantity = self.calculate_order_quantity(self._symbol, 1)
    self.market_order(self._symbol, quantity)
    # Place the stop loss $0.0x below the predicted low price, so
    # that the stop loss is only hit if the market is more volatile
    # than we expected.
    self.stop_market_order(
        self._symbol, -quantity,
        round(predicted_low_price - self._stop_loss_buffer, 2)
    )
```

Algorithm 3: ML Put Option Hedge

This strategy builds on the previous version by substituting the stop market order with a put option. Both methods protect against downside price movements, each with its pros and cons:

- With a stop market order, we can accurately specify the stop price. However, with a stop market order, we don't know the fill price. If the market is volatile or there is a big overnight gap in price, the stop market order may be filled below our chosen stop price. In addition, when the stop market order is activated, it executes as a market order, exposing us to spread costs and market impact.
- The key drawback to using a put option is that we must pay extra to buy the option contract. With a put option hedge, we cannot choose an exact strike price. We can only select a strike price nearest our preferred stop-loss level. However, we know precisely what price our "stop loss" will fill with a put option hedge because it's the strike price of the option we select. With a put option hedge, we are not vulnerable to gaps, spread costs, or market impact because if the underlying price falls below the strike price, we can always exercise the option to sell the shares.

Fees are included by default in LEAN trading algorithms. This example, however, demonstrates how to change the fee model, even though the `InteractiveBrokersFeeModel` is the default fee model.

```
class CaseOfTheMondaysAlgorithm(QCAlgorithm):

    def initialize(self):
        # ...
        self.set_security_initializer(
            IBFeesSecurityInitializer(
                self.brokerage_model,
                FuncSecuritySeeder(self.get_last_known_prices)
            )
        )

class IBFeesSecurityInitializer(BrokerageModelSecurityInitializer):

    def __init__(self, brokerage_model, security_seeder):
        super().__init__(brokerage_model, security_seeder)

    def initialize(self, security):
        super().initialize(security)
        security.set_fee_model(InteractiveBrokersFeeModel())
```

We create identical factors and training data frames as in Algorithm 2.

The trading algorithm works as follows:

- At 9:32 am, at the start of every week, we buy KO shares and calculate the share's week's low price as in Part II.
- However, instead of using a stop market order this time, we select the first put contract with a strike price below the `predicted_low_price + contract.ask_price`. We add the `contract.ask_price` to the prediction because the contract costs money, so we tighten up the stop loss to pay for it.
- The contracts expire on Friday. If they expire ITM, they exercise automatically and close our long position in the underlying stock. If they expire OTM, they expire worthless, and we liquidate the underlying stock with a market on an open order.


```

class CaseOfTheMondaysAlgorithm(QCAlgorithm):

    def initialize(self):
        # ...

        date_rule = self.date_rules.week_start(self._symbol)
        self.schedule.on(
            date_rule,
            self.time_rules.after_market_open(self._symbol, 2),
            self._enter
        )
        self.schedule.on(
            date_rule,
            self.time_rules.after_market_open(self._symbol, -30),
            self._liquidate_if_possible
        )

        self._model = Lasso(alpha=10**(-self.get_parameter('alpha_exponent', 4)))

    def _liquidate_if_possible(self):
        self.liquidate(self._symbol)
        for symbol, security_holding in self.portfolio.items():
            # If it's an Option contract and we have no open orders for
            # it, liquidate it.
            if (security_holding.type == SecurityType.OPTION and
                not list(self.transactions.get_open_order_tickets(symbol))):
                self.liquidate(symbol)

    def _enter(self):
        # Train a model to predict the return from today's open to the
        # low price of the upcoming week.
        training_samples = self._samples.dropna()
        self._model.fit(
            training_samples.iloc[:, :-1],
            training_samples.iloc[:, -1]
        )

        prediction = self._model.predict(
            [self._samples.iloc[:, :3].dropna().iloc[-1]]
        )[0]
        predicted_low_price = self._security.open * (1 + prediction)
        self.plot("Stop Loss", "Distance", 1 + prediction)

        for chain in self.current_slice.option_chains.values():
            # Buy the underlying Equity.
            quantity = self.calculate_order_quantity(self._symbol, 1)
            self.market_order(self._symbol, quantity)

            # Select the put Contract.
            puts = [
                contract
                for contract in chain
                if contract.strike < predicted_low_price + contract.ask_price
            ]
            contract=sorted(puts, key=lambda contract: contract.strike)[-1]

            # Buy the put Contract.
            tag=f"Predicted week low price: {round(predicted_low_price, 2)}"
            self.market_order(contract.symbol, quantity // 100, tag=tag)

```

Discussion of the Performance of Algorithm 1, Algorithm 2, and Algorithm 3

All three algorithm versions trade in isolation, and we can easily compare their relative performances.

The results show that the put Option hedge underperforms a stop market order usage. Algorithm 1: Fixed Percentage Stop Loss produced superior results. There are plenty of ways to improve the strategies, and these implementations should be used as a springboard for your research.

Example 9—ML Trading Pairs Selection

Predicting	Preprocessing likely pairs clusters
Technology	PCA, clustering for pairs
Asset Class	US equities
Difficulty	Medium-hard
Type	Research rotebook
Source Code	gnt.co/book-example9

Summary

We use the PCA to transform the standardized returns of assets into their principal components. Then, we cluster assets with OPTICS, run a cointegration test, and calculate the Hurst exponent for mean reversion. Finally, we calculate the pair’s half-life and the number of spread-crosses in 12 months.

Motivation

In finance, **PCA** is crucial for identifying the underlying factors that drive asset price movements. The first three principal components (PCs) in the context of stock returns can be interpreted as follows:

1. First Principal Component (PC1)—Market Factor: The first principal component captures the most significant variance in the stock returns data. Since most stocks tend to move with the broader market, this principal component is usually interpreted as the market factor reflecting the general market trend.
2. Second Principal Component (PC2)—Sector/Industry Factor: The second principal component captures the most significant variance, often interpreted as reflecting sector or industry-specific factors.
3. Third Principal Component (PC3)—Style/Size Factor: The third principal component explains the variance not captured by the first two principal components and can be interpreted as a style/size factor cap-

turing variations due to the differences in investment style (growth vs. value stocks) or company size (small-cap vs. large cap stocks).

Once the assets are transformed into their principal components, we employ the **OPTICS clustering algorithm** to group assets with similar characteristics. By grouping similar assets, we can identify potential pairs or groups that exhibit comparable behaviors, which is valuable for pairs trading.

Testing for **cointegration** is critical for identifying pairs of assets that share a long-term equilibrium relationship despite short-term price deviations. **The Hurst exponent** measures the tendency of a time series to revert to its mean or to exhibit trending behavior. By combining cointegration and the Hurst exponent, we can rigorously test and confirm the mean-reverting properties of asset pairs.

Finally, we calculate the pair's **half-life** and the **number of spread-crosses** over 12 months. The half-life of a pair measures how quickly the spread between two cointegrated assets reverts to its mean. A shorter half-life indicates a faster mean reversion, which is desirable for trading as it allows for more frequent and reliable profit opportunities. The number of spread crosses indicates how often the spread between the pair crosses its mean. A higher number of spread-crosses suggests more frequent trading opportunities, making the pair attractive for a mean-reversion strategy.

Implementation Insights

In this example, based upon the MLFinLab blog post ([qnt.co/book-hudson-pairs](https://mlfinlab.com/blog/quant-co-book-hudson-pairs)), we use ML to help us discover candidate pairs of stocks for a pairs trading strategy.

Step 1: Calculate daily returns of assets under consideration.

We first select all assets trading on a single day (January 3, 2024). We gather 3 years of historical closing prices for all the assets (roughly January 2022–2024). We drop assets with `NaN` values in the historical data, leaving 927 assets in total. From the series of closing prices, we calculate the daily returns of each asset.

```
qb = QuantBook()
end_date = datetime(2024, 1, 3)

universe_history = qb.universe_history(
    qb.universe.etf('IWB'), end_date - timedelta(1), end_date
)
symbols = [fundamental.symbol for fundamental in universe_history.iloc[0]]

prices = qb.history(
    symbols, end_date - timedelta(3 * 365), end_date, Resolution.DAILY
)['close'].unstack(0).dropna(axis=1)
daily_returns = prices.pct_change().dropna()
```

Step 2: Standardize the daily returns.

We then standardize the daily returns in preparation for principal component analysis.

```
from sklearn.preprocessing import StandardScaler
standardized_returns = StandardScaler().fit_transform(daily_returns)
```

Step 3: Run PCA.

We apply the PCA to reduce the dimensionality of the dataset to three principal components.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=3, random_state=0)
pca.fit(standardized_returns)
```

Step 4: Load Principal Components into a Data Frame.

We load the principal components into a data frame to show the exposure of each asset to each principal component.

```
factor_exposures = pd.DataFrame(
    index=[f"component_{i}" for i in range(pca.components_.shape[0])],
    columns=daily_returns.columns,
    data=pca.components_.T)
```

Let's display the results ([Figures 6.30](#) and [6.31](#)):

```
go.Figure(
    [
        go.Scatter3d(
            x=[row['component_0']],
            y=[row['component_1']],
            z=[row['component_2']],
            mode='markers',
            marker=dict(size=3, color='blue'),
            text=symbol,
            showlegend=False
        )
        for symbol, row in factor_exposures.iterrows()
    ],
    dict(
        scene=dict(
            xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
            camera=dict(
                eye=dict(x=2, y=1.5, z=1.5)
            )
        ),
        title='3D Representation of Assets<br><sup>The coordinates represent '
            + 'the contribution the asset has to each component</sup>', height=600, width=
    )
).show()

# Each row of pca.components_ corresponds to a principal component, and
# the values within each row indicate the contribution of the original
```

```
# features to that principal component.  
factor_exposures
```

3D Representation of Assets
The coordinates represent the contribution the asset has to each component

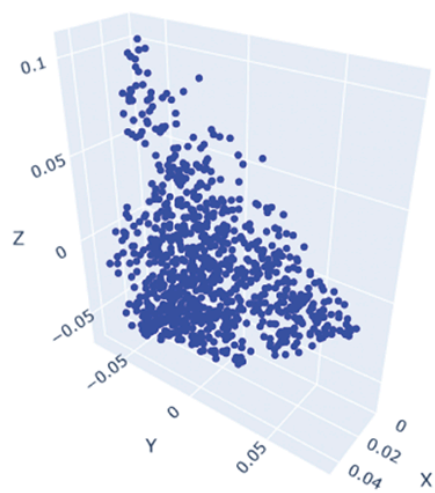


Figure 6.30 Asset representations in 3D space.

FIGURE 6.31
Asset coordinates in 3D space.

	component_0	component_1	component_2
Symbol			
A RPTMYV3VC57P	0.036965	0.031454	0.030914
AA R735QTJ8XC9X	0.040697	-0.023039	-0.035844
AA WF7IHVI76I5H	0.031304	-0.017483	-0.042513
AAL VM9RIYHM8ACL	0.034789	0.006323	-0.047596
AAP SA48O8J43YAT	0.027324	-0.013965	-0.000761
...	...	...	...
ZION R735QTJ8XC9X	0.036183	-0.039567	-0.046786
ZM X3RPXTZRW09X	0.027722	0.076046	0.003985
ZMH S6ZZPKTVDY05	0.033683	-0.000613	0.010533

	component_0	component_1	component_2
ZS	0.028628	0.081103	0.001394
WSVUOMELFQED			
ZTS	0.033949	0.030440	0.049610
VDRJHVQ4FNFP			
927 rows × 3 columns			

Step 5: Organize the assets into clusters using the OPTICS algorithm.

Using the OPTICS clustering algorithm, we organize the assets into clusters based on their 3D-factor exposures ([Figure 6.32](#)).

```
from sklearn.cluster import OPTICS

clustering = OPTICS().fit(factor_exposures)

# Display the results.
group_by_cluster_id = {
    cluster_id: factor_exposures.iloc[
        [i for i, x in enumerate(clustering.labels_) if x == cluster_id]
    ]
    for cluster_id in sorted(set(clustering.labels_))
}

go.Figure(
    [
        go.Scatter3d(
            x=group['component_0'], y=group['component_1'],
            z=group['component_2'], mode='markers',
            marker=dict(size=3), text=group.index,
            name=f"Group {cluster_id}" if cluster_id >= 0 else "Noisy group",
            visible=True if cluster_id >= 0 else 'legendonly'
        )
        for cluster_id, group in group_by_cluster_id.items()
    ],
    dict(
        scene=dict(
            xaxis_title='X', yaxis_title='Y', zaxis_title='Z',
            camera=dict(eye=dict(x=1.752, y=1.25, z=1.25))
        ),
        title='3D Representation of Assets<br><sup>The coordinates represent '
            + 'the contribution the asset has to each component</sup>',
        height=600, width=
    )
).show()

# Drop noisy samples.
labels = clustering.labels_[clustering.labels_ != -1]
print(
    f"Out of {len(clustering.labels_)} assets, OPTICS found {len(labels)}",
    f"non-noisy samples and organized them into {len(set(labels))} groups."
)
```

Out of 927 assets, OPTICS found 347 non-noisy samples and organized them into 44 groups.

3D Representation of Assets

The coordinates represent the contribution the asset has to each component

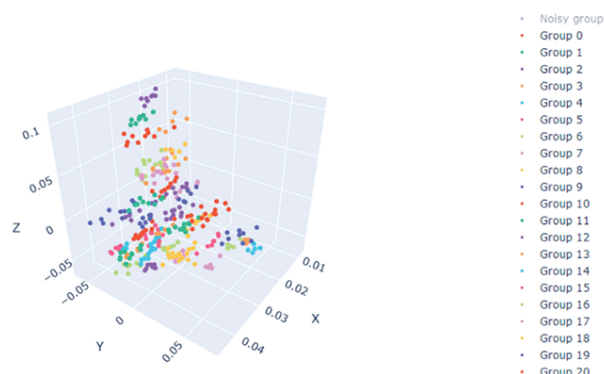


Figure 6.32 Asset representations in 3D space, color coded by group clusters.

Step 6: Identify potential trading pairs.

With the assets separated into distinct groups, we now look for candidate pairs within each group. Within each group, we analyze every possible combination of its assets. (Each pair has two assets from the same group.)

We select a pair if it meets the following four tests:

1. The pair's constituents are cointegrated.
2. The pair's spread's Hurst exponent exhibits a mean-reverting characteristic.
3. The pair's spread diverges and converges within a reasonable time frame.
4. The pair's spread reverts to the mean frequently enough.

Let's discuss each test in detail.

Test 6.1: The pair's constituents are cointegrated.

To accomplish this, we perform Engle-Granger tests (swapping the independent variable for each test) and select the test result with the lowest (i.e., most significant) test statistic with its p-value $\leq 1\%$.

```
from arch.unitroot.cointegration import engle_granger
# ...

cointegration_test_results = [
    engle_granger(prices[symbol_a], prices[symbol_b]),
    engle_granger(prices[symbol_b], prices[symbol_a])
]

cointegration_test_result = sorted(cointegration_test_results, key=lambda x: x.stat)[0]

if cointegration_test_result.pvalue > 0.01:
    test_results.loc[test_results_index[0], 'count'] += 1
    continue # Test failed: This pair is not cointegrated
```

Engle-Granger test is an econometric method used to test cointegration between two non-stationary time series.

“If two or more series are individually integrated (in the time series sense) but some linear combination of them has a lower order of integration, then the series are said to be cointegrated.”

—Wikipedia

The test follows two key steps. First, assuming the two non-stationary variables are integrated of order 1, we estimate their long-run linear relationship. Second, the residuals from the first step's regression are tested for stationarity; if so, the relationship between the two time series is stable and long term.

Test 6.2: The pair's spread's Hurst exponent exhibits a mean-reverting characteristic.

To accomplish this, we calculate the spread using the hedge ratio from the preceding regression test, calculate the Hurst exponent, and ensure it is < 0.5.

```
spread = prices[cointegration_test_result.cointegrating_vector.index[0]] + cointegration_test_resu
H, _, _ = compute_Hc(spread, kind='price', simplified=False)
if H >= 0.5:
    test_results.loc[test_results_index[1], 'count'] += 1
    continue # Test failed: The spread doesn't lean towards mean-reverting
```

Hurst exponent ("H") is a statistical measure of long-term time series memory (i.e., the degree of mean-reversion of a time series):

- $H < 0.5$: the time series tends to return to its average value after moving up or down (mean reversion).
- $H = 0.5$: the future time series values will tend to be independent of the past trajectory.
- $H > 0.5$: the future time series values will boost the trend in the time series—the time series is strongly monotonous—an increasing time series will remain increasing, and a decreasing time series will tend to decrease.

Test 6.3: The pair's spread diverges and converges within a reasonable time frame.

To accomplish this, we calculate the half-life of the spread (i.e., the expected time for the spread to mean-revert) and ensure that “1 day < half-life < 1 year.”

```
lagged_spread = np.roll(spread, 1)
lagged_spread[0] = 0
spread_delta = spread - lagged_spread
spread_delta.iloc[0] = 0
# run OLS regression to find the regression coefficient
model = OLS(spread_delta, add_constant(lagged_spread))
beta = model.fit().params.iloc[1]
half_life = -np.log(2) / beta
```

```
if not (1 < half_life < 252):
    test_results.loc[test_results_index[2], 'count'] += 1
    continue # Test failed: Half-life not between 1 day and 1 year
```

Test 6.4: The pair’s spread reverts to the mean frequently enough.

To accomplish this, we check if the spread crosses the mean at least, on average, 12 times per year or 36 times in total.

```
mean_value = spread.mean()
crossings = (spread > mean_value) & (spread.shift(1) <= mean_value) | (spread < mean_value) & (spr
num_crossings = crossings.sum()
if num_crossings < 3*12: # 36 months in 3 years
    test_results.loc[test_results_index[3], 'count'] += 1
    continue # Test failed: Spread didn't cross the mean at least 12 times per year
```

Once all these conditions have been met, we plot the results, showing the number of pairs tested, the results of the four tests, and the pairs detected at the end (Figure 6.33).

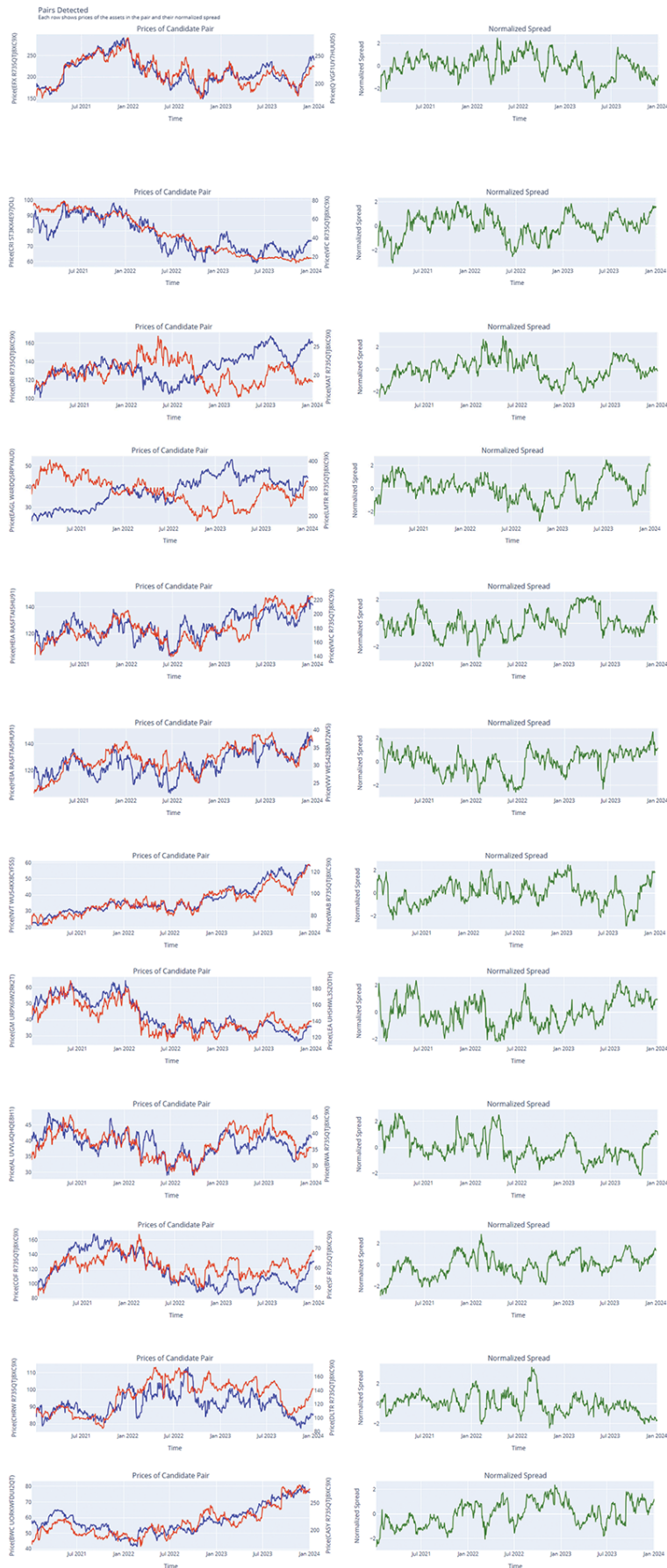
Pairs tested: 1357.

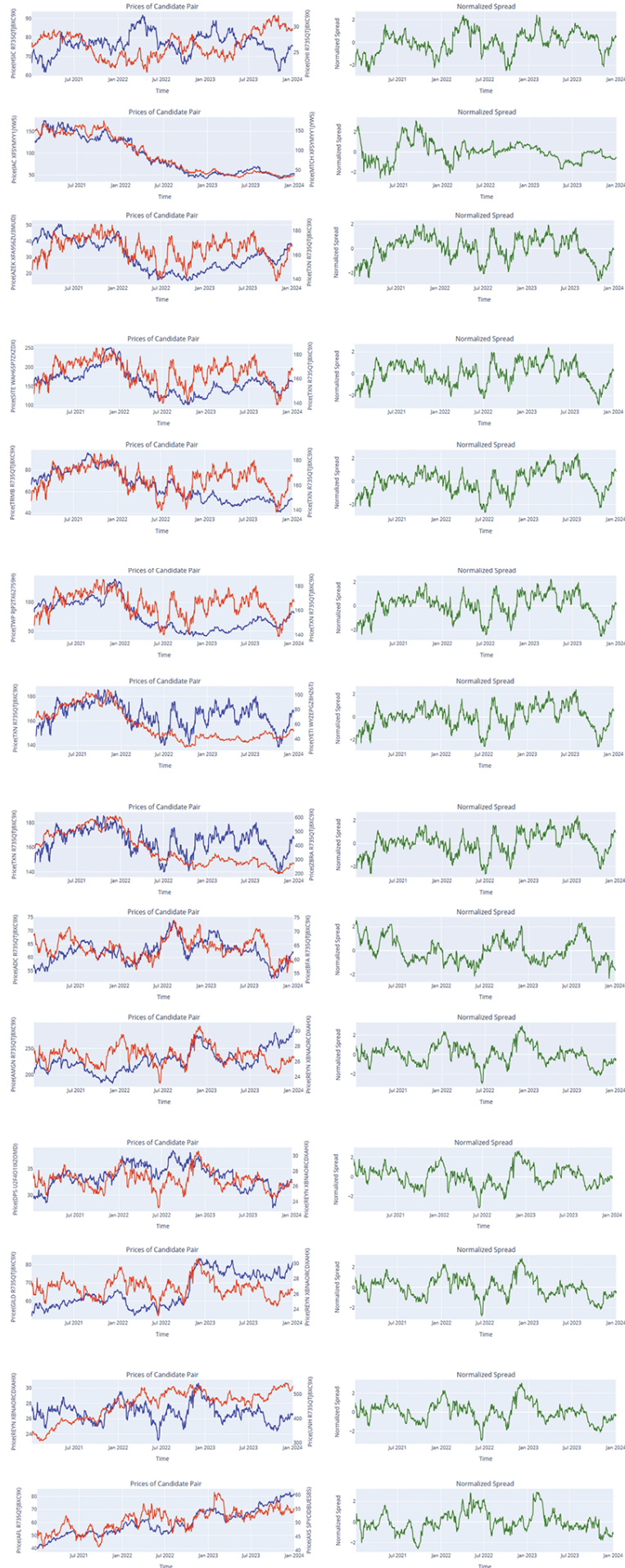
FIGURE 6.33
Output of pairs detection process.

	count	percent_of_tested	cumulative_percent_of_tested
Failed at cointegration test	1315	0.9690	0.9690
Failed at Hurst test	12	0.0088	0.9778
Failed at half-life test	0	0.0000	0.9778
Failed at crossing the mean	0	0.0000	0.9778

Pairs detected: 30.

We then plot the price of each pair and their spread. This helps to visually confirm they’re good pairs for trading (Figure 6.34).





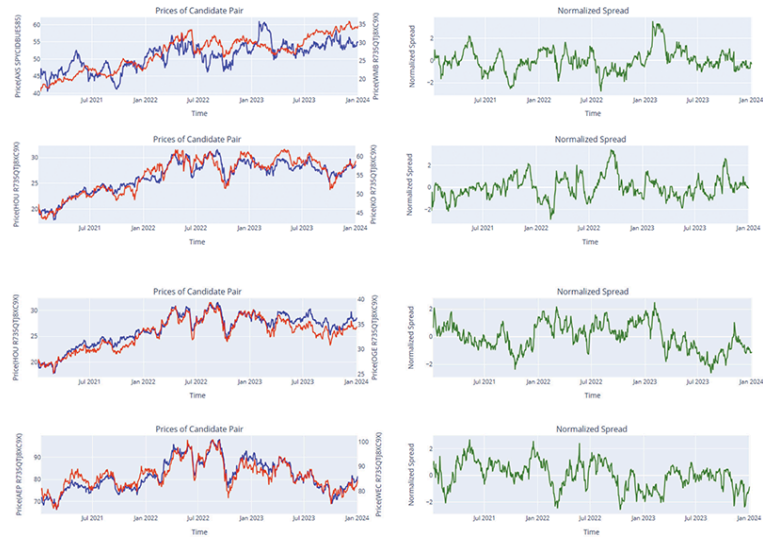


Figure 6.34 Prices of candidate pairs and their normalized spread.



Example 10—Stock Selection through Clustering Fundamental Data

Predicting	Performance rank
Technology	PCA, learning to rank algorithm
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	qnt.co/book-example10

Summary

We apply principal component analysis (PCA) to reduce hundreds of fundamental factors to their principal components. We then use the resulting vectors to predict the rank of stock relative performance. We use a “learning-to-rank algorithm” to form a portfolio of the subset of stocks that we expect to have the most significant future returns.

Motivation

By applying Principal Component Analysis (PCA), we reduce hundreds of fundamental factors into their principal components, capturing the critical drivers of stock returns and minimizing noise. Utilizing these principal component vectors, we employ a learning-to-rank algorithm to predict the relative performance of stocks. Learning-to-rank, a technique commonly used in information retrieval and machine learning, is designed to order items such that the most relevant ones appear at the top. This example enables us to rank stocks based on their predicted future returns. This approach is compelling because it predicts individual stocks’

absolute performance and relative performance. By forming a portfolio from the highest-ranked stocks, we aim to maximize future returns, leveraging the power of PCA for effective data reduction and the precision of learning-to-rank algorithms for optimal stock selection.

Model

Model Features	The first five PCA principal components from standardized 100 fundamental factors for 100 most liquid stocks in the market.
Predicted Label	Rank based upon the return from holding the stock for 22 full trading days.
Model	LGBMRanker.

The LGBMRanker model has been chosen for its balance of performance, flexibility, and efficiency. Compared to simpler models like linear regression, which are easy to interpret but limited in capturing complex relationships, LGBMRanker excels in modeling non-linear interactions between features. It outperforms traditional methods like SVM and random forests in terms of speed and memory efficiency while being less resource-intensive than neural networks. Although it requires careful tuning of hyperparameters, LGBMRanker offers robust performance and effective handling of large datasets.

Trading Universe

The algorithm does fundamental universe selection at the start of each month. First, we select the top 100 most liquid equities from all assets with available fundamental data. Second, we reduce the set of assets to just the 10 equities that are ranked the highest for future returns.

Portfolio Construction

Model Training Time	At the start of each month, after the algorithm selects the 10 stocks to trade.
Portfolio Rebalancing Time	At the start of each month, 1 minute after the market opens.
Portfolio Weights	We form an equal-weighted portfolio of 10 stocks, i.e., each stock weighs 1/10.

Tearsheet

The results show the following:

- The most significant Sharpe ratio was achieved using the smallest value for both parameter `final_universe_size` and components.

- All the Sharpe ratios are ≥ 0 .
- The performance can be sensitive to small changes in the `components` parameter. Changing it from 3 to 4 when the universe size is 5 drops the Sharpe ratio from 0.45 to 0.09.
- See [Figures 6.35](#) and [6.36](#).

Parameter `final_universe_size`:

- The minimum is 5 because anything smaller will concentrate the portfolio on a few assets.
- The maximum is 25 because it makes the universe select the top quartile of assets, a common approach in practice.
- The step size is 5 because it leads to round numbers that traders would likely choose.

Parameter `components`:

- The minimum is 3 because the first three components typically explain $\geq 80\%$ of the variation in the data.
- The maximum is 7 because each additional component explains less of the variation in the data.
- The step size is 1 because it's the smallest possible step size.

Backtest parameters:

- `liquid_universe_size`: 100.
- `final_universe_size`: 10.
- `lookback_period`: 365.
- `components`: 5.

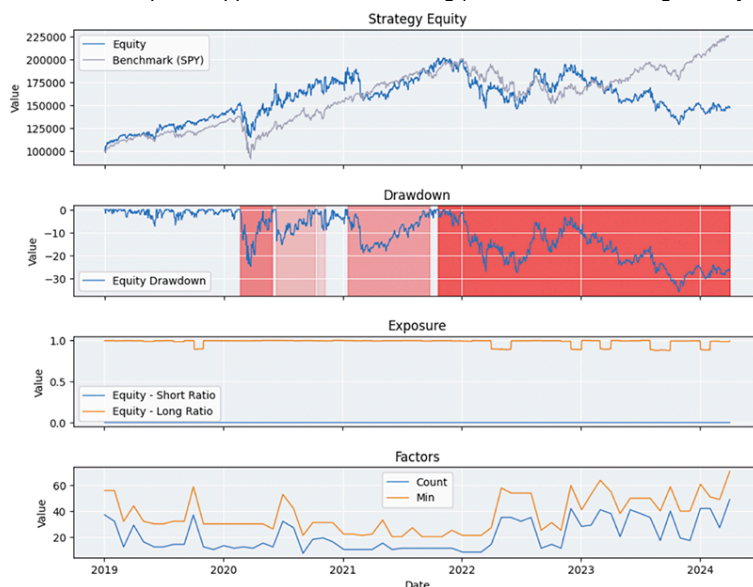


Figure 6.35 Equity curve, performance plots, and custom plots for Example 10.

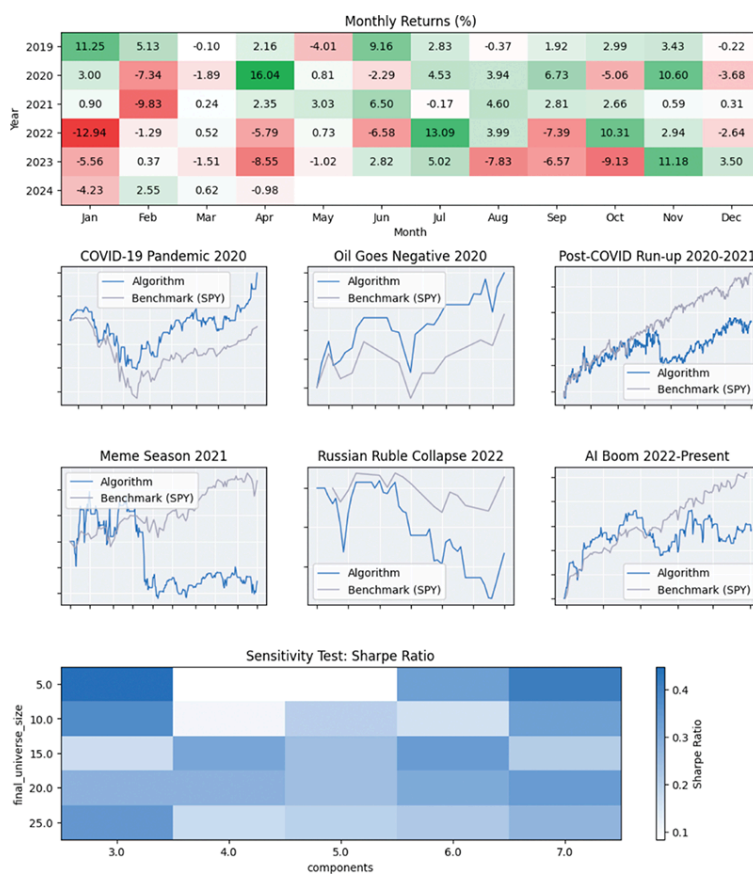


Figure 6.36 Monthly returns, crisis events, and sensitivity tests for Example 10.

Implementation Insights

Notice we infer the trading calendar by using an auxiliary symbol object, SPY.

```
schedule_symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
date_rule = self.date_rules.month_start(schedule_symbol)
```


We store the factors' history in a dictionary indexed by the symbol, with the corresponding value being a pandas' data frame, with the column being each factor of interest and rows including the historical values of each factor:

```
factors_by_symbol = {
    symbol: pd.DataFrame(columns=self._factors)
    for symbol in liquid_symbols
}
history = self.history[Fundamental](
    liquid_symbols, self._lookback_period + timedelta(2)
)
for fundamental_dict in history:
    for symbol, asset_fundamentals in fundamental_dict.items():
        factor_values = []
        for factor in self._factors:
            factor_values.append(eval(f"asset_fundamentals.{factor}"))
        t = asset_fundamentals.end_time
        factors_by_symbol[symbol].loc[t] = factor_values
```

To apply the PCA, we cannot have any NaN values, and we discard any asset that has fewer than 20 factors remaining. The result of this step is the lists `tradable_symbols` and `factors_to_use`.

```
all_non_nan_factors = []
tradable_symbols = []
min_accepted_non_nan_factors = len(self._factors)
for symbol, factor_df in factors_by_symbol.items():
    non_nan_factors = set(factor_df.dropna(axis=1).columns)
    if len(non_nan_factors) < 20:
        # Let's say an asset needs at least 20 factors (otherwise
        # the `intersection` operation will remove almost all
        # factors).
        continue
    min_accepted_non_nan_factors = min(
        min_accepted_non_nan_factors, len(non_nan_factors)
    )
    tradable_symbols.append(symbol)
    all_non_nan_factors.append(non_nan_factors)
if not all_non_nan_factors:
    return []
factors_to_use = all_non_nan_factors[0]
for x in all_non_nan_factors[1:]:
    factors_to_use = factors_to_use.intersection(x)
factors_to_use = sorted(list(factors_to_use))
```

Our model aims to predict the rank of the future return of holding each asset for one month (i.e., 22 trading days). Let's calculate the future holding return:

```
history = self.history(
    tradable_symbols,
    self._lookback_period + timedelta(1),
    Resolution.DAILY
)
label_by_symbol = {}
for symbol in tradable_symbols[:]:
```

```
# Remove the asset if there is not data for it.
if symbol not in history.index:
    tradable_symbols.remove(symbol)
    continue
open_prices = history.loc[symbol]['open'].shift(-1)
# `shift(-1)` so that the open price here represents the
# fill of a M00 order immediately after this universe
# selection.

# Calculate the future return of holding for 22 full
# trading days (1 month).
label_by_symbol[symbol] = open_prices.pct_change(
    self._prediction_period
).shift(-self._prediction_period).dropna()
```

The `prediction_period` is defined in the algorithm's `initialize` method as:

```
self._prediction_period = 22
```

Notice this line:

```
label_by_symbol[symbol] = open_prices.pct_change(self.prediction_period).shift(-self.prediction_pe
```

We calculate the percentage change between the current row and `prediction_period` away from the prior row. We shift the data backward (i.e., future data will become historical data), and finally, we remove rows with `NaN` values.

After potentially eliminating some factors and assets, we must create a single data frame for the factor matrix and the label vector for training.

```
X_train = pd.DataFrame()
y_train = pd.DataFrame()
for symbol in tradable_symbols:
    labels = label_by_symbol[symbol]
    factors = factors_by_symbol[symbol][factors_to_use].reindex(
        labels.index).ffill()
    X_train = pd.concat([X_train, factors])
    y_train[symbol] = labels
X_train = X_train.sort_index()
```

Now that we have a matrix of factor values, we apply PCA to reduce the dimensionality of the factor matrix to just five factors, and then transform the original data into the space defined by the five principal components.

```
def initialize(self):
    # ...
    self._components = self.get_parameter('components', 5)
    self._scaler = StandardScaler()
    self._pca = PCA(n_components=self._components, random_state=0)
```

And then in `_select_assets`:

```
def _select_assets(self, fundamental):
    # ...
    X_train_pca = self._pca.fit_transform(
        self._scaler.fit_transform(X_train)
    )
```

Before using the LGBRanker algorithm, we must convert the label vector into a ranked one. The ranking algorithm starts with a rank of 1, which we need to convert to 0. In addition, we need to remove NaN values.

```
y_train = y_train.rank(axis=1, method='first').values.flatten() - 1
y_train = y_train[~np.isnan(y_train)]
```

Notice this line:

```
y_train.rank(axis=1, method='first').values.flatten()-1
```

- `y_train.rank(axis=1, method='first')`: This assigns each element along each row (`axis=1`) a rank in the data frame. If some elements have an identical value, the `method='first'` argument will give ranks based on their order (whoever comes first gets a lower rank). The output is another data frame with the same shape as `y_train` but with the original values changed by their ranks.
- `.values`: This converts the data frame into a NumPy array, which is required for the subsequent processing.
- `.flatten()`: This collapses the NumPy array into one dimension. If the original array was 2D (like a data frame), the resulting array is a 1D array with concatenated rows.
- `-1`: This subtracts one from all the elements in the array so that the ranks are zero-based.

Next, we train a learning-to-rank LGBMRanker model with the factors and labels. We train the algorithm to rank “assets” for a given “rebalance date.” Therefore, before training, we count the possible assets for every rebalance date and provide it to the `fit` method. We are using the LambdaRank objective.

```
model = LGBMRanker(
    objective="lambdaRank",
    label_gain=list(range(len(tradable_symbols)))
)
# The `group` is a mapping from rebalance time to the number of
# assets at that time.
group = X_train.reset_index().groupby("time")["time"].count()
model.fit(X_train_pca, y_train, group=group)
```

Now, we can predict the ranking of assets over the upcoming month:

```
X = pd.DataFrame()
for symbol in tradable_symbols:
    X = pd.concat(
        [X, factors_by_symbol[symbol][factors_to_use].iloc[-1:]]
```

```
)
prediction_by_symbol = {
    tradable_symbols[i]: prediction
    for i, prediction in enumerate(
        model.predict(self._pca.transform(self._scaler.transform(X)))
    )
}
```

Let's select the highest-ranking assets:

```
sorted_predictions = sorted(
    prediction_by_symbol.items(), key=lambda x: x[1]
)
```

We select just the best 10 assets with the greatest prediction labels:

```
return [x[0] for x in sorted_predictions[-self._final_universe_size:]]
```

Where `final_universe_size` is defined as 10 in the algorithm's initialize method:

```
self._final_universe_size = self.get_parameter(
    'final_universe_size', 10
```



Example 11—Inverse Volatility Rank and Allocate to Future Contracts

Predicting	Volatility
Technology	Ridge regression
Asset Class	Futures
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example11

Summary

We construct a portfolio of futures contracts weighted by the inverse of the corresponding expected opening price volatility over the next week. The expected opening price volatility over the next week is predicted using a ridge regression model on the past closing price volatility, average true range (ATR), standard deviation, and open interest.

Motivation

The strategy leverages the inverse volatility ranking method, assuming lower volatility assets tend to have more stable and predictable returns.

By weighing assets inversely proportionally to their expected volatility, the strategy inherently allocates more capital to less volatile investments, which tends to reduce the overall risk of the portfolio, and at the same time, this tends to improve the Sharpe ratio of the portfolio.

The trading universe has been chosen to be widely diversified to limit the impact of any single asset’s performance on the overall portfolio.

Model

Model	
Features	• <u>Volatility (standard deviation of daily returns) over the last 3 months.</u> We monitor both the opening and closing price volatility—the future opening volatility will be predicted using the historical closing price volatility. • <u>ATR over the last 3 months:</u> • The true range is the greatest of the following: • The difference between the current high and the current low. • The <i>absolute</i> difference between the current low and the previous close. • The <i>absolute</i> difference between the current high and the previous close. • The average true range is then an exponential moving average of the true range. • <u>Open interest:</u> Open interest is the total number of derivative contracts that are still active and not settled. The contract is considered “open” from when the buyer or seller starts it until the other party ends it.
Predicted Label	Open prices volatility.
Model	Ridge regression.

Ridge regression was chosen for the following reasons:

1. Ridge regression effectively handles **multicollinearity** (when model features could be correlated).
2. Ridge regression **includes a regularization term** (a penalty on the size of coefficients: the coefficients of unimportant factors shrink

more than the coefficients of essential factors), which prevents overfitting and makes them more stable and less sensitive to minor changes in data.

3. Ridge regression is simple and computationally efficient.

Trading Universe

We limit ourselves to the universe of the *front-month* contracts of these futures contracts:

- Indices
 - VIX
 - S&P 500 E-Mini
 - Nasdaq 100 E-Mini
 - DOW 30 E-Mini
- Energy
 - Brent crude
 - Gasoline
 - Heating oil
 - Natural gas
- Grains
 - Corn
 - Oats
 - Soybeans
 - Wheat

Portfolio Construction

Model Training Time	On the first day of every week, 2 minutes after the market open.
Portfolio Rebalancing Time	Immediately after training the model.
Portfolio Weights	The asset weights are a function of their respective volatilities: $\text{weight} = 3/\sigma/\sum(\sigma)/\text{Futures Contract Multiplier}$ This weighting function is designed to give greater weight to less volatile assets. The numerator 3 scales the positions to achieve the desired position size: • If it is set to 1, the algorithm only trades a few of the futures in the universe because of the <code>minimum_order_margin_portfolio_percentage</code> setting. • If it is set too high, we run into margin calls. • Three was the middle ground where the algorithm trades most futures without setting <code>minimum_order_margin_portfolio_percentage</code> to zero.

Tearsheet

The results show the following:

- The Sharpe ratio is typically highest when at least one of the indicators uses a 6-month lookback.
- The Sharpe ratio is typically lowest when at least one of the indicators uses a 3-month lookback.
- All parameter combinations are profitable.
- See [Figures 6.37](#) and [6.38](#).

Parameter `std_months` :

- The minimum is 1 because it is the smallest possible integer.
- The maximum is 6 because it is half a year.
- The step size is one because it is the smallest possible step size.

Parameter `atr_months` :

- The minimum is 1 because it is the smallest possible integer.
- The maximum is 6 because it is half a year.
- The step size is 1 because it is the smallest possible step size.

Used backtest parameters:

- `std_months`: 3
- `atr_months`: 3
- `training_set_duration`: 365

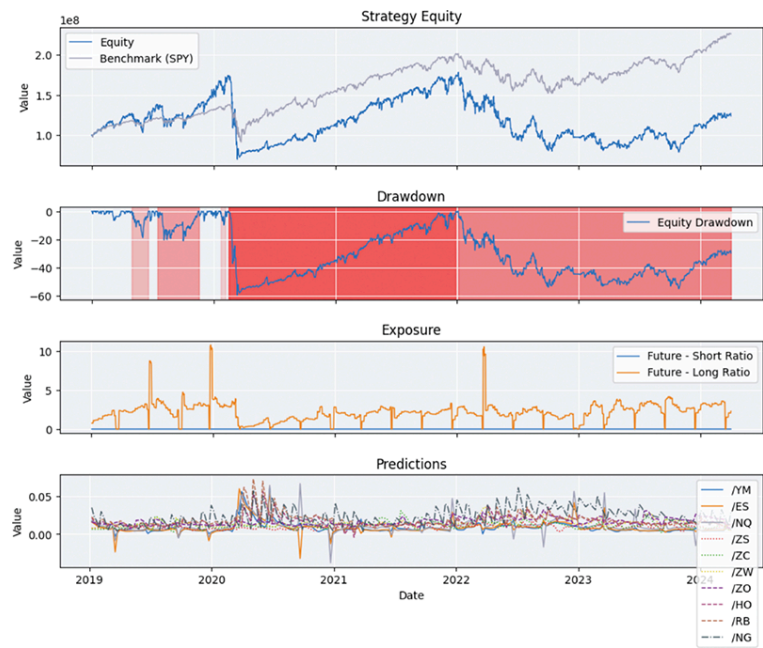


Figure 6.37 Equity curve, performance plots, and custom plots for Example 11.

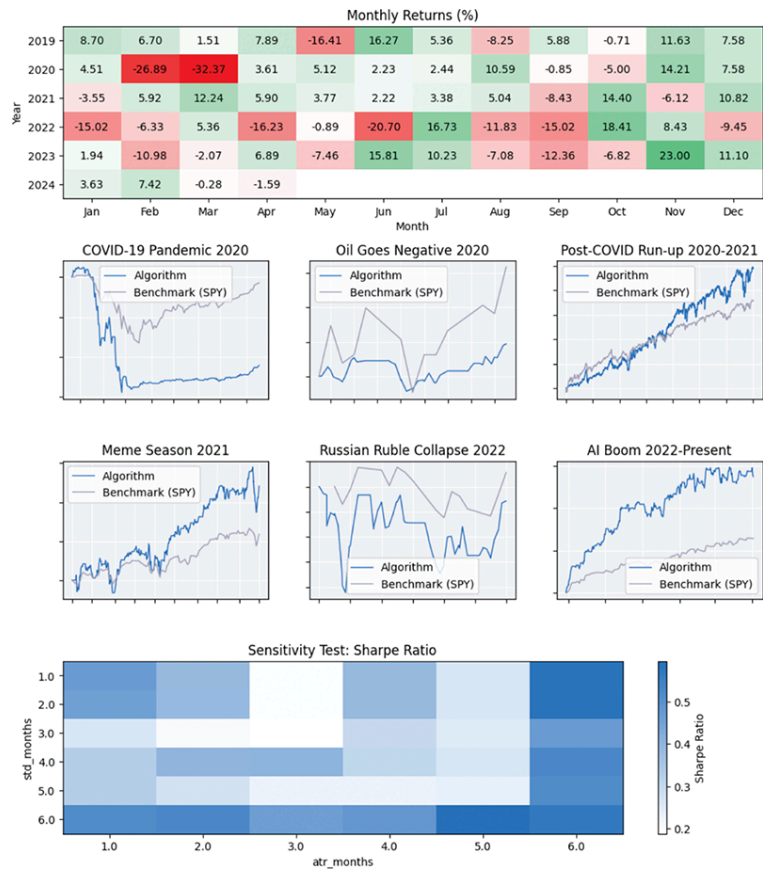


Figure 6.38 Monthly returns, crisis events, and sensitivity tests for Example 11.

Implementation Insights

We implement the volatility model features via the chaining of indicators:

Step 1: We create an indicator for daily closing and opening price returns.

```
security.close_roc = RateOfChange(1)
security.open_roc = RateOfChange(1)
```

Step 2: We create an indicator for the standard deviation of daily closing and opening price returns.

```
security.std_of_close_returns = IndicatorExtensions.of(
    StandardDeviation(self._std_period),
    security.close_roc
)

security.std_of_open_returns = IndicatorExtensions.of(
    StandardDeviation(self._future_std_period), security.open_roc
)
```

We implement the **ATR model feature** via an indicator:

```
security.atr = AverageTrueRange(self._atr_period)
```

Where we defined `self._atr_period` in the algorithm's initialize method to be by default about the number of days in 3 months ($3 * 26$):

```
def initialize(self):
    # ...
    self._atr_period = self.get_parameter('atr_months', 3) * 26
```

We **populate the indicator data** by consolidating the model's features (`indicator_history`) and the dependent variable (`label_history`) into daily bins.

```
security.indicator_history = pd.DataFrame()
security.label_history = pd.Series()

security consolidator = self consolidate(
    security.symbol, Resolution.DAILY, self._consolidation_handler
)
```

For each security, the `indicator_history` will store the daily values of the features `atr`, `std_of_close_returns`. The `label_history` is a series that contains the future volatility of the upcoming week.

```
def _consolidation_handler(self, consolidated_bar):
    security = self.securities[consolidated_bar.symbol]
    t = consolidated_bar.end_time
    if security.atr.update(consolidated_bar):
        security.indicator_history.loc[t, 'atr'] = \
            security.atr.current.value
    security.close_roc.update(t, consolidated_bar.close)
```

```

if security.std_of_close_returns.is_ready:
    security.indicator_history.loc[t, 'std_of_close_returns'] = \
        security.std_of_close_returns.current.value
    security.open_roc.update(t, consolidated_bar.open)

if (security.std_of_open_returns.is_ready and
    len(security.indicator_history.index) > self._future_std_period):
    security.label_history.loc[
        security.indicator_history.index[-self._future_std_period - 1]
    ] = security.std_of_open_returns.current.value

security.indicator_history = security.indicator_history[
    (security.indicator_history.index >=
     self.time - self._training_set_duration)
]
security.label_history = security.label_history[
    (security.label_history.index >=
     self.time - self._training_set_duration)
]

```

Notice the following:

- The indicator's `update()` method returns `True` if the indicator is ready; in this case, we store the successful value in the `indicator_history`.
- There is a special handling for `close_roc` and `open_roc`. For `open_roc`, we proceed differently:
 - We do not check the return value of the `open_roc.update()` because we are interested in the success of the dependent indicator `std_of_open_returns`.
 - We are only interested in predicting the volatility over the upcoming `_future_std_period` trading days (1 week).
 - We shift the future indices into the past in preparation for regression predicting the future opening price volatility.
- Finally, we filter out older data that are not interesting for the training; see `_training_set_duration`.

To train the model, we first retrieve the Open Interest data for all active futures contracts in the portfolio.

```

open_interest = self.history(
    OpenInterest, [c.symbol for c in self._contracts],
    self._training_set_duration, fill_forward=False
)
open_interest.index = open_interest.index.droplevel(0)

```

Notice the data frame's index is a multi-index with three levels (expiry, symbol, time) and we drop the expiry index level.

Then, we construct the factors data frame column-wise (`axis=1`), appending the `open_interest` data frame for the given symbol to the data frame of `indicator_history`.

```

factors = pd.concat(
    [security.indicator_history, open_interest.loc[symbol]],

```

```
axis=1
).ffill().loc[security.indicator_history.index].dropna()
```

We also forward populate missing data (`ffill`) and align the concatenated data frame's index to the index of the `indicator_history`'s `index`, and remove any rows that contain `NaN` (`dropna`).

We also create the labels data structures:

```
label = security.label_history
idx = sorted(
    list(set(factors.index).intersection(set(label.index)))
)
if len(idx) < 20:
    continue
```

We are interested only in the labels associated with the feature dates and only if there are at least 20 values; otherwise, the training process would not have enough data.

Now, we are ready to train the model and store the predictions in the dictionary `expected_volatility_by_security`:

```
model = Ridge()
model.fit(factors.loc[idx], label.loc[idx])

prediction = model.predict([factors.iloc[-1]])[0]
if prediction > 0:
    expected_volatility_by_security[security] = prediction
```

Once we establish the expected volatility, we can place orders to rebalance the portfolio.

Example 12—Trading Costs Optimization

Predicting	Trading costs
Technology	Decision tree regression
Asset Class	Crypto
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example12

Summary

This algorithm demonstrates using machine learning to reduce trading costs. We build a `DecisionTreeRegressor` model to predict the trading

costs with these factors:

- Absolute order quantity.
- Average true range.
- Average daily volume.
- Spread percentage ((ask – bid)/bid).
- Top of book size (in dollars).

Then, we place an order only if the predicted costs per invested dollar are lower than the average trading cost over the last 10 trading days. We store extensive details for further analysis in the research Jupyter Notebook.

Motivation

In financial trading, minimizing costs is crucial for maximizing returns. This trading algorithm leverages machine learning to reduce trading costs by building a `DecisionTreeRegressor` model.

Modeling trading costs involves predicting the expenses associated with executing trades, which various market factors can influence. The algorithm considers the following:

- Absolute order quantity, which is the size of the trade order, directly impacts the cost due to potential market impact and liquidity constraints.
- The ATR is included as it measures market volatility, indicating the potential cost variations based on price movements.
- Average daily volume is critical, as higher trading volumes imply better liquidity and lower costs. In comparison, lower volumes can lead to higher costs due to slippage and market impact.
- Spread percentage ((ask – bid)/bid) is another essential component, representing the difference between the bid and ask prices. A more extensive spread indicates higher costs for executing trades.
- Lastly, the top-of-book size (in dollars) is considered, which reflects the available liquidity at the best bid and ask prices and helps gauge the immediate cost implications for the order size being executed.

Model

Model Features	
• Absolute order quantity. • Average true range. • Average daily volume. • Spread percent ((ask – bid)/bid). • Top of book size (in dollars).	
Predicted Label	Actual trading costs
Model	<code>DecisionTreeRegressor</code>

Trading Universe

We trade only one asset, BTCUSD.

Portfolio Construction

Model Training Time	At midnight on the first day of each month.
Portfolio Rebalancing Time	At midnight of every day, and then with every new market data.
Portfolio Weights	We buy 10 BTC in each entry order.

Trading Logic

The trading algorithm runs in two modes:

- Mode 1 (Benchmark Run):
 - First, we run a benchmark algorithm (set `self._benchmark = True`), which buys 10 Bitcoin at midnight and liquidates the position at 1 am daily.
 - As the benchmark algorithm runs, it records the orders' fill price, quantity, and cost.
 - At the end of the algorithm, all captured data is saved in QuantConnect's Object Store for further analysis in the Jupyter Notebook.
- Mode 2 (Candidate Algorithm—Trading Costs Optimization):
 - After the benchmark algorithm finishes, we can run the candidate algorithm (set `self._benchmark = False`).
 - The candidate algorithm buys 10 Bitcoin at midnight but liquidates it at some point between 1 am and 11:59 pm when it predicts that the liquidation costs are lower than usual.
 - If it gets to 11:59 pm and has not liquidated the position yet, it liquidates the position regardless of the predicted costs; we tag such an order with "Hit time limit."

In other words, the Mode 1 algorithm liquidates the position every day with no intelligence and is used to measure the value of the machine-language (ML) model in Mode 2. The Mode 2 algorithm incorporates some sophistication in finding the ideal time to liquidate the position.

Implementation Insights

We defined a `SpreadSlippageModel` to simulate the impact of market spread on the execution price of trades. We are using the spread as a proxy for a high-cost environment for market orders.

```
class SpreadSlippageModel:
    def get_slippage_approximation(self, asset, order):
        return asset.ask_price - asset.bid_price
```

The Mode 2 (Candidate Algorithm—Trading Costs Optimization) algorithm places the same trades as the Mode 1 (Benchmark Run) version at the beginning of the trading window; once it has gained enough samples to train the ML model, it finally starts delaying trades to reduce costs.

The Candidate algorithm uses a `DecisionTreeRegressor` model to predict the costs of placing a trade.

- We train the model to predict the costs of an order (commission + slip-page), and we capture these in `_costs`.
- We convert the predicted cost of an order to the “predicted cost per dollar traded.”
- The model only trades when the “predicted cost per dollar traded” is lower than the trailing average cost of filled trades.

We capture all orders in a data frame `_order_fills` with the following columns:

- `fill_price`.
- `quantity`.
- `cost`.
- `tag`—used to add comments on the order (e.g., if the trade was executed due to the time limit).

We persist the orders into a CSV file at the end of the algorithm to allow detailed trade analysis. For Mode 1, the file is called `benchmark_order_fills`, while for Mode 2 it is `candidate_order_fills`.

```
def on_end_of_algorithm(self):
    key = ("benchmark" if self._benchmark else "candidate") + "_order_fills"
    self._order_fills.to_csv(self.object_store.get_file_path(key))
```

Running QuantConnect’s backtest of the trading algorithm illustrates in the charts “Costs” and “Costs per Dollar” how the algorithm work; see [Figure 6.39](#).

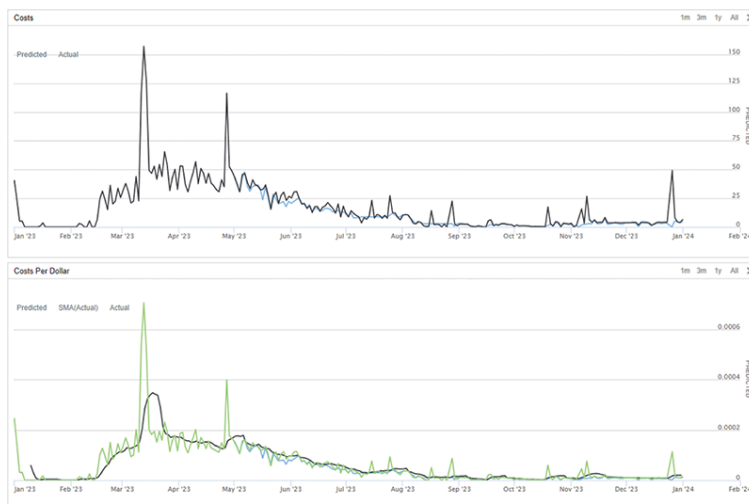


Figure 6.39 Plots of trading costs. Trading costs begin to decrease after the model has gathered a sufficient number of samples.

The Research Jupyter Notebook has been designed to analyze the costs of both backtest modes.

Step 1: Load the orders csv files.

```
qb = QuantBook()
def get_order_fills(key):
    return pd.read_csv(
        qb.object_store.get_file_path(f"{key}_order_fills"),
        index_col=0, parse_dates=True
    )
benchmark_orders = get_order_fills("benchmark")
candidate_orders = get_order_fills("candidate")
```

Step 2: Calculate summary statistics.

```
same_index = candidate_orders.index == benchmark_orders.index
matching_fill_times = candidate_orders[same_index].index
different_fill_times_for_candidate = candidate_orders[~same_index].index
different_fill_times_for_benchmark = benchmark_orders[~same_index].index

print(f"Number of trades: {len(benchmark_orders)}")
print("Number of trades with the same fill times:",
      len(matching_fill_times))
print(
    "Number of trades with different fill times:",
    len(different_fill_times_for_candidate)
)
costs_saved = sum(
    benchmark_orders['cost'].values - candidate_orders['cost'].values
)
print("Costs saved by delaying orders:", f"${round(costs_saved, 2)}")

# For all pairs of orders between the two algorithms, ensure the
# quantities match.
candidate_quantities = candidate_orders['quantity'].values
benchmark_quantities = benchmark_orders['quantity'].values
if not all(candidate_quantities == benchmark_quantities):
    raise Exception('Error: The algorithms traded different quantities')
```

```
# For the orders that fill at the same time for both algorithms, ensure
# the fill price and costs also match.
candidate_info = candidate_orders.loc[matching_fill_times].drop('tag', axis=1)
benchmark_info = benchmark_orders.loc[matching_fill_times].drop('tag', axis=1)
if not all(candidate_info == benchmark_info):
    raise Exception(
        'Error: Fill prices and costs of identical orders do not match'
    )
```

With the output:

```
Number of trades: 1827
Number of trades with the same fill times: 798
Number of trades with different fill times: 1029
Costs saved by delaying orders: $36429.7
```

Step 3: Analyze the delayed orders that filled before the time limit.

Let us compare the costs of executing the delayed orders and benchmark orders (see [Figure 6.40](#)):

```
from collections import Counter
import plotly.graph_objects as go

# Calculate number of orders that the algorithm delayed (and didn't hit
# the time limit).
index_numbers_of_time_limited_orders = []
index_numbers_of_delayed_orders = []
for i in range(len(benchmark_orders.index)):
    # Skip orders that filled at the same time.
    if benchmark_orders.index[i] == candidate_orders.index[i]:
        continue
    # Skip orders that hit the time limit.
    if "Hit time limit" in candidate_orders.iloc[i]['tag']:
        index_numbers_of_time_limited_orders.append(i)
        continue
    index_numbers_of_delayed_orders.append(i)

def display_order_results(title, index_numbers):
    cost_deltas = (
        benchmark_orders.iloc[index_numbers]['cost'].values
        - candidate_orders.iloc[index_numbers]['cost'].values
    )
    candidate_dollar_volume = (
        abs(candidate_orders.iloc[index_numbers]['quantity'])
        * candidate_orders.iloc[index_numbers]['fill_price']
    ).values
    cost_deltas_per_dollar_volume = cost_deltas / candidate_dollar_volume

    fig = go.Figure(data=[go.Histogram(x=cost_deltas_per_dollar_volume, nbinsx=100)])
    fig.update_layout(
        title="Distribution of Costs Saved<br><sup>Costs saved for "</sup>
        + f"Delayed Orders that {title}</sup>",
        xaxis_title="Costs Saved Per Dollar Volume (>0 => Candidate Saved Money)",
        yaxis_title="Count"
    )
    fig.show()
```



```

print("Number of orders:", len(index_numbers))
pct_of_cheaper_orders = (
    len(cost_deltas[cost_deltas > 0])
    / len(candidate_orders) * 100
)
print(
    "Number of orders that lowered costs:",
    len(cost_deltas[cost_deltas > 0]),
    f"({round(pct_of_cheaper_orders, 2)}% of all orders)"
)
pct_of_no_change = (
    len(cost_deltas[cost_deltas == 0])
    / len(candidate_orders) * 100
)
print(
    "Number of orders that didn't change costs:",
    len(cost_deltas[cost_deltas == 0]),
    f"({round(pct_of_no_change, 2)}% of all orders)"
)
pct_of_raised_orders = (
    len(cost_deltas[cost_deltas < 0])
    / len(candidate_orders) * 100
)
print(
    "Number of orders that raised costs:",
    len(cost_deltas[cost_deltas < 0]),
    f"({round(pct_of_raised_orders, 2)}% of all orders)"
)
print(f"Total costs saved: ${round(sum(cost_deltas), 2)}")

display_order_results(
    "Filled Before the Time Limit", index_numbers_of_delayed_orders
)

minutes_delayed = (
    candidate_orders.iloc[index_numbers_of_delayed_orders].index
    - benchmark_orders.iloc[index_numbers_of_delayed_orders].index
)
num_orders_by_delay_duration = pd.Series(
    Counter([x.total_seconds() / 60 for x in minutes_delayed])
).sort_index()

fig = go.Figure(
    go.Scatter(
        x=num_orders_by_delay_duration.index,
        y=num_orders_by_delay_duration.values, mode='lines'
    )
)
fig.update_layout(
    title="Number of Orders Filled Per Delay Duration<br><sup>Most order fill "
        + "within 90 minutes</sup>",
    xaxis_title="Minutes Delayed",
    yaxis_title="Number of Orders Filled"
)
fig.show()

```

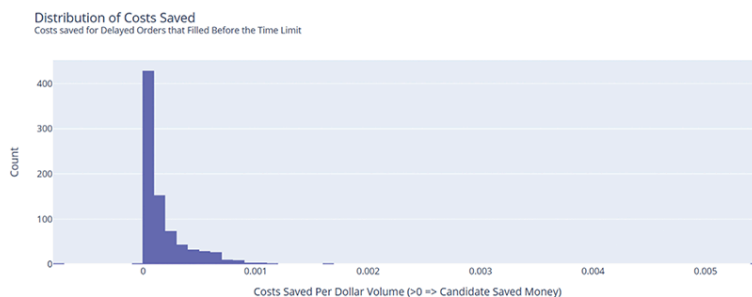


Figure 6.40 Distribution of costs saved for delayed orders that filled before the time limit.

We can see that by delaying the orders, we saved \$36,260.20.

```
Number of orders: 815
Number of orders that lowered costs: 774 (42.36% of all orders)
Number of orders that did not change costs: 28 (1.53% of all orders)
Number of orders that raised costs: 13 (0.71% of all orders)
Total costs saved: $36260.2
```

Figure 6.41 shows how long the orders were delayed:

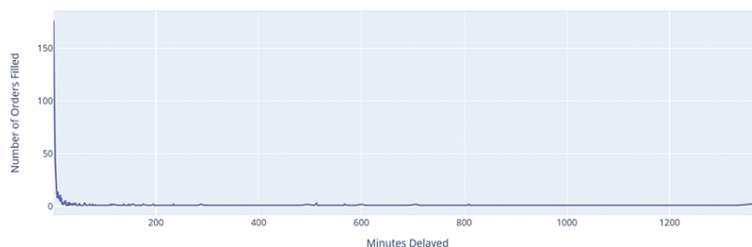


Figure 6.41 Distribution that shows the number of orders that filled at each delay time.

Step 4: Analyze the delayed orders that hit the time limit.

Let us also compare these orders' execution costs versus the execution costs of the benchmark (**Figure 6.42**):

```
display_order_results(
    "Hit the Time Limit", index_numbers_of_time_limited_orders
)
```

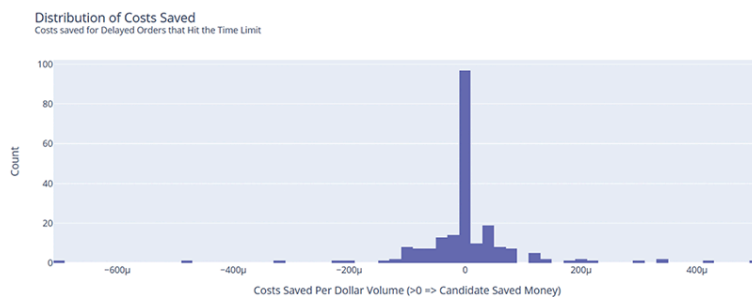


Figure 6.42 Distribution of costs saved for delayed orders that hit the time limit.

```
Number of orders: 214
Number of orders that lowered costs: 90 (4.93% of all orders)
Number of orders that did not change costs: 43 (2.35% of all orders)
```

Number of orders that raised costs: 81 (4.43% of all orders)
Total costs saved: \$169.5



Example 13—PCA Statistical Arbitrage

Mean Reversion

Predicting	Price
Technology	PCA
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example13

Summary

This algorithm describes using principal component analysis and linear regression for statistical arbitrage. Statistical arbitrage strategies use mean-reversion models to exploit price differences between related securities.

Motivation

This algorithm leverages principal component analysis (PCA) and linear regression to implement an effective statistical arbitrage strategy.

The first three PCA's principal components represent the underlying factors driving the price movements of the securities, making them crucial for accurate modeling and prediction. The linear regression model then uses the principal components and historical prices to predict each security's price. By comparing the actual prices to these predictions, the algorithm identifies deviations, which are the regression model's residuals.

Mean-reversion, the central principle of this strategy, posits that prices will revert to their long-term average over time. Therefore, when a security's price deviates significantly from its expected value, as indicated by a large residual, it is likely to revert.

Based on these deviations, the algorithm assigns weights to each stock in the portfolio, positioning itself to profit when the prices move back toward their mean. This systematic approach ensures that the portfolio is dynamically adjusted to capitalize on temporary mispricings, enhancing the potential for profitable trades.

Model

Model Features	The first three PCA's principal components of standardized log returns.
Predicted Label	Standardized log return.
Model	Linear regression.

We chose linear regression since it is simple, easily interpretable, and computationally efficient. However, depending on the specific dataset, we should consider alternatives, such as ridge regression, LASSO regression, random forest regression, SVM regression, or NN, to enhance the model's performance.

Trading Universe

At the start of each month, we select the trading universe defined as the 100 most liquid stocks in the market with their price above 5 USD.

Portfolio Construction

Model Training Time	One minute after the market opens on the first trading day of a month.
Portfolio Rebalancing Time	After model training.
Portfolio Weights	We select only the assets with residuals significantly below the mean residual (their z-score is less than -1.5, corresponding to approximately 93.32nd percentile). The z-score of its residuals weighs each selected stock in the portfolio.

We choose the assets with their most recent z-scores of the residuals much smaller than -1.5 , that is, the assets with residuals significantly below the mean. In other words, we select the assets with log returns much lower than the model predicted. Assuming mean reversion, this indicates the assets with the most likely appreciation.

Trading Logic

At the start of every month, 1 minute after the market has opened, we get the historical price data and run the principal component analysis to transform the trailing prices of all the stocks in the universe to the first three component factors. Then, we fit an ordinary least squares (OLS)

model for each stock using the principal components, choose the stocks with statistically significant residuals, and adjust their portfolio weight (see gnt.co/book-stat-arb-us-equities).

Tearsheet

The results show the following:

- The Sharpe ratio is maximized when `num_components=3` and `lookback_days=126` (6 months).
- The Sharpe ratio is typically the lowest when using 3/4 months for `lookback_days`.
- All parameter combinations are profitable.
- See [Figures 6.43](#) and [6.44](#).

Parameter `num_components` :

- The minimum is 2 because it is the minimum number to perform multiple linear regression.
- The maximum is 6 because each consecutive component explains less of the variance in the data. The first three components already explain more than 90% of the variance, and each additional component explains less than the previous component.
- The step size is 1 because it is the smallest possible step size.

Parameter `lookback_days` :

- The minimum is 21 because it is 1 month of trading days.
- The maximum is 126 because it is 6 months of trading days.
- The step size is 21 because it is the smallest step possible.

Backtest parameters:

- `num_components` : 3
- `lookback_days` : 63
- `z_score_threshold` : 1.5
- `universe_size` : 100

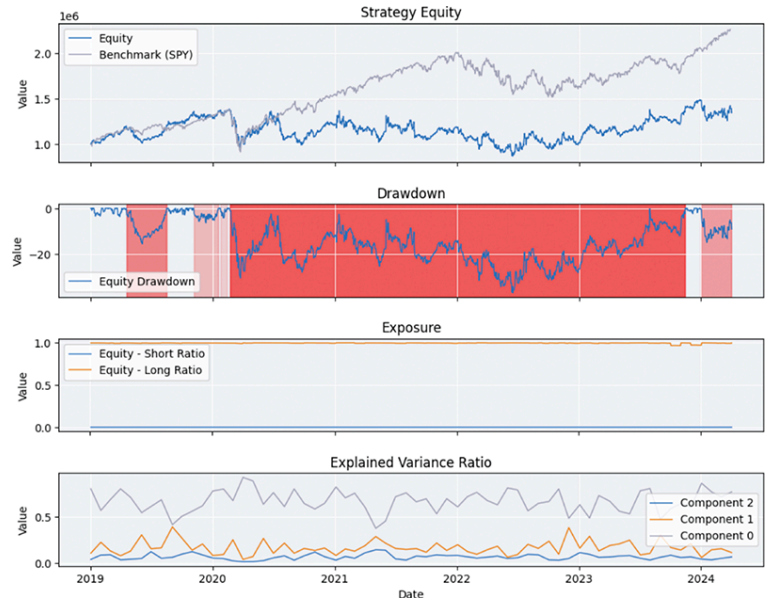


Figure 6.43 Equity curve, performance plots, and custom plots for Example 13.



Figure 6.44 Monthly returns, crisis events, and sensitivity tests for Example 13.

Implementation Insights

The trading algorithm consists of three steps:

Step 1: Get the historical price data.

```
tradeable_assets = [
    symbol
    for symbol in self._universe.selected
    if (self.securities[symbol].price and
```

```

symbol in self.current_slice.quote_bars)

history = self.history(
    tradeable_assets, self._lookback, Resolution.DAILY,
    data_normalization_mode=DataNormalizationMode.SCALED_RAW
).close.unstack(level=0)

```

Notice the `data_normalization_mode` chosen; we require the historical pricing data to be adjusted for splits and dividends.

Step 2: Run PCA and select the assets and their weights according to their level of deviation of the residuals from the linear regression after PCA for each symbol.

```

def _get_weights(self, history):
    # Get sample data for PCA (smooth it using np.log function).
    sample = np.log(history.dropna(axis=1))
    sample -= sample.mean() # Center it column-wise

    # Fit the PCA model for sample data.
    model = PCA().fit(sample)

    # Get the first n_components factors.
    factors = np.dot(sample, model.components_.T)[:,:self._num_components]

    # Add 1's to fit the linear regression (intercept).
    factors = sm.add_constant(factors)

    # Train Ordinary Least Squares linear model for each stock.
    model_by_ticker = {
        ticker: sm.OLS(sample[ticker], factors).fit()
        for ticker in sample.columns
    }

    # Get the residuals from the linear regression after PCA for each stock.
    resids = pd.DataFrame(
        {ticker: model.resid for ticker, model in model_by_ticker.items()}
    )

    # Get the Z scores by standarizing the given pandas dataframe.
    # This is the residual of the most recent day.
    zscores = ((resids - resids.mean()) / resids.std()).iloc[-1]

    # Get the stocks far from their mean (for mean reversion).
    selected = zscores[zscores < -self._z_score_threshold]

    # Return the weights for each selected stock.
    weights = selected * (1 / selected.abs().sum())
    return weights.sort_values()

```

Step 3: Enter the position.

For each selected stock, if the standardized residual deviates from the zero, we enter the position oppositely, expecting mean reversion.

```

self.set_holdings(
    [

```

```
PortfolioTarget(symbol, -weight)
    for symbol, weight in weights.items()
],
True
)
```


Example 14—Temporal CNN Prediction

Predicting	Price direction
Technology	Convolutional neural networks (CNN)
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example14

Summary

We apply a temporal CNN to predict the direction of future stock prices (up, down, stationary) based on trailing open/high/low/close/volume (OHLCV) stock data.

Motivation

Predicting whether stock prices will increase or decrease allows traders to make informed decisions on buying, selling, or holding securities. By entering and exiting positions at the correct times, they can maximize their returns on investment.

This trading algorithm employs a temporal CNN utilizing trailing OHLCV data to forecast whether stock prices will move up, down, or remain stationary.

Temporal CNNs have been chosen for the following reasons:

- Temporal patterns—designed to handle sequential time series data, allowing them to recognize patterns over different time frames.
- Feature extraction—CNNs automatically extract relevant significant features from the raw OHLCV data.
- Non-linear relationships—CNNs can model complex, non-linear relationships in stock price movements.
- Robustness to noise—CNNs filter out noise from the data by focusing only on significant patterns.

CNN provides superior performance for this use case to other models:

- Linear regression would not be suitable due to the non-linear relationships in the stock price data.
- Decision trees or random forests frequently overfit the data and cannot capture temporal patterns.
- Support vector machines can handle non-linear relationships and classification well but provide no natural support for sequential data.

Model

Model Features	For each stock, we use five factors: • Open • High • Low • Close • Volume
Predicted Label	Up/down/stationary
Model	Temporal CNN

We build a temporal CNN to model the price movements of a single stock:

- Temporal CNNs are called temporal since they operate on data that varies over time (“temporal”).
- Temporal CNNs are called convolutional since they implement convolutional layers to extract features.

For each security, as inputs, we use five factors:

```
factor_names = ['open', 'high', 'low', 'close', 'volume']
```

We model three price movement labels as integers:

```
class Direction:
    UP = 0
    DOWN = 1
    STATIONARY = 2
```

We chose the Keras deep learning library for this model.

Input Layer

The input layer shall accept the 15 days of history for each of the factors OHLCV, meaning a shape of the 15 * 5.

```
inputs = Input(shape=(15, 5))
```

We then feed the input layer into the convolutional layer used to extract the features; we split the layer into three temporal regions based on time:

long-term, mid-term, and short-term.

```
long_term = Lambda(
    lambda x: tf.split(x, num_or_size_splits=3, axis=1)[0]
)(feature_extraction)
mid_term = Lambda(
    lambda x: tf.split(x, num_or_size_splits=3, axis=1)[1]
)(feature_extraction)
short_term = Lambda(
    lambda x: tf.split(x, num_or_size_splits=3, axis=1)[2]
)(feature_extraction)

long_term_conv = Conv1D(1, 1, activation='relu')(long_term)
mid_term_conv = Conv1D(1, 1, activation='relu')(mid_term)
short_term_conv = Conv1D(1, 1, activation='relu')(short_term)
```

Next, we combine these three layers and flatten them as we work with 2D matrices.

```
# Combine the three layers back into one.
combined = Concatenate(axis=1)(
    [long_term_conv, mid_term_conv, short_term_conv]
)
flattened = Flatten()(combined)
```

Finally, we define the output layer as having three outputs: up, stationary, and down.

```
outputs = Dense(3, activation='softmax')(flattened)
```

The entire model can be depicted as shown in [Figure 6.45](#).

```
def _create_model(self):
    """Creates the neural network model."""
    inputs = Input(shape=(self._n_tsteps, len(factor_names)))

    # Extract features using a Convolutional layers ("CNN").
    feature_extraction = Conv1D(30, 4, activation='relu')(inputs)

    # Split layer into three regions based on time, ("Temporal").
    long_term = Lambda(
        lambda x: tf.split(x, num_or_size_splits=3, axis=1)[0]
    )(feature_extraction)
    mid_term = Lambda(
        lambda x: tf.split(x, num_or_size_splits=3, axis=1)[1]
    )(feature_extraction)
    short_term = Lambda(
        lambda x: tf.split(x, num_or_size_splits=3, axis=1)[2]
    )(feature_extraction)

    long_term_conv = Conv1D(1, 1, activation='relu')(long_term)
    mid_term_conv = Conv1D(1, 1, activation='relu')(mid_term)
    short_term_conv = Conv1D(1, 1, activation='relu')(short_term)

    # Combine the three layers back into one.
    combined = Concatenate(axis=1)(
```

```
[long_term_conv, mid_term_conv, short_term_conv]
)

# Flattening is required since our input is a 2D matrix.
flattened = Flatten()(combined)

# 1 output neuron for each class (Up, Stationary, Down).
# See the Direction class.
outputs = Dense(3, activation='softmax')(flattened)

# Specify the input and output layers of the model.
self._cnn = Model(inputs=inputs, outputs=outputs)

# Compile the model.
self._cnn.compile(
    optimizer='adam',
    loss=CategoricalCrossentropy(from_logits=True)
)
```

Figure 6.45 Architecture of the temporal CNN model.

Notice we are using the `CategoricalCrossentropy` function designed for multiclass classification problems with one-hot encoded target labels. (We will use the `utils.to_categorical` function for this.)

Trading Universe

At the start of each week, we choose the three components of the QQQ ETF with the most significant weight.

Portfolio Construction

Model Training Time	On the first trading day of each week at 9 am.
Portfolio Rebalancing Time	On the first trading day of each week, 2 minutes after the market opens.
Portfolio Weights	The weights are set by assigning each security a value based on the predicted direction (up or down) and the confidence level of the prediction, adjusting for overall portfolio balance to ensure the total weight absolute sum is lower than or equal to 1.

Trading Logic

We trade at the start of each week 2 minutes after the market opens.

```
def initialize(self):
    #...
```

```

self.schedule.on(
    date_rule, self.time_rules.after_market_open(etf, 2), self._trade
)

def _trade(self):
    # Get predictions for all the assets.
    weight_by_symbol = {}
    for symbol in self._universe.selected:
        security = self.securities[symbol]
        symbol_df = security.history.tail(15)
        prediction, confidence = security.cnn.predict(symbol_df)
        if (prediction != Direction.STATIONARY and
            not math.isnan(confidence) and
            confidence > .55):
            factor = (-1 if prediction == Direction.DOWN else 1)
            weight_by_symbol[security.symbol] = factor * confidence
            self.plot("Confidence", str(security.symbol.id), confidence)

    # Calculate portfolio weights and rebalance.
    weight_sum = sum([abs(x) for x in weight_by_symbol.values()])
    weight_factor = 1 if weight_sum <= 1 else 1 / weight_sum
    portfolio_targets = [
        PortfolioTarget(symbol, weight * weight_factor)
        for symbol, weight in weight_by_symbol.items()
    ]
    self.set_holdings(portfolio_targets, True)

```

The following occurs for each selected security:

- The `trade` method retrieves the OHLCV stock data for the last 15 trading days.
- Then, it calls our model's prediction method.
- If the predicted direction is not `STATIONARY` and has a confidence more significant than 55%, we assign each security a weight of `confidence` for the predicted `UP` direction and `-confidence` for the predicted `DOWN` prediction.
- Finally, we normalize the target weights of each selected security; the weighting ensures that the target weights are proportional to the confidence measure and that they have an absolute sum of 1.

In addition, this trading strategy provides an example of handling of stock splits.

```

def on_splits(self, splits):
    for symbol, split in splits.items():
        if split.type == SplitType.SPLIT_OCCURRED:
            self._initialize_security(self.securities[symbol])

```

Tearsheet

The results show the following:

- The Sharpe ratio typically increases as we increase the number of training samples.
- Most parameter combinations are unprofitable.
- See [Figures 6.46](#) and [6.47](#).

Parameter `training_samples`:

- The minimum is 300 because it is slightly longer than the number of trading days per year.
- The maximum is 700 because it is large but not too large to cause the model training to be too slow.
- The step size is 100 because it generates excellent round numbers.

Parameter `universe_size`:

- The minimum is 2 because we want the algorithm to be always invested. If only one asset and the model predicts a stationary direction, the algorithm will be uninvested for some rebalances.
- The maximum is 10 because the algorithm would take longer than an hour to backtest if we use more than 10.
- The step size is 2 because it gives us five different values of `universe_size` between 2 and 10.

Backtest parameters:

- `training_samples`: 500.
- `universe_size`: 3.

[Figure 6.46](#) Equity curve, performance plots, and custom plots for [Example 14](#).

[Figure 6.47](#) Monthly returns, crisis events, and sensitivity tests for [Example 14](#).

Implementation Insights

We call the `_create_model()` method in the constructor of the model class.

For the model training, we need to encode the time series data into the format accepted by the model:

- We construct the labels to be predicted.
 - We calculate a 5-day rolling average price—we chose five 5-day rolling averages since they are more stable than actual prices.
 - We calculate the return of the rolling average close prices and shift the time series backward as we build a prediction model that predicts the move UP/DOWN in the future based upon older data.
 - Then, if they are above/below the stationarity threshold, we label them as UP/DOWN.
- We create an input 2D matrix of the actual market data, rescale it using the `StandardScaler` to ensure convergence of the training algorithm, and return the result.

```
def _prepare_data(
    self, data, rolling_avg_window_size=5, stationary_threshold=.0001):
    df = data[factor_names]
    shift = -(rolling_avg_window_size - 1)

    def label_data(row):
```

```

        if row['close_avg_change_pct'] > stationary_threshold:
            return Direction.UP
        elif row['close_avg_change_pct'] < -stationary_threshold:
            return Direction.DOWN
        else:
            return Direction.STATIONARY

df['close_avg'] = df['close'].rolling(
    window=rolling_avg_window_size
).mean().shift(shift)
df['close_avg_change_pct'] = \
    (df['close_avg'] - df['close']) / df['close']

df['movement_labels'] = df.apply(label_data, axis=1)

data = []
labels = []
for i in range(len(df)-self._n_tsteps+1+shift):
    label = df['movement_labels'].iloc[i + self._n_tsteps - 1]
    data.append(df[factor_names].iloc[i:i + self._n_tsteps].values)
    labels.append(label)
data = np.array(data)

dim1, dim2, dim3 = data.shape
data = data.reshape(dim1 * dim2, dim3)

data = self._scaler.fit_transform(data)

data = data.reshape(dim1, dim2, dim3)

return data, utils.to_categorical(labels, num_classes=3)

```

Notice that sci-kit's `StandardScaler` works only in 2D, so we must convert the data into 2D temporarily and then back:

```

dim1, dim2, dim3 = data.shape
data = data.reshape(dim1 * dim2, dim3)
data = self._scaler.fit_transform(data)
data = data.reshape(dim1, dim2, dim3)

```

The training code then is simply the preparation of the data and fitting the prepared data into the model created in the constructor of the class.

```

def train(self, data):
    data, labels = self._prepare_data(data)
    self._cnn.fit(data, labels, epochs=20)

```

We chose 20 epochs for demonstration purposes.

Finally, the prediction code is straightforward:

- We standardize the market data by filling up missing data forward (`ffill`)—meaning we propagate the last known value forward.
- We then use the Keras' `prediction` method to return the predicted label and confidence.

```
def predict(self, input_data):
    input_data = self._scaler.transform(
        input_data.fillna(method='ffill').values
    )
    prediction = self._cnn.predict(input_data[np.newaxis, :])[0]
    direction = np.argmax(prediction)
    confidence = prediction[direction]
    return direction, confidence
```



Example 15—Gaussian Classifier for Direction Prediction

Predicting	Returns direction
Technology	Gaussian Naive Bayes classifier
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example15

Summary

We apply Gaussian Naive Bayes (GNB) classifiers to predict the daily returns of the technology stocks class (positive, negative, or flat).

Motivation

This example solves the same problem as the previous example. However, instead of using CNNs, this trading algorithm employs GNB classifiers to forecast the daily returns of technology stocks, categorizing them as positive, negative, or flat.

GNB is particularly well-suited for this task due to its simplicity, efficiency, and effectiveness in handling high-dimensional data.

Unlike CNN, GNB models are simple, computationally efficient, and highly interpretable due to the GNB’s assumption that the features are typically distributed and independent. In addition, GNB relies on manual feature engineering and preprocessing, while CNNs automatically extract the relevant features from raw data.

The choice between GNB and CNN depends on the trading strategy’s specific requirements, including the data’s nature and whether it meets GNB’s assumptions, computational resources, and the importance of interpretability versus predictive power.

Model

Model	Four daily returns (open to close).
Features	
Predicted Label	The signs (-1, 0, 1) of the return in 22 trading days into the future (we intend to hold each stock for 30 calendar days).
Model	GNB classifier.

The model has been inspired by this paper (see qnt.co/book-stock-market-index-direction).

Trading Universe

At the beginning of each week, we choose the 10 largest stocks from the technology sector as defined by Morningstar.

Portfolio Construction

Model Training Time	On the first trading day of each week at 9 am.
Portfolio Rebalancing Time	On the first trading day of each week, 2 minutes after the market opens.
Portfolio Weights	Equally weighted portfolio of all stocks the model identifies as expecting positive return (the sign of +1).

Tearsheet

The results show the following:

- The Sharpe ratio is most significant with a small universe size (5). It is probably because the number of independent variables in the ML model grows as the universe grows, potentially overwhelming the model with noise and not allowing it to learn.
- All parameter combinations are profitable.
- See [Figures 6.48](#) and [6.49](#).

Parameter `days_per_sample` :

- The minimum is 2 because we have two factors for each asset instead of just one.
- The maximum is 8 because anything more significant may cause the number of factors to be so large that it floods the model with noise.
- The step size is 1 because it is the smallest possible step size.

Parameter `universe_size`:

- The minimum is 5 because anything smaller may cause the algorithm to resort to just cash during one of the rebalances.
- The maximum is 25 because including too many assets in the universe will cause the number of independent variables in the ML model to grow very large.
- The step size is 5 because it leads to excellent round numbers between the minimum and maximum.

Backtest parameters:

- `days_per_sample`: 4
- `samples`: 100
- `universe_size`: 10

[Figure 6.48](#) Equity curve, performance plots, and custom plots for [Example 15](#).

[Figure 6.49](#) Monthly returns, crisis events, and sensitivity tests for [Example 15](#).

Implementation Insights

This algorithm illustrates the difference between its backtesting and live modes. For the live mode, the algorithm's field `live_mode` is `True`, and we utilize a pickle file to store the trained models; we load them in the `initialize()` method, update them in the `_train` method, and then save them in the `on_end_of_algorithm` method:

```
def initialize(self):
    # ...
    if self.live_mode:
        self._models_by_symbol = {}
        self._key = 'gnb_models.pkl'
        if self.object_store.contains_key(self._key):
            self._models_by_symbol = pickle.loads(
                self.object_store.read_bytes(self._key)
            )
def _train(self):
    # ...
    for security in self._tradable_securities:
        symbol = security.symbol
        security.model = GaussianNB().fit(
            features.loc[idx], labels_by_symbol[symbol].loc[idx]
        )
        # If we're live trading, save the trained model in case the
        # algorithm is stopped and re-deployed before the
        # `_trade` method runs.
        if self.live_mode:
            key = str(symbol.id)
            self._models_by_symbol[key] = pickle.dumps(security.model)

def on_end_of_algorithm(self):
    if self.live_mode:
        self.object_store.save_bytes(
            self._key, pickle.dumps(self._models_by_symbol)
        )
```

In this example, we utilize customized features—open to close intraday return and the sign of the change of open price from one day to the next day—and for these, we need to consolidate the intraday price data ourselves.

Anytime we add new security to the trading universe, we set up for the security a daily consolidator of the market data and warm up the security's features (`security.features_by_day`) and labels history data structure (`security.labels_by_day`) used to train the model.

```
def on_securities_changed(self, changes):
    for security in changes.added_securities:
        security.model = None
        self._set_up_consolidator(security)
        self._warm_up(security)

    for security in changes.removed_securities:
        self.subscription_manager.remove_consolidator(
            security.symbol, security.consolidator
        )
```

The consolidator's code is called on each market data record received and adjusts the security's data structure used for model training.

```
def _set_up_consolidator(self, security):
    security.consolidator = self consolidate(
        security.symbol, Resolution.DAILY, self._consolidation_handler
    )

def _consolidation_handler(self, bar):
    security = self.securities[bar.symbol]
    time = bar.end_time
    open_ = bar.open
    close = bar.close

    # Update the features.
    open_close_return = (close - open_) / open_
    if not self._update_features(security, time, open_close_return):
        return

    # Update the labels.
    open_days = security.previous_opens[
        (security.previous_opens.index <=
         time - timedelta(self._holding_period))
    ]
    if len(open_days) == 0:
        return
    open_day = open_days.index[-1]
    previous_open = security.previous_opens[open_day]
    open_open_return = (open_ - previous_open) / previous_open
    security.labels_by_day[open_day] = np.sign(open_open_return)
    security.labels_by_day = security.labels_by_day[:-self._samples:]

    security.previous_opens.loc[time] = open_
    security.previous_opens = security.previous_opens[
        -self._holding_period:
    ]
```

```
def _update_features(self, security, day, open_close_return):
    """
    Updates the training data features.

    Inputs
    - day
        Timestamp of when we're aware of the open_close_return
    - open_close_return
        Open to close intraday return

    Returns T/F, showing if the features are in place to start
    updating the training labels.
    """
    security.roc_window = np.append(
        open_close_return, security.roc_window
    )[:self._days_per_sample]

    if len(security.roc_window) < self._days_per_sample:
        return False

    security.features_by_day.loc[day] = security.roc_window
    security.features_by_day = security.features_by_day[
        -(self._samples + self._holding_period + 2):
    ]
    return True
```

The warm-up method sets up the critical data structures for training the model (`labels_by_day` , `features_by_day`) and helping data structures (`roc_window` , `previous_opens`). It prepopulates them with daily historical data, and then leaves them to be updated in real time using the consolidator explained earlier.

```
def _warm_up(self, security):
    security.roc_window = np.array([])
    security.previous_opens = pd.Series()
    security.labels_by_day = pd.Series()
    security.features_by_day = pd.DataFrame(
        {
            f'{security.symbol.id}_{t-{i}}' : []
            for i in range(1, self._days_per_sample + 1)
        }
    )

    # Get historical prices.
    history = self.history(
        security.symbol, self._lookback, Resolution.DAILY,
        data_normalization_mode=DataNormalizationMode.SCALED_RAW
    )
    if history.empty or 'close' not in history:
        self.log(f"Not enough history for {security.symbol} yet")
        return

    # Calculate the features.
    history = history.loc[security.symbol]
    history['open_close_return'] = (
        (history.close - history.open) / history.open
    )

    # Calculate the labels.
```

```

start = history.shift(-1).open
end = history.shift(-22).open # Trading days instead of calendar days.
history['future_return'] = (end - start) / start

for day, row in history.iterrows():
    security.previous_opens[day] = row.open

    # Update the features.
    if not self._update_features(security, day, row.open_close_return):
        continue

    # Update the labels.
    if not pd.isnull(row.future_return):
        security.labels_by_day[day] = np.sign(row.future_return)
        security.labels_by_day = security.labels_by_day[
            -self._samples:
        ]
    security.previous_opens = security.previous_opens[
        -self._holding_period:
    ]

```

Whenever there is a stock split, we must reinitiate the datasets:

```

def on_splits(self, splits):
    for symbol, split in splits.items():
        if split.type != SplitType.SPLIT_OCCURRED:
            continue
        security = self.securities[symbol]
        # Reset the consolidator and warm-up the factors
        # and labels.
        self.subscription_manager.remove_consolidator(
            symbol, security consolidator
        )
        self._set_up_consolidator(security)
        self._warm_up(security)

```

We train the model every trading day at 9 am, and it is run if we have enough data points (`_is_ready` method).

```

def initialize(self):
    # ...

    schedule_symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
    date_rule = self.date_rules.week_start(schedule_symbol)
    self.train(date_rule, self.time_rules.at(9, 0), self._train)

def _is_ready(self, security):
    return (
        security.features_by_day.shape[0] ==
        self._samples + self._holding_period + 2
    )

def _train(self):
    """
    Trains the Gaussian Naive Bayes classifier model.
    """
    features = pd.DataFrame()
    labels_by_symbol = {}

```

```

self._tradable_securities = []
for symbol in self._universe.selected:
    security = self.securities[symbol]
    if self._is_ready(security):
        self._tradable_securities.append(security)
        features = pd.concat(
            [features, security.features_by_day],
            axis=1
        )
        labels_by_symbol[symbol] = security.labels_by_day

# The first and last row can have NaNs because this `_train`
# method fires when the universe changes, which is before the
# consolidated bars close. Let's remove them.
features.dropna(inplace=True)

# Find the intersection of the indices for the features and
# labels.
idx = set([t for t in features.index])
for i, (symbol, labels) in enumerate(labels_by_symbol.items()):
    a = set([t for t in labels.index])
    idx &= a
idx = sorted(list(idx))

for security in self._tradable_securities:
    symbol = security.symbol
    security.model = GaussianNB().fit(
        features.loc[idx], labels_by_symbol[symbol].loc[idx]
    )
    # If we're live trading, save the trained model in case the
    # algorithm is stopped and re-deployed before the
    # `_trade` method runs.
    if self.live_mode:
        key = str(symbol.id)
        self._models_by_symbol[key] = pickle.dumps(security.model)

```

Trading

We trade on every trading day 2 minutes after the market opens.

```

def initialize(self):
    # ...
    schedule_symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
    date_rule = self.date_rules.week_start(schedule_symbol)

    self.schedule.on(
        date_rule,
        self.time_rules.after_market_open(schedule_symbol, 2),
        self._trade
    )

def _trade(self):
    # Get the features.
    features = [[]]
    for security in self._tradable_securities:
        features[0].extend(security.features_by_day.iloc[-1].values)

    # Select the assets that the model predicts will have
    # positive returns.

```

```

long_symbols = []
for security in self._tradable_securities:
    # If the live algorithm hasn't called _train yet, load
    # the trained model from the Object Store.
    key = str(security.symbol.id)
    if (self.live_mode and
        not hasattr(security, 'model') and
        key in self._models_by_symbol):
        security.model = pickle.loads(self._models_by_symbol[key])
    # If the model predicts 1, save this asset to long.
    if security.model.predict(features) == 1:
        long_symbols.append(security.symbol)
if len(long_symbols) == 0:
    return
# Rebalance the portfolio.
weight = 1 / len(long_symbols)
self.set_holdings(
    [PortfolioTarget(symbol, weight) for symbol in long_symbols],
    True
)

```



Example 16—LLM Summarization of Tiingo News Articles

Predicting	News sentiment
Technology	Large language model (LLM)
Asset Class	US equities
Difficulty	Medium hard
Type	Research notebook
Source Code	gnt.co/book-example16

Summary

We build a rolling sentiment OpenAI GPT4 model on alternative news data linked to US tech companies and trade upon the sentiment trend.

Motivation

Traders seek out nontraditional data sources to obtain a competitive advantage by uncovering insights extending past conventional financial indicators.

Sentiment analysis applied to news data captures the market's emotional reaction to events and announcements. It offers early detection of upcoming market trends, including potential downturns, and helps quantify investors' psychological impact of the news.

Model

Model Features	News article
Predicted Label	A score from -10 to 10, where: -10 represents extremely negative sentiment. 0 represents neutral sentiment. 10 represents extremely positive sentiment.
Model	gpt-4

Trading Universe

We are interested in trading only TSLA stock, either holding or shorting it.

Portfolio Construction

Model Training Time	On-demand recalculating in the research notebook.
Portfolio Rebalancing Time	Hourly.
Portfolio Weights	100% of the portfolio held in a long TSLA position if the sentiment score is increasing and we are not yet holding a long TSLA position. 100% of the portfolio held in a short position if the sentiment is negative, is decreasing, and we are not yet short TSLA.

Trading Logic

The research notebook generates a CSV file for each date with three columns:

- `hour` (the index)—each row corresponds to a specific hour.
- `sentiment`—the column holds the aggregated sentiment score for all news articles reviewed during that hour. The sentiment score ranges from -10 (extremely negative) to +10 (extremely positive), with 0 indicating neutral sentiment.
- `volume`—the number of news articles reviewed during that hour.

The trading algorithm monitors the corresponding CSV file in the object store, and on each update, after the market opens, it checks the following:

- If sentiment is flat/increasing and not already long, long. We intentionally do not check if the sentiment is non-negative since, in case it is negative, given the sentiment trend, we expect the sentiment to reverse and then benefit from the upward price trend.
- If the sentiment is negative, decreasing, and not already short, short.

```
if self._roc.current.value >= 0 and not self._tsla.holdings.is_long:
    self.set_holdings(self._tsla.symbol, 1)
elif (sentiment < 0 and
      self._roc.current.value < 0 and
      not self._tsla.holdings.is_short):
    self.set_holdings(self._tsla.symbol, -1)
```

Tearsheet

This strategy outperforms the benchmark TSLA (buy and hold TSLA) in terms of risk-adjusted returns:

- The algorithm achieves a 1.695 Sharpe ratio.
- In contrast, the benchmark (buy and hold TSLA) achieves a -0.06 Sharpe ratio.
- See [Figure 6.51](#).

Figure 6.50 Equity curve, performance plots, and custom plots for [Example 16](#).

[Figure 6.51](#) Monthly returns for [Example 16](#).

Implementation Insights

We start in the **Research Environment (research.ipynb)** to gather the required data, train the model, and store its results in QuantConnect's object store.

Step 1: Subscribe to the news articles for the asset of interest.

We subscribe to the TiingoNews data of TSLA, i.e., news articles that feature TSLA, for the backtesting time window from November 1, 2023, to March 1, 2024.

```
qb = QuantBook()
symbol = qb.add_equity("TSLA").symbol

dataset_symbols = qb.add_data(TiingoNews, symbol).symbol

news_articles = qb.history[TiingoNews](
    dataset_symbols, datetime(2023, 11, 1), datetime(2024, 3, 1),
    Resolution.DAILY
)
```

Step 2: Group the articles by date, remove duplicate articles.

We aim to produce sentiment scores for each hour, and we start by grouping first by date, removing duplicate articles, and storing the results in the dictionary `deduplicated_articles_by_date`.

Step 3: Get hourly sentiment values from OpenAI.

We group the articles by hour for these reasons:

1. Hourly sensitivity scores are more stable.
2. There are fewer calls to OpenAI API.

We pass all the articles within each hour to ChatGPT and ask it to provide an hourly sentiment score with a prompt

```
Article <i> title: <title>
Article <i> description: <Description>
. . .
Review the news titles and descriptions above and then create an aggregated sentiment score which
```

Then, we save the scores into a CSV file in QuantConnect's object store:

```
from openai import OpenAI
client = OpenAI(api_key="<your_api_key>")

# Iterate through each day.
for date, articles in deduplicated_articles_by_date.items():
    print(date)
    # Group this day's articles into hourly buckets.
    Articles_by_hour = {}
    for article in articles:
        hour = article.end_time.hour
        if hour not in articles_by_hour:
            articles_by_hour[hour] = []
        articles_by_hour[hour].append(article)

    # Create a series to hold the sentiment scores for the hours in this
    # day.
    Sentiment_by_hour = pd.DataFrame(dtype=float)
    for hour, articles in articles_by_hour.items():
        # Create a prompt for OpenAI.
        Prompt = ""
        for I, article in enumerate(articles):
            prompt += (
                f"Article {i+1} title: {article.title}\n"
                + f"Article {i+1} description: {article.description}\n\n"
            )
        prompt += (
            "Review the news titles and descriptions above and then create an "
            + "aggregated sentiment score which represents the emotional "
            + "positivity towards TSLA after seeing all of the news articles. "
            + "-10 represents extreme negative sentiment, +10 represents "
            + "extreme positive sentiment, and 0 represents neutral sentiment."
            + " Reply ONLY with the numerical value in JSON format. For "
            + 'example, `{ "sentiment-score": 0 }`'
        )

        # Call the OpenAI API to get the sentiment.
        chat_completion = client.chat.completions.create(
```

```

        messages=[{"role": "user", "content": prompt}],
        model="gpt-4"
    )
    sentiment = json.loads(
        chat_completion.choices[0].message.content
    )['sentiment-score']

    # Save the factors.
    sentiment_by_hour.loc[hour, 'sentiment'] = sentiment
    sentiment_by_hour.loc[hour, 'volume'] = len(articles)

    # Save the dataset file to the Object Store.
    file_path = qb.object_store.get_file_path(
        f"tiingo-{date.strftime('%Y-%m-%d')}.csv"
    )
    sentiment_by_hour.to_csv(file_path)

```

We are ready to process the generated CSV files in QuantConnect's back-testing environment.

Step 4: Apply the trained results in a backtesting algorithm.

Now, we transition to the backtesting environment to build an algorithm that uses the sentiment values from ChatGPT. In the main.py file, we define a custom data class that reads the CSV data from the Object Store and injects it into the algorithm.

```

class TiingoNewsSentiment(PythonData):

    def get_source(self, config, date, is_live):
        return SubscriptionDataSource(
            f"tiingo-{date.strftime('%Y-%m-%d')}.csv",
            SubscriptionTransportMedium.OBJECT_STORE,
            FileFormat.CSV
        )

    def reader(self, config, line, date, is_live):
        # Skip the header line.
        if line[0] == ",":
            return None

        # Parse the CSV line into a list.
        data = line.split(',')

        # Construct the new sentiment data point.
        t = TiingoNewsSentiment()
        t.symbol = config.symbol
        t.time = date.replace(hour=int(data[0]), minute=0, second=0)
        t.end_time = t.time + timedelta(hours=1)
        t.value = float(data[1])
        t["sentiment"] = t.value
        t["volume"] = float(data[2])

        return t

```

We apply a `RateOfChange` indicator to the data set to observe the direction of sentiment changes.

```
def initialize(self):
    self.set_start_date(2023, 11, 1)
    self.set_end_date(2024, 3, 1)
    self.set_cash(100_000)
    self._tsla = self.add_equity("TSLA")
    self._dataset_symbol = self.add_data(
        TiingoNewsSentiment, "TiingoNewsSentiment", Resolution.HOUR
    ).symbol
    self._roc = self.roc(self._dataset_symbol, 2)

    self.set_benchmark(self._tsla.symbol)
```

The trading logic is as follows:

- If sentiment is flat/increasing and we are not already long, we are long.
- If sentiment is negative and is decreasing and we are not already short, we are short.

```
def on_data(self, data):
    # Get the current sentiment.
    if self._dataset_symbol not in data:
        return
    sentiment = data[self._dataset_symbol].value

    # If the market isn't open right now, do nothing.
    if not self.is_market_open(self._tsla.symbol):
        return

    if self._roc.current.value >= 0 and not self._tsla.holdings.is_long:
        self.set_holdings(self._tsla.symbol, -1)
    elif (sentiment < 0 and
          self._roc.current.value < 0 and
          not self._tsla.holdings.is_short):
        self.set_holdings(self._tsla.symbol, -1)
```



Example 17—Head Shoulders Pattern

Matching with CNN

Predicting	Pattern matching
Technology	CNN
Asset Class	Forex
Difficulty	Hard
Type	Research notebook
Source Code	qnt.co/book-example17

Summary

This example introduces a one-dimensional CNN to detect a technical head-and-shoulder (H&S) trading pattern. We construct a synthetic training set, and then train a CNN to detect similar H&S patterns in a forex market.

Motivation

Technical analysis utilizes historical price and volume data to forecast future market movements. It reflects market psychology and the collective behavior of participants, providing rule-based criteria for trading decisions.

The head and shoulders pattern is a technical analysis chart formation used to predict upcoming bullish-to-bearish trend reversals. It is characterized by its distinctive shape, a head surrounded by two shoulders:

- Left shoulder—prices rise to a peak and then subsequently decline.
- Head—after forming the left shoulder, prices peak and then decline again.
- Right shoulder—prices rise but less than in the head and then fall back.
- The neckline acts as a level of support and is drawn, connecting the low points of the left and right shoulders.

The usual way to trade this pattern is to short the asset when the price closes below the neckline after the right shoulder, expecting the price to rise. See [Figure 6.52](#).

[Figure 6.52](#) Head and shoulder pattern and the typical entry timing.

Detecting and acting upon trading patterns can significantly enhance trading performance and profitability. This trading algorithm employs a one-dimensional CNN to detect the head and shoulders pattern. We chose CNNs since they excel at identifying complex patterns within data, automatically learning, extracting relevant features from raw price data, and filtering out noise.

By focusing on the forex market, where price movements are continuous and highly responsive to global events, this algorithm aims to capitalize on the pattern's frequent occurrence and high reliability.

Model

Model Features	25 close pricing data points.
Predicted Label	The probability of each class: • class 0 = no pattern. • class 1 = the pattern is present.
Model	One-dimensional CNN.

The architecture of the sequential (we add layers one by one in sequence) CNN is as follows:

- Input layer accepts 25 data points (features), with each feature being treated as a unique channel.
- A one-dimensional convolutional layer (32 filters or kernels with a kernel size of 5) produces tensors of new features by sliding the filters over the filter. The relu activation feature helps the network learn complex, non-linear patterns in the data.
- Max pooling layer—we reduce the dimension of the outputs from the convolutional layer to 2 and take the max of each pool to make the recognition invariant to scale and reduce the computational complexity.
- Flatten layer that then connects to the output layer.
- The output layer uses the sigmoid activation function, the standard for binary classification problems.

We implement in Keras:

```
Model = Sequential()
model.add(Input(shape=(X_train.shape[1], 1)))
model.add(Conv1D(filters=32, kernel_size=5, activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))
```

Trading Universe

We are interested only in the USD/CAD currency pair.

Portfolio Construction

Model Training Time	We train the model only once on synthetic data in the research notebook.
Portfolio Rebalancing Time	When each monitored closing price arrived.
Portfolio Weights	We take the historical price path over 25–100 days (in steps of 10) and down-sample each one to 25 data points, then input it into the CNN. For each sample that the model is >=50% confident contains the heads and shoulder pattern, we short 10,000 USD of the forex pair.

Trading Logic

For backtesting, we gather daily closing prices of the USD/CAD pair from January 1, 2019, to January 1, 2024.

The trading algorithm responds to the new USD/CAD daily closing prices:

- At each point, we slice the history into various lookback window sizes; the minimum window size is 25 closing prices (the input size for the CNN), and the maximum is 100 closing prices. We increase the window size in step 10.
While the trained CNN model accepts 25 data points as input, the H&S pattern can occur over a more extended period, and thus, we need to down-sample the input data to 25 data points while retaining the general shape of the price path. An analogy of the down-sampling process would be reducing a 1920 × 1080 image to a 1280 × 720 image. The visual of the down-sampled image is almost identical to the original photo but has fewer data points.
This down-sampling process is used to search for the pattern’s presence over several lookback window sizes. Once we down-sample each lookback window’s data points to 25, we standardize them and feed them into CNN to get a prediction.
- The output of the model is the probability of each class (class 0 = no pattern; class 1 = the pattern is present).
- For each lookback window, when the model returns a probability of 50% or higher for class 1, we short 10,000 USD and hold the position for 10 days.

The trading code:

```
def initialize(self):
    self.set_start_date(2019, 1, 1)
    self.set_end_date(2024, 4, 1)
    self.set_cash(100_000)
    self._security = self.add_forex("USDCAD", Resolution.DAILY)
    self._symbol = self._security.symbol
```

```

self._max_size = self.get_parameter('max_size', 100)
self._step_size = self.get_parameter('step_size', 10)
self._confidence_threshold = self.get_parameter(
    'confidence_threshold', 0.5
) # 0.5 => 50%
self._holding_period = timedelta(
    self.get_parameter('holding_period', 10)
)

self._model = load_model(
    self.object_store.get_file_path("head-and-shoulders-model.keras")
)
self._trailing_prices = pd.Series()
self._liquidation_quantities = []

def on_data(self, data):
    t = self.time
    price = data[self._symbol].close

    # Update the trailing window.
    self._trailing_prices.loc[t] = price
    self._trailing_prices = self._trailing_prices.iloc[-self._max_size:]

    # Calculate the order quantity.
    quantity = 0

    for size in range(self._min_size, self._max_size + 1, self._step_size):
        # Ensure there are enough trailing data points to fill this
        # window size.
        if len(self._trailing_prices) < size:
            continue

        window_trailing_prices = self._trailing_prices.iloc[-size:]
        # Downsample the trailing prices in this window to be 25
        # data points.
        low_res_window = downsample(window_trailing_prices.values)
        # Standardize the downsampled trailing prices.
        factors = np.array(
            (
                (low_res_window - low_res_window.mean())
                / low_res_window.std()
            ).reshape(1, self._min_size, 1)
        )
        # Get the probability of the technical trading pattern in
        # the downsampled and standardized window.
        prediction = self._model.predict(factors, verbose=0)[0][0]
        if prediction > self._confidence_threshold:
            self.log(
                f"{t}: Pattern detected between "
                + f"{window_trailing_prices.index[0]} and "
                + f"{window_trailing_prices.index[-1]} with "
                + f"{round(prediction * 100, 1)}% confidence."
            )

            quantity -= 10_000

    if quantity:
        self._cad_before_sell = self.portfolio.cash_book['CAD'].amount
        # Place the entry order.
        self.market_order(self._symbol, quantity)

```

```

# Schedule the exit order.
t_exit = t + self._holding_period
self.schedule.on(
    self.date_rules.on(t_exit.year, t_exit.month, t_exit.day),
    self.time_rules.at(t_exit.hour, t_exit.minute),
    self._liquidate_position
)

def _liquidate_position(self):
    quantity = round(
        self._liquidation_quantities.pop(0) / self._security.ask_price
    )
    if quantity:
        self.market_order(self._symbol, quantity)

def on_order_event(self, order_event):
    # When the entry order fills, record the amount of CAD
    # traded so that we can liquidate the correct amount later.
    if (order_event.status == OrderStatus.FILLED and
        order_event.direction == OrderDirection.SELL):
        self._liquidation_quantities.append(
            self.portfolio.cash_book['CAD'].amount - self._cad_before_sell
        )

```

Tearsheet

The results show the following:

- The Sharpe ratio increases with the holding period.
- The Sharpe ratio decreases with the confidence threshold (probably since there are so few trades).
- All parameter combinations are profitable.
- See [Figures 6.53](#) and [6.54](#).

Parameter `confidence_threshold`:

- The minimum is 0.3 because going lower than that means <30% confidence; we found the pattern, which is low/unlikely.
- The maximum is 0.9 because there are already only a few trades.
- The step size is 0.1 because it leads to excellent round values.

Parameter `holding_period`:

- The minimum is 2 because it is the shortest holding period with daily data.
- The maximum is 10 because of our experience.
- The step size is 1 because it is the smallest possible step size.

Backtest parameters:

- `max_size`: 100
- `step_size`: 10
- `confidence_threshold`: 0.8
- `holding_period`: 10

[Figure 6.53](#) Equity curve, performance plots, and custom plots for [Example 17](#).

[Figure 6.54](#) Monthly returns, crisis events, and sensitivity tests for [Example 17](#).

Implementation Insights

We trained the neural network in the research notebook.

Step 1: Create a synthetic input dataset.

We start by creating 100,000 samples of the head and shoulders pattern.

See [Figure 6.55](#).

[Figure 6.55](#) Line plot of our synthetic H&S pattern.

Each sample consists of 25 data points, but each sample has an element of randomness to make it unique in the training set. We standardize each sample in preparation for training the CNN.

```
from scipy.stats import norm, uniform

np.random.seed(1)
ref_count = 100_000
v1 = np.array([0] * ref_count) + 0.02 * norm.rvs(size=(ref_count,))
p1 = np.array([1] * ref_count) + 0.2 * norm.rvs(size=(ref_count,))
v2 = v1 + 0.2 * norm.rvs(size=(ref_count,))
v3 = v1 + 0.2 * norm.rvs(size=(ref_count,))
p3 = p1 + 0.02 * norm.rvs(size=(ref_count,))
p2 = 1.5 * np.maximum(p1, p3) + abs(uniform.rvs(size=(ref_count,)))
v4 = v1 + 0.02 * norm.rvs(size=(ref_count,))
ref = pd.DataFrame([
    v1,
    (v1*.75 + p1*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    (v1+p1)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v1*.25 + p1*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    p1,
    (v2*.25 + p1*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    (v2+p1)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v2*.75 + p1*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    v2,
    (v2*.75 + p2*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    (v2+p2)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v2*.25 + p2*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    p2,
    (v3*.25 + p2*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    (v3+p2)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v3*.75 + p2*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    v3,
    (v3*.75 + p3*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    (v3+p3)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v3*.25 + p3*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    p3,
    (v4*.25 + p3*.75) + 0.2 * norm.rvs(size=(ref_count,)),
    (v4+p3)/2 + 0.2 * norm.rvs(size=(ref_count,)),
    (v4*.75 + p3*.25) + 0.2 * norm.rvs(size=(ref_count,)),
    v4
])
```

```

ref = ((ref - ref.mean()) / ref.std()).T

positive_samples = []
for _, row in ref.iterrows():
    positive_samples.append(row.values)

```

We generate the vertical coordinates of the points `v1`, `p1`, `v2`, `p2`, `v3`, `p3`, and `v4`, and for each consecutive pair of the key points, we put in three points interpolating the line between them. In total, each sample has 25 data points.

For example, we put in three points between the points `v1` and `p1`:

```

(v1*.75 + p1*.25) + 0.2 * norm.rvs(size=(ref_count,)),
(v1+p1)/2 + 0.2 * norm.rvs(size=(ref_count,)),
(v1*.25 + p1*.75) + 0.2 * norm.rvs(size=(ref_count,)),

```

Notice that we generate 100,000 such points with random noise from normal (Gaussian) distribution (`norm.rvs`).

Next, we create 100,000 samples of 25 price paths, which are random walks.

```

equity_curves = (
    np.random.randn(ref.shape[0], ref.shape[1]) / 1000 + 1
).cumprod(axis=1)
negative_samples = (
    (equity_curves - np.mean(equity_curves, axis=1, keepdims=True))
    / np.std(equity_curves, axis=1, keepdims=True)
)

```

We organize the training set to have one positive sample (the simulated H&S pattern), one negative sample, one positive sample, and so on...

```

X = np.array(
    [
        value
        for pair in zip(positive_samples, negative_samples)
        for value in pair
    ]
)
y = np.array([1 if i % 2 == 0 else 0 for i in range(len(X))])

```

Step 2: Split the input dataset into a training and testing dataset.

We split the 200,000 samples into training and testing sets (80%/20% split).

```

from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Input
from keras.utils import set_random_seed

```

```

set_random_seed(0)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

```

Step 3: Create the neural network model.

We implement the sequential neural network model.

```

model = Sequential()
model.add(Input(shape=(X_train.shape[1], 1)))
model.add(Conv1D(filters=32, kernel_size=5, activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

```

Step 4: Compile, train, and evaluate the model.

We compile, fit, and evaluate the model on the test set. On the test set, it achieves an accuracy of 99.9%.

```

# Compile the model.
model.compile(
    optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']
)

# Train the model.
model.fit(
    X_train, y_train, epochs=10, batch_size=16,
    validation_data=(X_test, y_test)
)

# Evaluate the model.
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {test_acc}')

```

Step 5: Save the model to the object store.

```

model.save(qb.object_store.get_file_path("head-and-shoulders-model.keras"))

```

Once we save the model, we can use it in a backtest.



Example 18—Amazon Chronos Model

Predicting	Future price path
Technology	Amazon Chronos
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	gnt.co/book-example18

Summary

This example demonstrates an application of a pre-trained HuggingFace “amazon/chronos-t5-tiny” model to forecast the future performance of the five most liquid assets in the market. Using these forecasts, the algorithm employs the SciPy package to optimize the portfolio weights to maximize the future risk-adjusted returns as measured by the Sharpe ratio. The portfolio is rebalanced every 3 months to align with the latest market predictions and conditions.

Motivation

This example builds upon two powerful components: the advanced forecasting capabilities of the Amazon Chronos model and the robust portfolio optimization using the Sharpe ratio.

The pre trained HuggingFace Amazon Chronos model, specifically the “amazon/chronos-t5-tiny,” is utilized for its superior performance in time-series forecasting. By leveraging a pre trained model, the algorithm saves considerable time and resources that would otherwise be spent on training, allowing for quick deployment and immediate application. The model’s scalability enables it to handle large datasets on powerful hardware such as graphics processing units (GPUs). Compared to traditional models like ARIMA or other AI solutions like long short-term memory (LSTM) networks, Amazon Chronos specializes in financial market predictions.

The algorithm employs the Sharpe ratio for portfolio optimization to maximize risk-adjusted returns, ensuring that the portfolio is optimized for both returns and risk and aligning with modern portfolio theory’s best practices. However, the Sharpe ratio assumes normally distributed returns, which may not always reflect real-world conditions and can sometimes overlook extreme events or tail risks. Alternatives like the Sortino ratio, which focuses on downside risk, value at risk (VaR), which estimates potential loss, and maximum drawdown, which measures the most

significant peak-to-trough decline, offer different perspectives but do not integrate return and risk as comprehensively as the Sharpe ratio.

Model

Model	Daily closing prices.
Features	
Predicted Label	Daily closing prices over the next 3 months (3 * 21 days).
Model	Amazon Chronos.

Amazon Chronos is a state-of-the-art machine-learning model specialized for financial market predictions, offering high accuracy, scalability, and ease of use.

This example implements two trading strategies, one using the base and one with a fine-tuned Amazon Chronos model.

The critical alternative forecasting models are ARIMA, Prophet, LSTM, support vector machines, and XGBoost:

- Autoregressive integrated moving average (ARIMA) is a widely used time-series forecasting model that is particularly effective for stationary data with linear patterns.
- Prophet (by Facebook) is a model designed to handle seasonality and trends.
- Long short-term memory (LSTM) is a neural network model that captures long-term dependencies in sequential data. LSTM is a type of recurrent neural network.
- Support vector machines (SVM) is a machine-learning model effective for forecasting time series by handling high-dimensional data and capturing non-linear relationships through kernel functions.
- XGBoost is a gradient-boosting algorithm that excels at handling tabular data and can be customized for time-series forecasting.

Choosing a suitable forecasting model depends on the specific use case, and exploring alternative models is always recommended.

Trading Universe

At the beginning of each month, the algorithm selects the universe of the top five most liquid assets from SPY, an ETF tracking the S&P 500 index.

Portfolio Construction

Model Training Time	The first trading strategy uses the base Amazon Chronos model with no training. The second trading strategy fine-tunes the Amazon Chronos model at midnight on the first trading day of each month.
Portfolio Rebalancing Time	Midnight on the first trading day of every third month (quarterly).
Portfolio Weights	The weights are determined by maximizing the forward Sharpe ratio.

Trading Logic

The first trading strategy downloads the history of closing prices of the assets in the universe for the last year, calculates the median predicted price for each day in the next 3 months, and sets the portfolio weights from the portfolio optimization results by maximizing the Sharpe ratio.

The second trading strategy enhances the first trading strategy by retraining the Amazon Chronos model before predicting the closing daily prices for the next 3 months.

The portfolio optimization we are solving aims at finding the best combination of assets to maximize returns for a given level of risk.

Formally, the objective function is

$$\max \frac{R_p - R_f}{\sigma_p} \Bigg)$$

Subject to these constraints:

- The sum of portfolio weights equals 1: $\sum w_i = 1$.
- No short selling: $0 \leq w_i \leq 1$.

Where:

- R_p is the return of the portfolio.
- R_f is the risk-free rate, which is the primary credit rate from the Federal Open Market Committee.
- σ_p is the standard deviation of the portfolio's returns.
- w_i are the portfolio weights.

In Python, we solve using SciPy's `minimize` function: we convert the maximization problem into a minimization problem by multiplying the objective function by `-1`. Sequential least squares programming is a popular algorithm for this task, effectively handling the constraints and finding the optimal weights.

Tearsheet

The pretrained HuggingFace models may contain lookahead bias if they were originally trained on data after January 1, 2019.

-
- Trading Strategy 1 (Base Model):
 - See [Figures 6.56](#) and [6.57](#).
-

[Figure 6.56](#) Equity curve and performance plots for Example 18.1.

[Figure 6.57](#) Monthly returns and crisis event for Example 18.1.

-
- Trading Strategy 2 (Fine-Tuned Model):
 - The `context_length` argument defines the length of the context window, which determines how many past tokens are used to predict the subsequent tokens. The default is 512, but we reduced it to 126 (6 months) since we only provided the model with 252 data points (1 year) for each asset.
 - The `max_steps` argument defines the maximum number of training steps to perform. Training stops after this number of steps. The default is 200,000, but we reduced it to 3 to ensure the algorithm finishes training within the 10-minute quota.
 - See [Figures 6.58](#) and [6.59](#).
-

[Figure 6.58](#) Equity curve and performance plots for Example 18.2.

[Figure 6.59](#) Monthly returns and crisis event for Example 18.2.

Implementation Insights

The Amazon Chronos model produces a range of forecasted values for each future close price, with each value assigned its probability. For simplicity, we take the median forecast.

```
# Get historical equity curves.
history = self.history(symbols, self._lookback_period)['close'].unstack(0)

# Forecast the future equity curves.
all_forecasts = self._pipeline.predict(
    [
        torch.tensor(history[symbol].dropna())
        for symbol in symbols
    ],
    self._prediction_length
)

# Take the median forecast for each asset.
forecasts_df = pd.DataFrame(
    {
```

```

        symbol: np.quantile(
            all_forecasts[i].numpy(), 0.5, axis=0 # 0.5 = median
        )
        for i, symbol in enumerate(symbols)
    }
)

```

Where the model is either a pretrained default model (the base model).

```

# Load the pre-trained model.
self._pipeline = ChronosPipeline.from_pretrained(
    "amazon/chronos-t5-tiny",
    device_map="cuda" if torch.cuda.is_available() else "cpu",
    torch_dtype=torch.bfloat16,
)

```

Or is the output of the fine-tuned model trained on the actual data considered in this example.

```

# Gather the training data.
training_data_by_symbol = {}
for symbol in symbols:
    df = history[[symbol]].dropna()
    if df.shape[0] < 10: # Skip this asset if there is very little data
        continue
    adjusted_df = df.reset_index()[['time', symbol]]
    adjusted_df = adjusted_df.rename(columns={str(symbol.id): 'target'})
    adjusted_df['time'] = pd.to_datetime(adjusted_df['time'])
    adjusted_df.set_index('time', inplace=True)
    adjusted_df = adjusted_df.resample('D').asfreq()
    training_data_by_symbol[symbol] = adjusted_df
tradable_symbols = list(training_data_by_symbol.keys())

# Fine-tune the model.
output_dir_path = self._train_chronos(
    list(training_data_by_symbol.values()),
    context_length=int(252/2), # 6 months
    prediction_length=self._prediction_length,
    optim=self._optimizer,
    model_id=self._model_name,
    output_dir=self._model_path,
    learning_rate=1e-5,
    # Requires Ampere GPUs (e.g., A100)
    tf32=False,
    max_steps=3
)

# Load the fine-tuned model.
pipeline = ChronosPipeline.from_pretrained(
    output_dir_path,
    device_map=self._device_map,
    torch_dtype=torch.bfloat16,
)

```

Where the method `_train_chronos` executes the training.

The portfolio optimization is implemented by maximizing the forecasted Sharpe ratio of the portfolio:

```
optimal_weights = self._optimize_portfolio(forecasts_df)
```

Where the method `_optimize_portfolio` runs the SciPy's minimize function:

```
def _sharpe_ratio(
    self, weights, returns, risk_free_rate, trading_days_per_year=252):
    # Define how to calculate the Sharpe ratio so we can use
    # it to optimize the portfolio weights.

    # Calculate the annualized returns and covariance matrix.
    mean_returns = returns.mean() * trading_days_per_year
    cov_matrix = returns.cov() * trading_days_per_year

    # Calculate the Sharpe ratio.
    portfolio_return = np.sum(mean_returns * weights)
    portfolio_std = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
    sharpe_ratio = (portfolio_return - risk_free_rate) / portfolio_std

    # Return negative Sharpe ratio because we minimize this
    # function in optimization.
    return -sharpe_ratio

def _optimize_portfolio(self, equity_curves):
    returns = equity_curves.pct_change().dropna()
    num_assets = returns.shape[1]
    initial_guess = num_assets * [1. / num_assets,]
    # Find portfolio weights that maximize the forward Sharpe
    # ratio.
    result = minimize(
        self._sharpe_ratio,
        initial_guess,
        args=(
            returns,
            self.risk_free_interest_rate_model.get_interest_rate(self.time)
        ),
        method='SLSQP',
        bounds=tuple((0, 1) for _ in range(num_assets)),
        constraints=(
            {'type': 'eq', 'fun': lambda weights: np.sum(weights) - 1}
        )
    )
    return result.x
```



Example 19—FinBERT Model

Predicting	News sentiment
Technology	LLM
Asset Class	US equities
Difficulty	Hard
Type	Full strategy
Source Code	qnt.co/book-example19

Summary

This algorithm demonstrates applying a pretrained “ProsusAI/finbert” HuggingFace model to assess the sentiment of the latest news releases related to an asset. At the beginning of each month, the algorithm selects the most volatile asset from the top 10 most liquid assets and computes an aggregated sentiment score based on news releases from the past 10 days. If the sentiment is more positive than negative, the algorithm initiates a long position; otherwise, it enters a short position. Similar to [Example 18](#), we demonstrate the base model and then fine-tuned the model.

Motivation

Leveraging advanced natural language processing (NLP) models to analyze market sentiment can provide a significant edge, especially in uncovering upcoming trends. Positive sentiment typically leads to price increases, while negative sentiment can signal potential declines.

This algorithm utilizes the pre trained “ProsusAI/finbert” model from HuggingFace to assess the sentiment of the latest news releases related to the most volatile asset of the top 10 most liquid assets in the S&P 500 index. Volatility serves as an indicator of potential significant price movements. The algorithm ensures it can capitalize on the highest potential for substantial returns by targeting the most volatile assets. Additionally, selecting from the most liquid assets ensures ease of entry and exit from positions, minimizing slippage and transaction costs.

Model

Model	News article.
Features	
Predicted Label	Confidence probabilities for each sentiment category: positive, neutral, negative. The positive sentiment indicates a favorable outlook for the asset in the text. The negative sentiment indicates an unfavorable outlook, while the neutral sentiment indicates an indifferent outlook.
Model	FinBERT.

In [Example 16](#), we have covered sentiment analysis with OpenAI’s GPT-4 model. When comparing FinBERT or Bidirectional Encoder Representations from Transformers (BERT) models with OpenAI’s GPT-4 models, several distinctions emerge in terms of performance and cost:

- **Performance**—FinBERT, a variant of BERT fine-tuned specifically for financial sentiment analysis, excels in understanding the context and nuances of financial news. FinBERT and BERT models typically provide faster inference speeds because they are smaller than larger models like GPT-4.
On the other hand, GPT-4, as a general-purpose model, enables deeper contextual understanding and generates more nuanced interpretations of news articles. However, this comes at the cost of increased computational resources and processing time.
- **Costs**—FinBERT and BERT models are more cost-effective, requiring less computational power for training and inference, which translates to lower operational costs. Their implementation is also more straightforward, reducing development time and associated costs.
Conversely, the larger size and complexity of GPT-4 demands more computational resources, leading to higher operational costs. While GPT-4’s versatility is advantageous, integrating it for financial sentiment analysis might require additional fine-tuning, further increasing development costs.

We implement the trading strategy in both the base FinBERT model (using the pre trained model) and a fine-tuned FinBERT model. (We retrain the model using the actual articles and judging the sentiment by the following stock return.)

Trading Universe

On the first trading day of each month, we select the most volatile stock from the top 10 most liquid S&P Index components.

Portfolio Construction

Model Training Time	For the base model, no further training is performed. For the fine-tuned model, we retrain the model at midnight on the first trading day of each month.
Portfolio Rebalancing Time	At midnight on the first trading day of each month.
Portfolio Weights	If the aggregate sentiment is more likely positive than negative, we hold 100% of our portfolio in the asset. Otherwise, we short ¼ of our portfolio in the asset.

Trading Logic

In the base FinBERT model, we get the last 10 days of news articles related to the asset in the universe from TiingoNews, tokenize the articles, and apply the pre trained model.

```
# Get the target security.
security = self.securities[list(self._universe.selected)[0]]

# Get the latest news articles.
articles = self.history[TiingoNews](security.dataset_symbol, 10, Resolution.DAILY)
article_text = [article.description for article in articles]
if not article_text:
    return

# Prepare the input sentences.
inputs = self._tokenizer(article_text, padding=True, truncation=True, return_tensors='tf')

# Get the model outputs.
outputs = self._model(**inputs)

# Apply softmax to the outputs to get probabilities.
scores = tf.nn.softmax(outputs.logits, axis=-1).numpy()
self.log(f "{str(scores)}")
scores = self._aggregate_sentiment_scores(scores)

# Rebalance the portfolio.
weight = 1 if scores[2] > scores[0] else -0.25
self.set_holdings(security.symbol, weight, True)
self._last_rebalance_time = self.time
```

Where `_aggregate_sentiment_scores` calculates an exponentially weighted score for the articles, giving more importance to the recent sentiment scores (latest news is more valuable), and then aggregates it into a single score.

In the fine-tuned FinBERT model, we get the last 30 days of close prices and the 30 days of articles.

In the training dataset we generate, we label each day's sentiment using this logic:

1. We calculate the daily return of the close prices.
2. We sort the positive daily returns in descending order and select the first 75% as having the positive label.
3. Similarly, we sort the negative daily returns in ascending order and select the first 75% as having the negative label.
4. The non-assigned values are labeled with the neutral label.

We retrain the model using this training dataset and proceed like in the Base Model.

Tearsheet

The pretrained HuggingFace models may contain lookahead bias if they were originally trained on data after January 1, 2019.

-
- Trading Strategy 1 (Base Model):
 - See [Figures 6.60](#) and [6.61](#).
-

[Figure 6.60](#) Equity curve, performance plots, and custom plots for Example 19.1.

[Figure 6.61](#) Monthly returns and crisis events for Example 19.1.

-
- Trading Strategy 2 (Fine-Tuned Model):
 - See [Figures 6.62](#) and [6.63](#).
-

[Figure 6.62](#) Equity curve, performance plots, and custom plots for Example 19.2.

[Figure 6.63](#) Monthly returns and crisis events for Example 19.2.

Implementation Insights

FinBERT uses a random numbers generator for data splits, weight initialization, and training. For reproducibility, we recommend setting the initial random seed, for example, like this:

```
set_seed(1, True)
```

We aggregate the sentiment scores using exponential weighting to ensure the latest news sentiment has the most significant weight in the aggregated score.

```
def _aggregate_sentiment_scores(self, sentiment_scores):
    n = sentiment_scores.shape[0]
```

```

# Generate exponentially increasing weights.
weights = np.exp(np.linspace(0, 1, n))

# Normalize weights to sum to 1.
weights /= weights.sum()

# Apply weights to sentiment scores.
weighted_scores = sentiment_scores * weights[:, np.newaxis]

# Aggregate weighted scores by summing them.
aggregated_scores = weighted_scores.sum(axis=0)

return aggregated_scores

```

In Trading Strategy 2 (Fine-Tuned Model), we create the training dataset, train the model, and proceed as in Trading Strategy 1 (Base Model).

```

security = self.securities[list(self._universe.selected)[0]]

# Get samples to fine-tune the model
samples = pd.DataFrame(columns=['text', 'label'])
news_history = self.history(security.dataset_symbol, 30, Resolution.DAILY)
if news_history.empty:
    return
news_history = news_history.loc[security.dataset_symbol]['description']
asset_history = self.history(
    security.symbol, timedelta(30), Resolution.SECOND
).loc[security.symbol]['close']
for i in range(len(news_history.index)-1):
    # Get factor (article description).
    factor = news_history.iloc[i]
    if not factor:
        continue

    # Get the label (the market reaction to the news).
    release_time = self._convert_to_eastern(news_history.index[i])
    next_release_time = self._convert_to_eastern(news_history.index[i+1])
    reaction_period = asset_history[
        (asset_history.index > release_time) &
        (asset_history.index < next_release_time + timedelta(seconds=1))
    ]
    if reaction_period.empty:
        continue
    label = (
        (reaction_period.iloc[-1] - reaction_period.iloc[0])
        / reaction_period.iloc[0]
    )

    # Save the training sample.
    samples.loc[len(samples), :] = [factor, label]

samples = samples.iloc[-100:]

if samples.shape[0] < 10:
    self.liquidate()
    return

# Classify the market reaction into positive/negative/neutral.
# 75% of the most negative labels => class 0 (negative)

```

```

# 75% of the most positive labels => class 2 (positive)
# Remaining labels => class 1 (neutral)
sorted_samples = samples.sort_values(
    by='label', ascending=False
).reset_index(drop=True)
percent_signed = 0.75
positive_cutoff = (
    int(percent_signed
        * len(sorted_samples[sorted_samples.label > 0]))
)
negative_cutoff = (
    len(sorted_samples)
    - int(percent_signed * len(sorted_samples[sorted_samples.label < 0]))
)
sorted_samples.loc[
    list(range(negative_cutoff, len(sorted_samples))), 'label'
] = 0
sorted_samples.loc[
    list(range(positive_cutoff, negative_cutoff)), 'label'
] = 1
sorted_samples.loc[list(range(0, positive_cutoff)), 'label'] = 2

# Load the pre-trained model.
model = TFBertForSequenceClassification.from_pretrained(
    self._model_name, num_labels=3, from_pt=True
)
# Compile the model.
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=3e-5),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
)
# Create the training dataset.
dataset = Dataset.from_pandas(sorted_samples)
dataset = dataset.map(
    lambda sample: self._tokenizer(
        sample['text'], padding='max_length', truncation=True
    )
)
dataset = model.prepare_tf_dataset(
    dataset, shuffle=True, tokenizer=self._tokenizer
)
# Train the model.
model.fit(dataset, epochs=2)

```